Software Requirements Specification

for

DupIssue: Sistem Deteksi Duplikasi Bug/Issue Report Berbasis NLP Semantic Similarity untuk Efisiensi Triase pada Proyek Software

Version 1.0 approved

Prepared by Group 4

Joshua Charis Giovanni Mangialu (187241023)
Valentino Kevin Iversen (187241041)
Elsa Alfika Dyah K. P. (187241007)
Fakhri Reza Abinaya (187241093)

Universitas Airlangga

8 Oktober 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 5
2.3 User Classes and Characteristics 6
2.4 Operating Environment 7
2.5 Design and Implementation Constraints 10
2.6 User Documentation 12
2.7 Assumptions and Dependencies 13
3. External Interface Requirements **15**
3.1 User Interfaces 15
2. Reporter 16
3. Maintener 21
3.2 Hardware Interfaces 28
3.3 Software Interfaces 28
3.4 Communications Interfaces 30
4. System Features **31**
4.1 System Feature Overview 31
4.2 System Feature 1: Submit Issue (REQ-01) 34
4.3 System Feature 2: Lihat Status Issue (REQ-02) 43
4.4 System Feature 3: Mengelola Project & Anggota (REQ-03) 50
4.5 System Feature 4: Mengelola Sumber Issue (REQ-04) 59
4.6 System Feature 5: Review & Validasi Duplikat Issue (REQ-05) 69
4.7 System Feature 6: Mengelola Status Issue (REQ-06) 78
4.8 Fungsi Pendukung (REQ-07) 88
5. Other Nonfunctional Requirements **89**
5.1 Performance Requirements 89
5.2 Safety Requirements 89
5.3 Security Requirements 90
5.4 Software Quality Attributes 91
5.5 Business Rules 92
6. Other Requirements **94**
6.1 Database Requirements 94
6.2 Internationalization 94

6.3 Legal and Ethical Requirements 95
6.4 Reuse and Deployment 95
Appendix A: Glossary **96**
Appendix B: Analysis Models **98**
Appendix C: To Be Determined List **103**

Revision History

Name	Date	Reason For Changes	Version
Kelompok 4	8 Oktober 2026	Draf awal dokumen SRS	Version 1.0

1. Introduction

1.1 Purpose

Dokumen Software Requirements Specification (SRS) ini mendefinisikan kebutuhan perangkat lunak untuk DupIssue: Sistem Deteksi Duplikasi Bug/Issue Report Berbasis NLP Semantic Similarity untuk Efisiensi Triase pada Proyek Software, versi 1.0. Dokumen ini mencakup seluruh sistem DupIssue pada ruang lingkup Minimum Viable Product (MVP). Cakupannya meliputi enam fungsi utama, yaitu Submit Issue, Lihat Status Issue, Mengelola Project & Anggota, Mengelola Sumber Issue, Review & Validasi Duplikat Issue, dan Mengelola Status Issue, ditambah fungsi pendukung seperti autentikasi, dashboard, dan log aktivitas. Dokumen ini juga mencakup tiga komponen arsitektur: aplikasi web (frontend dan backend Laravel), layanan NLP berbasis FastAPI, dan integrasi dengan GitHub REST API. SRS ini menjadi acuan bersama bagi tim pengembang, penguji, dan pihak yang mengevaluasi kesesuaian sistem terhadap kebutuhan yang telah ditetapkan.

1.2 Document Conventions

Dokumen ini ditulis dalam bahasa Indonesia mengikuti template SRS karya Karl E. Wiegers. Istilah asing yang belum baku ditulis dengan huruf miring, sedangkan nama peran (Reporter, Maintainer) dan nama tombol atau menu antarmuka (misalnya Kirim Issue dan Konfirmasi Duplikat) ditulis dengan huruf kapital pada awal kata. Kata "harus" menyatakan kebutuhan yang wajib dipenuhi oleh sistem. Setiap fitur diberi ID unik berformat REQ-XX (REQ-01 sampai REQ-07), dan setiap kebutuhan fungsional rinci diberi ID berformat REQ-XX.Y. Prioritas ditetapkan pada tingkat fitur dengan nilai High atau Medium, dan kebutuhan rinci di dalamnya mewarisi prioritas fitur induknya. Status issue (OPEN, IN_PROGRESS, RESOLVED, CLOSED, DUPLICATE) dan status deteksi (PENDING, PROCESSING, COMPLETED, FAILED) ditulis dengan huruf kapital sesuai Physical Data Model. Tabel dan gambar diberi nomor berurutan.

1.3 Intended Audience and Reading Suggestions

Dokumen ini ditujukan untuk beberapa jenis pembaca:

1. Dosen pengampu dan reviewer menggunakannya untuk menilai kelengkapan dan kesesuaian kebutuhan sistem.
2. Pengembang (developer) menggunakannya sebagai acuan implementasi backend, frontend, layanan NLP, dan basis data.
3. Penguji (tester) menggunakannya untuk menyusun skenario dan kasus uji berdasarkan use case, user story scenario, dan kebutuhan fungsional.
4. Perancang antarmuka menggunakannya sebagai acuan tampilan dan alur interaksi pengguna.
5. Pengguna akhir, yaitu Reporter dan Maintainer, dapat memakainya untuk memahami fungsi yang disediakan sistem.

6. Penulis dokumentasi menggunakannya sebagai dasar panduan pengguna dan dokumentasi teknis.

1.4 Product Scope

DupIssue adalah aplikasi web untuk mengelola bug dan issue report pada proyek perangkat lunak, dilengkapi rekomendasi dan deteksi issue duplikat berbasis NLP semantic similarity. Masalah yang diselesaikan adalah laporan yang sama diajukan berulang kali dengan judul atau deskripsi berbeda. Akibatnya data menjadi redundan, dan Maintainer harus mencari serta membandingkan laporan secara manual saat triase. DupIssue menanganinya dengan menampilkan rekomendasi issue serupa ketika Reporter mengisi formulir, menyimpan kandidat duplikat setelah laporan dikirim, dan menyerahkan keputusan akhir kepada Maintainer melalui mekanisme *human-in-the-loop*.

Tujuan dan manfaat yang ingin dicapai sistem adalah sebagai berikut:

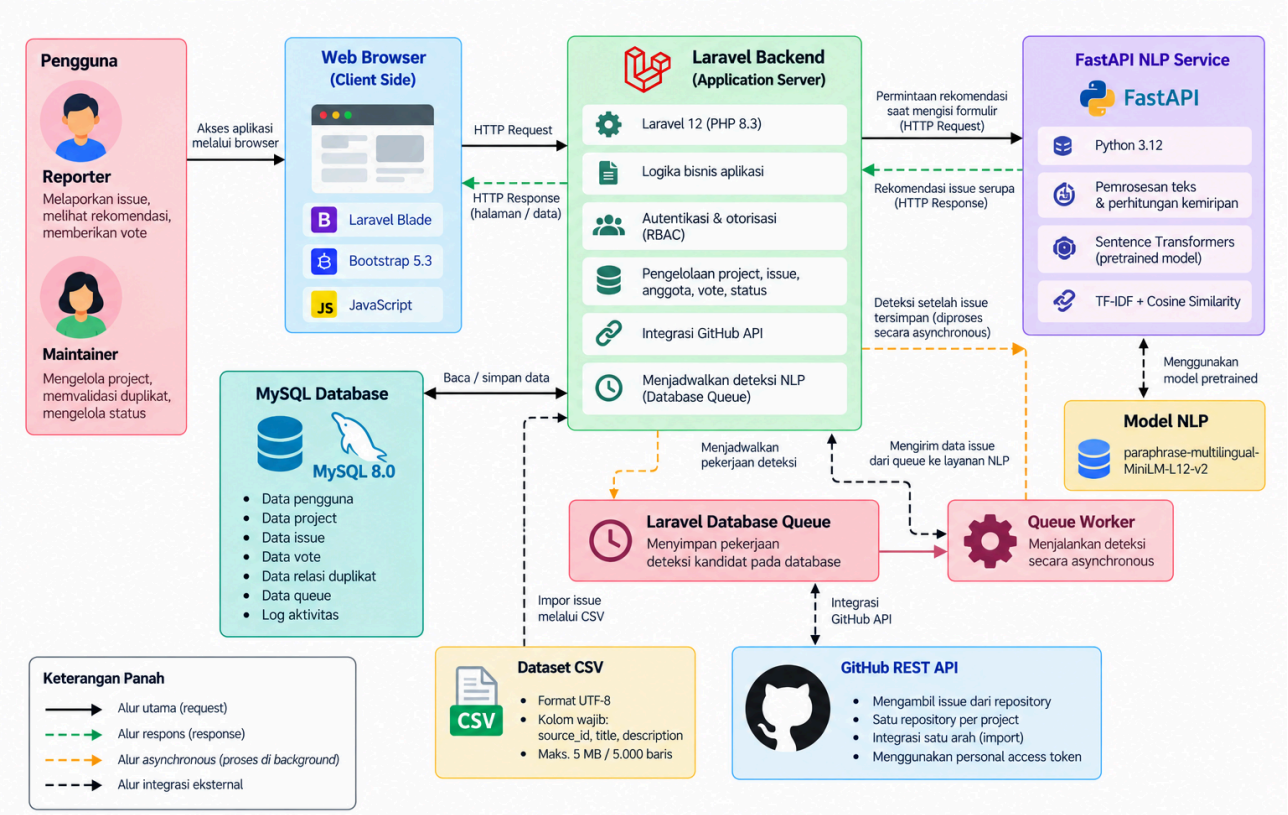
1. Mengurangi kemungkinan Reporter membuat laporan yang sebenarnya sudah tersedia, antara lain dengan menyediakan pilihan vote pada issue yang sama.
2. Membantu Maintainer memeriksa kemungkinan duplikat secara lebih terstruktur melalui skor kemiripan, perbandingan berdampingan, dan penetapan issue induk.
3. Mempermudah pengelolaan dan pemantauan status issue, termasuk penutupan otomatis issue duplikat ketika issue induk dinyatakan Resolved.

Ruang lingkup MVP mencakup tiga sumber data issue (formulir Reporter, dataset CSV, dan repository GitHub), pembatasan data per project, serta dua peran pengguna. Pelatihan model NLP dari awal, analisis lokasi kesalahan pada kode sumber, dan sinkronisasi GitHub dua arah tidak termasuk dalam ruang lingkup.

1.5 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.
2. ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*, 2018.
3. Wiegers, K. E., *Software Requirements Specification Template*, Copyright © 1999.
4. Jiang, Y., Su, X., Treude, C., Shang, C., & Wang, T. (2023). Does deep learning improve the performance of duplicate bug report detection? An empirical study. Journal of Systems and Software, 198, Article 111607. https://www.sciencedirect.com/science/article/abs/pii/S016412122300002X
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. https://aclanthology.org/D19-1410/

2. Overall Description



Gambar 2.1 *Architecture Diagram DupIssue*

2.1 Product Perspective

DupIssue merupakan perangkat lunak baru yang dikembangkan sebagai aplikasi berbasis web untuk membantu pengelolaan bug dan issue report pada proyek pengembangan perangkat lunak. Sistem ini mengintegrasikan fungsi *issue tracking* dengan kemampuan deteksi kemiripan laporan menggunakan pendekatan *Natural Language Processing* (NLP). DupIssue tidak dirancang untuk menggantikan platform issue tracker tertentu, melainkan menjadi solusi tersendiri yang menyediakan proses pelaporan, identifikasi duplikasi, validasi, dan pemantauan status issue dalam satu sistem.

Pengembangan DupIssue dilatarbelakangi oleh permasalahan laporan bug yang berulang dengan judul atau deskripsi berbeda meskipun membahas permasalahan yang sama. Kondisi tersebut mengharuskan Maintainer melakukan pencarian dan perbandingan secara manual untuk mengidentifikasi kemungkinan duplikasi. Oleh karena itu, DupIssue menyediakan rekomendasi

issue serupa ketika Reporter mengisi formulir serta menghasilkan kandidat duplikat setelah issue tersimpan untuk membantu proses peninjauan oleh Maintainer.

Arsitektur DupIssue menggunakan pendekatan *client-server* yang terdiri atas tiga komponen utama, yaitu:

1. Frontend (Client Side), merupakan antarmuka aplikasi yang dibangun menggunakan Laravel Blade, Bootstrap 5.3, dan JavaScript. Komponen ini menyediakan halaman yang dapat diakses Reporter dan Maintainer sesuai hak akses masing-masing. JavaScript digunakan untuk mendukung interaksi dinamis, termasuk menampilkan rekomendasi issue serupa tanpa perlu memuat ulang seluruh halaman.
2. Backend (Application Server), dibangun menggunakan PHP 8.3 dengan framework Laravel 12 dan database MySQL 8.0. Backend bertanggung jawab terhadap autentikasi, pengelolaan project, keanggotaan, issue, vote, sumber data, keputusan duplikasi, perubahan status, dan pencatatan aktivitas. Backend juga mengatur komunikasi dengan layanan NLP dan GitHub REST API. Pemrosesan deteksi setelah penyimpanan issue menggunakan Laravel Database Queue agar proses tersebut dapat dijalankan secara asynchronous.
3. NLP Service, merupakan layanan terpisah yang dibangun menggunakan Python 3.12 dan FastAPI. Layanan ini menjalankan pemrosesan teks dan perhitungan kemiripan menggunakan pustaka Sentence Transformers dengan model pretrained paraphrase-multilingual-MiniLM-L12-v2. Metode TF-IDF dengan cosine similarity digunakan sebagai baseline untuk membandingkan kualitas hasil deteksi. Model pretrained dijalankan menggunakan CPU tanpa memerlukan GPU khusus.

DupIssue mendukung tiga sumber data issue, yaitu laporan langsung dari Reporter, dataset CSV yang diunggah Maintainer, serta issue yang diimpor dari repository GitHub melalui GitHub REST API. Seluruh data disimpan berdasarkan project masing-masing sehingga proses pencarian kemiripan dan identifikasi kandidat duplikat hanya dilakukan terhadap issue dari project yang sama. Hasil NLP digunakan sebagai rekomendasi dan tidak secara otomatis menentukan status duplikasi, sehingga keputusan akhir tetap dilakukan oleh Maintainer melalui proses review dan validasi.

2.2 Product Functions

DupIssue menyediakan enam fungsi utama yang mendukung pengelolaan laporan bug secara terstruktur, mulai dari pelaporan issue hingga validasi duplikasi dan pemantauan penyelesaiannya. Ringkasan fungsi utama sistem dijelaskan pada tabel berikut.

Tabel 2.1 *Product Functions DupIssue*

No.	Fungsi Utama	Deskripsi
1	Submit Issue	Reporter mengisi judul dan deskripsi laporan. Sistem menampilkan rekomendasi issue serupa sehingga Reporter dapat melanjutkan pengiriman laporan baru atau memberikan vote pada issue yang sudah tersedia.
2	Lihat Status Issue	Reporter dapat mencari dan melihat detail serta perkembangan status issue dalam project yang diikuti, termasuk hubungan antara issue duplikat dengan issue induknya.
3	Mengelola Project & Anggota	Maintainer dapat membuat dan mengubah informasi project, mengundang Reporter, membatalkan undangan, dan menghapus anggota project.
4	Mengelola Sumber Issue	Maintainer dapat mengimpor issue melalui dataset CSV atau menghubungkan repository GitHub untuk mengambil data issue ke dalam project.
5	Review & Validasi Duplikat Issue	Maintainer dapat meninjau kandidat duplikat hasil NLP, membandingkan laporan, menentukan status duplikasi, menetapkan issue induk, dan mengoreksi keputusan sebelumnya.
6	Mengelola Status Issue	Maintainer dapat memperbarui status penanganan issue berdasarkan transisi yang diizinkan. Sistem juga mendukung penutupan otomatis issue duplikat ketika issue induk dinyatakan Resolved.

Selain fungsi utama tersebut, DupIssue dilengkapi dengan fungsi pendukung yang meliputi autentikasi pengguna, pembatasan hak akses, dashboard statistik, pencarian dan filter issue, riwayat aktivitas, pemberitahuan dalam aplikasi, serta navigasi relasi antara issue induk dan issue duplikat.

Dalam menjalankan fungsi deteksi, sistem menggunakan dua mekanisme pemrosesan. Rekomendasi sebelum pengiriman laporan diproses melalui permintaan dari aplikasi ke layanan NLP ketika Reporter mengisi formulir. Sementara itu, deteksi kandidat setelah penyimpanan issue dijalankan melalui mekanisme antrean menggunakan Laravel Database Queue. Dengan pendekatan tersebut, laporan yang valid tetap dapat disimpan meskipun pemrosesan NLP belum selesai.

Keseluruhan fungsi tersebut dirancang untuk mengurangi kemungkinan munculnya laporan berulang, membantu Maintainer melakukan validasi duplikasi, dan mempermudah pemantauan siklus penanganan issue.

2.3 User Classes and Characteristics

DupIssue memiliki dua kelas pengguna utama, yaitu Reporter dan Maintainer. Kedua kelas pengguna memiliki karakteristik, tanggung jawab, dan kewenangan berbeda sesuai perannya dalam proses pengelolaan issue.

1. Reporter

   Reporter merupakan pengguna yang bertugas menyampaikan laporan bug atau permasalahan yang ditemukan selama penggunaan maupun pengujian perangkat lunak.

   A. Karakteristik: Reporter dapat berupa anggota tim pengembangan, penguji perangkat lunak, atau pengguna yang terlibat dalam pelaporan masalah. Tingkat pengetahuan teknis Reporter dapat bervariasi, tetapi pengguna diharapkan memahami penggunaan aplikasi web dan mampu mendeskripsikan masalah yang ditemukan.

   B. Akses dan Hak: Reporter dapat mengakses project setelah menerima undangan dari Maintainer dan menjadi anggota aktif. Reporter dapat membuat laporan issue, membaca rekomendasi issue serupa, memberikan atau membatalkan vote, melihat detail issue, dan memantau perkembangan status penanganan. Reporter tidak memiliki kewenangan membuat project, mengelola anggota, mengimpor sumber issue, memvalidasi duplikasi, atau mengubah status penanganan.

   C. Prioritas: Tinggi, karena Reporter merupakan pengguna utama fitur pelaporan dan rekomendasi sebelum submit yang menjadi salah satu fokus pengembangan DupIssue.

2. Maintainer

   Maintainer merupakan pengguna yang bertanggung jawab atas pengelolaan project, pemeriksaan laporan, validasi duplikasi, dan pemantauan proses penanganan issue.

   A. Karakteristik: Maintainer merupakan anggota tim pengembang atau pengelola proyek yang memahami proses pelaporan dan penanganan bug. Pengguna ini diharapkan memiliki kemampuan teknis untuk membandingkan laporan, menilai hubungan duplikasi, dan mengelola status penanganan issue.
   B. Akses dan Hak: Maintainer dapat membuat dan mengubah project miliknya, mengelola undangan dan anggota, mengimpor issue melalui CSV atau GitHub, meninjau kandidat duplikat, menetapkan atau mengoreksi relasi issue induk, memperbarui status issue, serta mengakses dashboard statistik dan log aktivitas project.
   C. Prioritas: Tinggi, karena Maintainer memiliki kewenangan utama dalam menentukan keputusan duplikasi serta memastikan proses pengelolaan dan penanganan issue berjalan sesuai aturan sistem.

Pembagian kewenangan pengguna pada DupIssue menerapkan pendekatan Role-Based Access Control (RBAC). Dalam ruang lingkup Minimum Viable Product (MVP), satu project dikelola oleh satu Maintainer sebagai pemilik dan dapat memiliki beberapa Maintainer dan Reporter sebagai anggota. Modul NLP tidak dikategorikan sebagai kelas pengguna karena merupakan komponen internal yang mendukung proses pengelolaan issue oleh kedua aktor.

2.4 Operating Environment

DupIssue dirancang sebagai aplikasi berbasis web yang dapat dijalankan pada lingkungan pengembangan lokal maupun server pengujian. Sistem menggunakan Laravel sebagai backend utama, MySQL sebagai database, serta Python dan FastAPI sebagai layanan pemrosesan NLP. Pengguna dapat mengakses aplikasi melalui web browser modern tanpa memerlukan instalasi aplikasi khusus pada perangkat pengguna.

Lingkungan operasional ditetapkan dengan mempertimbangkan kebutuhan implementasi MVP, kemampuan perangkat pengembang, dan target penyelesaian proyek dalam waktu dua bulan. Spesifikasi perangkat lunak dan perangkat keras yang direncanakan ditunjukkan pada tabel berikut.

Tabel 2.2. Operating Environment DupIssue

Kategori	Komponen	Spesifikasi
Client Side	Perangkat	Laptop atau komputer desktop
	Web Browser	Google Chrome, Mozilla Firefox, Microsoft Edge, atau Safari versi modern
	Frontend	Laravel Blade, Bootstrap 5.3, JavaScript
Server Side	Sistem Operasi	Ubuntu Server 24.04 LTS
	Backend Framework	Laravel 12
	Backend Runtime	PHP 8.3
	Database	MySQL 8.0
	NLP Framework	FastAPI
	NLP Runtime	Python 3.12
	NLP Library	Sentence Transformers, scikit-learn
	Model NLP	paraphrase-multilingual-MiniLM-L12-v2
	Background Processing	Laravel Database Queue
	Integrasi Eksternal	GitHub REST API
Target Hardware	Processor	4 vCPU
	RAM	8 GB
	Storage	40 GB SSD

	GPU	Tidak diwajibkan
Deployment	Pengembangan	Local Development Environment
	Pengujian	Server lokal atau Ubuntu Server

Pada tahap pengembangan, sistem dapat dijalankan melalui laptop pengembang dengan sistem operasi Windows, macOS, atau Linux selama mendukung komponen perangkat lunak yang diperlukan. Lingkungan pengembangan lokal dapat menggunakan Laravel development server, MySQL, dan FastAPI yang dijalankan secara terpisah pada perangkat yang sama.

Pada sisi server, Laravel menjalankan logika bisnis dan pengelolaan database, sedangkan FastAPI menangani pemrosesan NLP. Laravel Database Queue digunakan untuk menjalankan pekerjaan deteksi kandidat setelah issue tersimpan melalui queue worker yang aktif. Rekomendasi sebelum submit diproses melalui permintaan langsung dari backend ke layanan NLP agar hasilnya dapat ditampilkan pada formulir Reporter.

Seluruh pemrosesan NLP direncanakan menggunakan CPU tanpa GPU khusus. Kapasitas perangkat keras pada tabel merupakan target awal untuk lingkungan pengujian, bukan kebutuhan minimum yang telah divalidasi. Kecukupan spesifikasi tersebut akan diperiksa melalui pengujian performa dan integrasi setelah sistem diimplementasikan.

2.5 Design and Implementation Constraints

Pengembangan DupIssue memiliki sejumlah batasan yang menentukan ruang lingkup fitur, teknologi, dan aturan operasional sistem. Batasan tersebut dijabarkan sebagai berikut :

1. Arsitektur dan Teknologi Pengembangan: Sistem menggunakan Laravel 12 dengan PHP 8.3 dan MySQL 8.0 sebagai backend utama. Antarmuka dikembangkan menggunakan Laravel Blade, Bootstrap 5.3, dan JavaScript tanpa penggunaan framework frontend terpisah seperti React. Pemrosesan NLP dijalankan melalui layanan Python 3.12 dan FastAPI.
2. Pembatasan Hak Akses: Sistem hanya menggunakan dua peran utama, yaitu Reporter dan Maintainer. Hanya Maintainer yang dapat membuat dan mengelola project, sedangkan Reporter baru memperoleh akses setelah menerima undangan dan menjadi anggota aktif. Validasi peran serta keanggotaan dilakukan pada sisi server.
3. Pembatasan Data Berdasarkan Project: Setiap issue terhubung dengan satu project. Pencarian kemiripan, rekomendasi NLP, pengelolaan anggota, dan pembacaan data harus dibatasi berdasarkan project yang dapat diakses pengguna. Sistem tidak melakukan perbandingan issue lintas project.
4. Keterbatasan Model NLP: Sistem menggunakan model pretrained paraphrase-multilingual-MiniLM-L12-v2 tanpa fine-tuning pada MVP. Model dijalankan menggunakan CPU sehingga waktu pemrosesan dipengaruhi oleh kapasitas perangkat keras, panjang teks, dan jumlah data. TF-IDF dengan cosine similarity digunakan sebagai metode pembanding untuk mengevaluasi kualitas deteksi.
5. Pembatasan Keputusan Duplikasi: Skor kemiripan yang dihasilkan layanan NLP hanya digunakan sebagai rekomendasi dan tidak dapat menetapkan status Duplicate secara otomatis. Keputusan akhir tetap dilakukan oleh Maintainer. Sistem juga harus mempertahankan konten dan histori issue meskipun suatu laporan dinyatakan sebagai duplikat.
6. Pembatasan Pemrosesan Asynchronous: Deteksi kandidat setelah penyimpanan issue menggunakan Laravel Database Queue dengan MySQL sebagai penyimpanan pekerjaan antrean. Queue worker harus dijalankan agar pekerjaan dapat diproses. Apabila layanan NLP

mengalami kegagalan, issue yang telah tersimpan tidak boleh dihapus dan Maintainer harus dapat memproses ulang deteksi yang gagal.

7. Pembatasan Impor Data: Sistem menerima dataset CSV dengan encoding UTF-8 dan kolom wajib source_id, title, serta description. Ukuran file dibatasi maksimal 5 MB dan 5.000 baris per proses impor. Sistem harus memvalidasi data dan mencegah penyimpanan issue yang sama akibat impor berulang.
8. Keterbatasan Integrasi GitHub: Integrasi GitHub hanya mencakup pengambilan issue melalui GitHub REST API. Sinkronisasi dua arah, pengelolaan pull request, dan pembaruan otomatis issue yang telah diimpor tidak termasuk dalam MVP. Setiap project hanya memiliki satu konfigurasi repository GitHub.
9. Pembatasan Status dan Relasi Issue: Perubahan status harus mengikuti transisi yang ditetapkan sistem. Setiap issue duplikat hanya dapat terhubung dengan satu issue induk dalam project yang sama dan tidak boleh membentuk relasi bersiklus. Perubahan status Duplicate harus dilakukan melalui proses validasi oleh Maintainer.
10. Keamanan Data dan Kredensial: Sistem harus memeriksa autentikasi dan otorisasi pada setiap operasi yang dilindungi. Kredensial GitHub harus disimpan secara terenkripsi dan tidak dicatat dalam log aktivitas. Perubahan data utama dan riwayat aktivitas harus dijaga konsistensinya.
11. Batasan Deployment dan Infrastruktur: Pengembangan awal dilakukan pada lingkungan lokal tanpa mewajibkan penggunaan Docker, Redis, GPU, atau layanan cloud berbayar. Sistem ditargetkan dapat dijalankan pada satu server pengujian dengan spesifikasi yang telah direncanakan. Penyesuaian infrastruktur dapat dilakukan berdasarkan hasil pengujian.
12. Batasan Ruang Lingkup MVP: Pengembangan difokuskan pada enam use case utama beserta fitur pendukung yang telah ditentukan. Pelatihan model NLP dari awal, analisis lokasi kesalahan kode sumber, editor kode, dan sinkronisasi GitHub dua arah tidak termasuk dalam ruang lingkup implementasi awal.

Batasan-batasan tersebut ditetapkan untuk menjaga pengembangan DupIssue tetap terarah pada kebutuhan utama, yaitu membantu identifikasi issue berulang dan mendukung proses validasi serta pengelolaan laporan secara terstruktur.

2.6 User Documentation

Dalam mendukung penggunaan, instalasi, dan pemeliharaan DupIssue, dokumentasi sistem direncanakan tersedia dalam bentuk berikut :

1. Panduan Pengguna (User Guide)

   Dokumentasi ini ditujukan kepada Reporter dan Maintainer untuk menjelaskan penggunaan fitur yang tersedia pada aplikasi. Panduan Reporter mencakup proses mengakses project, mengirim issue, membaca rekomendasi, memberikan vote, dan memantau status laporan. Panduan Maintainer mencakup pengelolaan project dan anggota, impor sumber issue, validasi duplikasi, perubahan status penanganan, serta penggunaan dashboard statistik.

2. Dokumentasi Teknis dan Instalasi (System Documentation)

   Dokumentasi ini ditujukan kepada pengembang dan pemelihara sistem. Isinya meliputi penjelasan arsitektur DupIssue, struktur database MySQL, konfigurasi Laravel dan FastAPI, pengunduhan model pretrained, serta integrasi GitHub REST API. Dokumentasi juga menjelaskan prosedur menjalankan backend, layanan NLP, dan Laravel Database Queue worker agar seluruh komponen dapat beroperasi secara terintegrasi pada lingkungan lokal maupun server pengujian.

3. Dokumentasi Pengujian (Testing Documentation)

   Dokumentasi ini memuat skenario pengujian berdasarkan use case, hasil pengujian fungsional, pengujian integrasi, evaluasi model NLP, pengujian performa, serta pengujian usability. Setiap pengujian didokumentasikan dengan mencatat kondisi pengujian, hasil yang diharapkan, hasil aktual, dan status keberhasilan untuk mengevaluasi kesesuaian implementasi terhadap kebutuhan sistem.

Seluruh dokumentasi direncanakan tersedia dalam format digital dan dilengkapi diagram, tangkapan layar, atau contoh penggunaan apabila diperlukan. Dokumentasi tersebut menjadi panduan bagi tim pengembang maupun pihak lain yang melakukan evaluasi terhadap sistem DupIssue.

2.7 Assumptions and Dependencies

Pengembangan dan penggunaan DupIssue didasarkan pada sejumlah asumsi serta ketergantungan yang dapat memengaruhi keberhasilan implementasi dan operasional sistem.

A. Asumsi (*Assumptions*)

1. Ketersediaan Perangkat dan Jaringan : Pengguna diasumsikan memiliki perangkat dengan browser modern dan koneksi jaringan yang memadai untuk mengakses server DupIssue. Pengembangan dapat dilakukan menggunakan laptop mahasiswa selama komponen perangkat lunak yang diperlukan dapat dijalankan.
2. Ketersediaan Akun Pengguna : Reporter dan Maintainer diasumsikan menggunakan akun yang telah disediakan melalui konfigurasi awal sistem. Reporter dapat mengakses project setelah menerima undangan dari Maintainer dan memiliki keanggotaan aktif.
3. Ketersediaan Data Issue : Setiap laporan diasumsikan memiliki judul dan deskripsi yang cukup untuk diproses menggunakan NLP. Kualitas informasi, termasuk penggunaan istilah teknis dan kelengkapan deskripsi, dapat memengaruhi relevansi rekomendasi yang dihasilkan.
4. Kesesuaian Sumber Data : Dataset CSV yang digunakan diasumsikan dapat disesuaikan dengan format impor DupIssue. Untuk integrasi GitHub, Maintainer diasumsikan memiliki akses repository dan kredensial yang diperlukan untuk mengambil data issue.
5. Ketersediaan Lingkungan Pengembangan : Tim pengembang diasumsikan memiliki perangkat yang mendukung PHP, Laravel, MySQL, Python, dan FastAPI. Pengembangan dilakukan menggunakan lingkungan lokal sebelum seluruh layanan diintegrasikan pada server pengujian.
6. Validasi oleh Maintainer : Maintainer diasumsikan memahami permasalahan pada project yang dikelola sehingga dapat menilai kandidat hasil NLP dan menentukan hubungan duplikasi secara tepat.

B. Ketergantungan *(Dependencies*)

1. Backend Laravel dan Database MySQL : Operasional utama DupIssue bergantung pada ketersediaan backend Laravel dan database MySQL. Komponen tersebut diperlukan untuk autentikasi, pengelolaan project, penyimpanan issue, pengelolaan anggota, pencatatan vote, dan perubahan status.
2. Layanan NLP dan Model Pretrained : Fungsi rekomendasi dan deteksi kandidat bergantung pada layanan FastAPI, pustaka Sentence Transformers, serta model paraphrase-multilingual-MiniLM-L12-v2. Model harus tersedia pada lingkungan pemrosesan sebelum fitur NLP dapat digunakan. Kegagalan layanan NLP tidak boleh menghapus issue yang telah disimpan.
3. Laravel Database Queue : Deteksi kandidat setelah penyimpanan issue bergantung pada mekanisme antrean berbasis MySQL dan queue worker Laravel yang berjalan aktif. Apabila queue worker berhenti, pekerjaan deteksi dapat tertunda hingga pemrosesan dilanjutkan.
4. GitHub REST API : Proses impor issue dari GitHub bergantung pada ketersediaan API, koneksi jaringan, izin akses repository, dan validitas kredensial. Gangguan pada GitHub tidak boleh menghentikan fungsi pengelolaan issue lokal maupun impor melalui CSV.
5. Konfigurasi Lingkungan Server : Seluruh komponen sistem bergantung pada konfigurasi runtime dan jaringan yang sesuai agar Laravel, MySQL, dan FastAPI dapat saling berkomunikasi. Kapasitas CPU, RAM, dan storage juga memengaruhi kemampuan sistem dalam menjalankan proses aplikasi dan NLP secara bersamaan.
6. Kualitas Dataset dan Evaluasi NLP : Hasil perbandingan model bergantung pada ketersediaan dataset bug report yang sesuai dan memiliki informasi label duplikasi untuk evaluasi. Kualitas dataset serta karakteristik teks dapat memengaruhi hasil pengukuran performa metode TF-IDF dan sentence embedding.

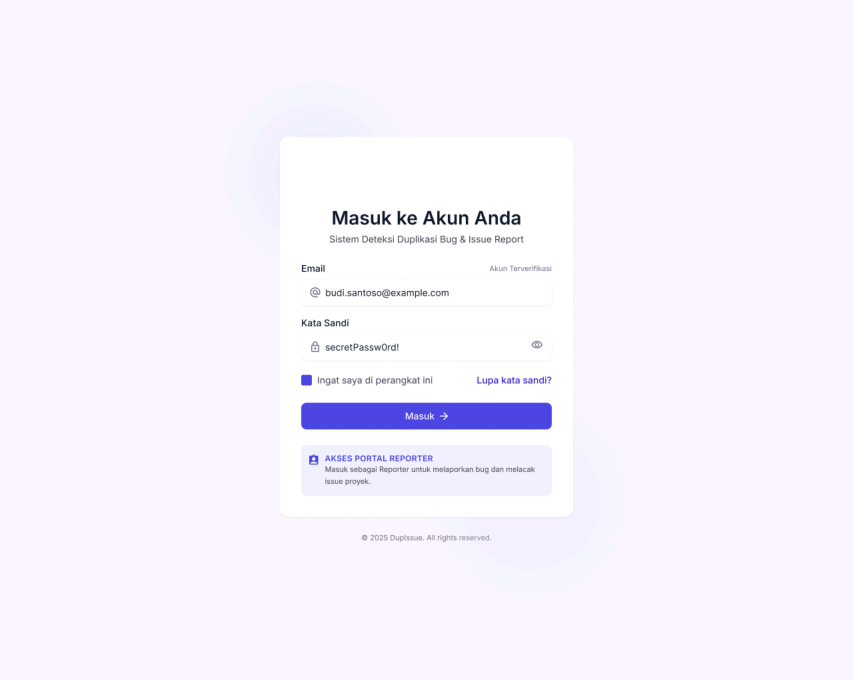
Asumsi dan ketergantungan tersebut perlu diperhatikan selama proses pengembangan dan pengujian sistem. Perubahan pada lingkungan operasional, layanan eksternal, kualitas data, atau konfigurasi komponen dapat memengaruhi fungsionalitas dan performa DupIssue sehingga memerlukan penyesuaian implementasi sesuai kebutuhan proyek.

3. External Interface Requirements

3.1 User Interfaces

Antarmuka DupIssue dirancang sebagai aplikasi web dengan pola tata letak yang konsisten pada seluruh halaman. Setiap halaman setelah login memiliki bilah navigasi di sisi kiri, bilah atas berisi *breadcrumb*, ikon notifikasi, dan profil pengguna, serta area konten utama di sebelah kanan. Antarmuka dibagi berdasarkan peran. Portal Reporter berfokus pada pelaporan dan pemantauan issue, sedangkan portal Maintainer berfokus pada pengelolaan project, impor data, validasi duplikat, dan pemantauan statistik. Warna ungu digunakan sebagai penanda aksi utama dan elemen yang berkaitan dengan deteksi NLP, sedangkan lencana status memakai warna berbeda agar mudah dibedakan. Bagian berikut menjelaskan fungsi, komponen, dan perilaku tiap antarmuka.

1. **Login**

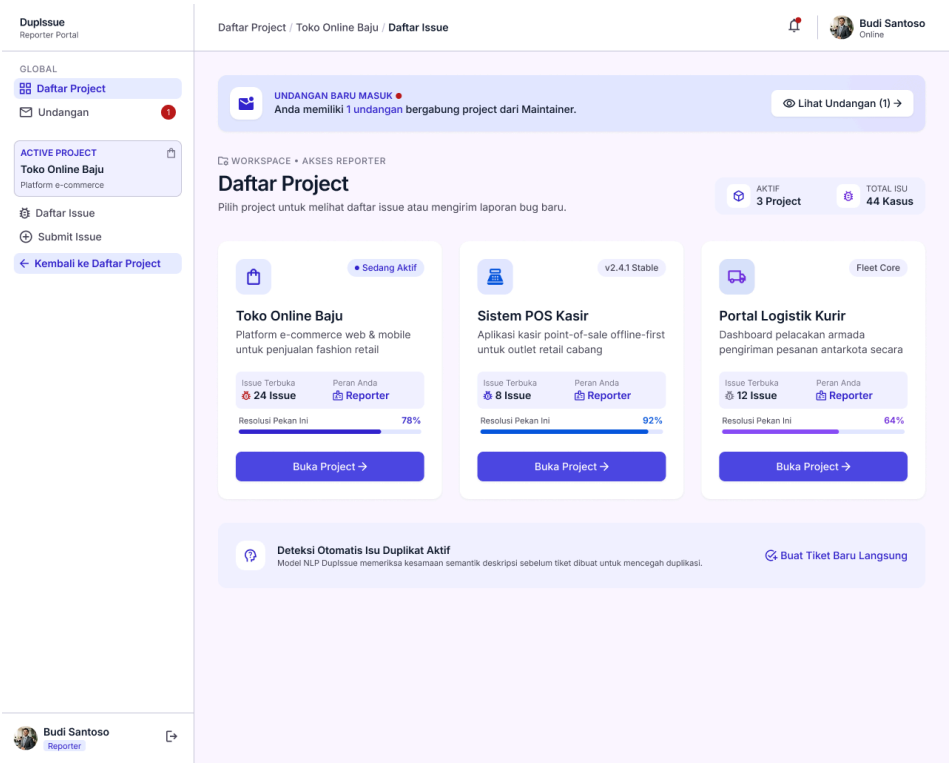


Halaman Login menampilkan kartu "Masuk ke Akun Anda" dengan subjudul Sistem Deteksi Duplikasi Bug & Issue Report. Pengguna mengisi kolom Email dan Kata Sandi, dengan ikon mata untuk menampilkan atau menyembunyikan kata sandi. Tersedia juga kotak centang "Ingat saya di perangkat ini", tautan "Lupa kata sandi?", dan tombol Masuk. Di bawah

tombol terdapat kotak informasi "Akses Portal Reporter" yang menjelaskan bahwa masuk sebagai Reporter dipakai untuk melaporkan bug dan melacak issue proyek. Setelah autentikasi berhasil, sistem mengarahkan pengguna ke portal sesuai perannya. Reporter menuju Daftar Project, sedangkan Maintainer menuju dashboard project miliknya.

2. Reporter

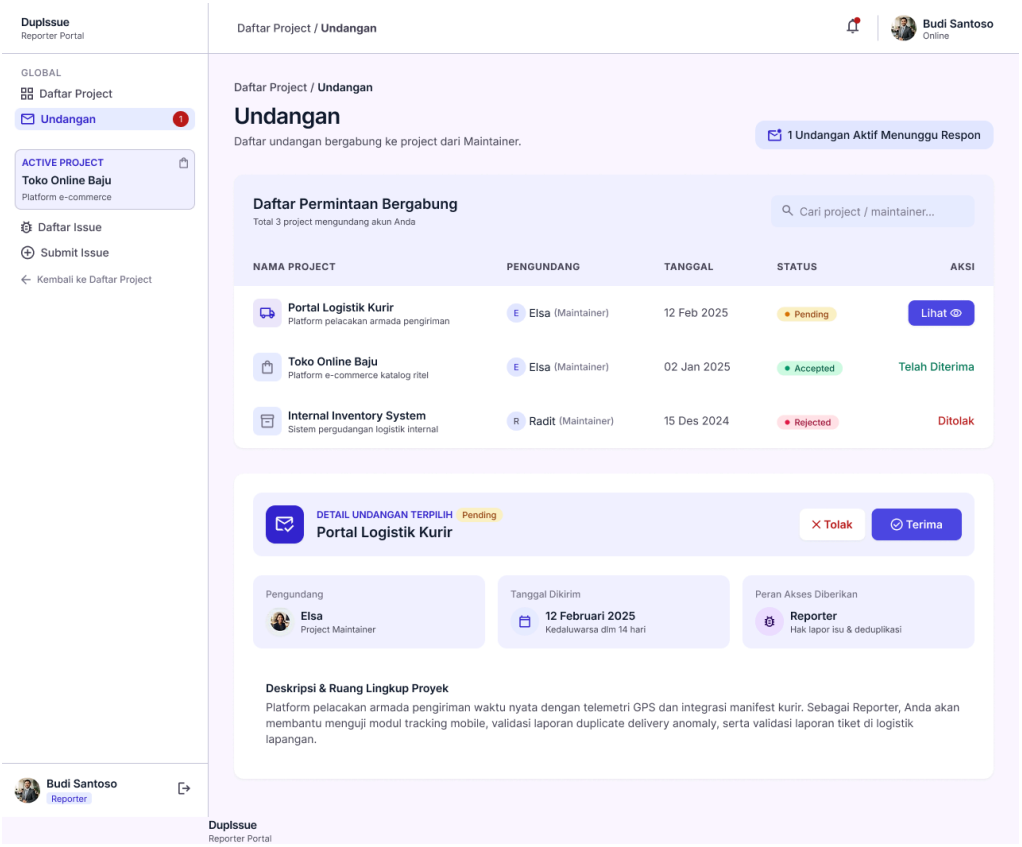
- Daftar Project



Halaman ini adalah beranda Reporter setelah login. Bagian atas menampilkan spanduk "Undangan Baru Masuk" beserta tombol Lihat Undangan, dan di kanan judul terdapat ringkasan jumlah project aktif serta total issue. Setiap project ditampilkan sebagai kartu berisi nama, deskripsi singkat, jumlah issue terbuka, peran pengguna (Reporter), indikator resolusi pekan ini, dan tombol Buka Project. Di bagian bawah terdapat informasi bahwa deteksi otomatis issue duplikat aktif, disertai tautan untuk langsung membuat laporan baru. Bilah samping memuat menu global (Daftar Project

dan Undangan dengan lencana jumlah undangan) serta panel project aktif yang berisi pintasan ke Daftar Issue dan Submit Issue. Dengan tampilan ini, Reporter hanya melihat project tempat ia menjadi anggota aktif.

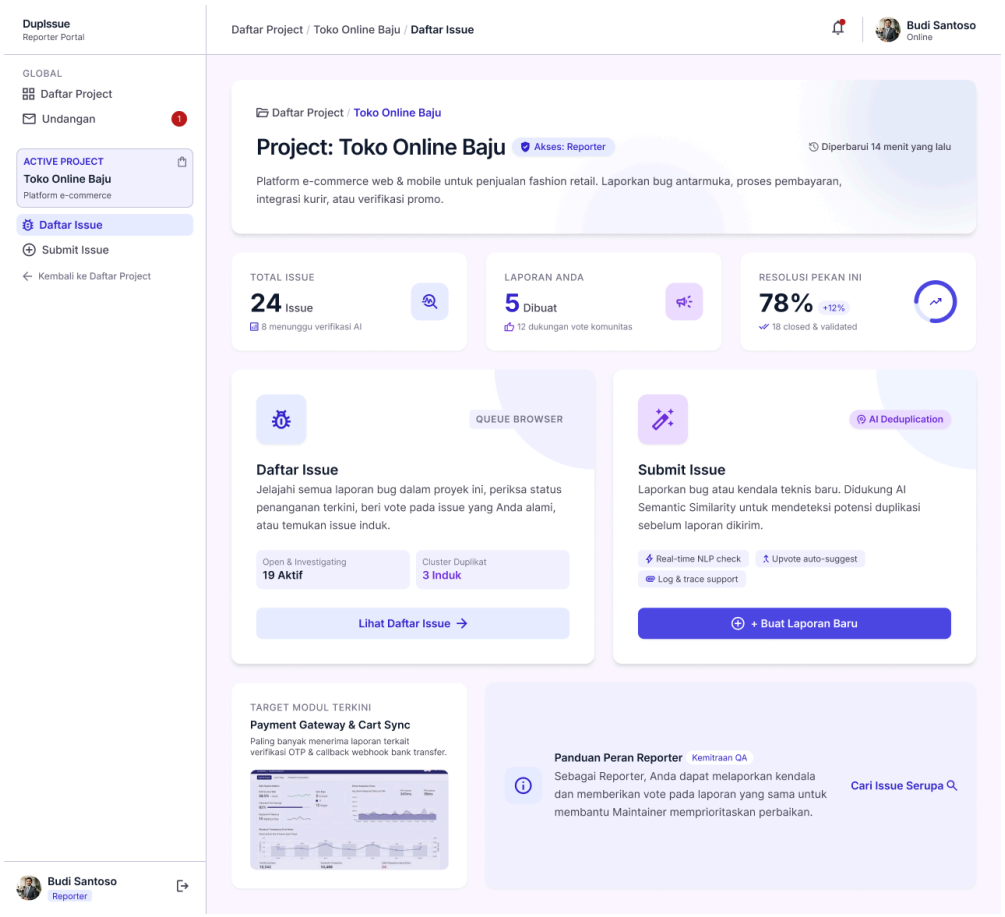
- **Undangan dan Detail Undangan**



Halaman ini membantu Reporter menanggapi undangan bergabung ke project. Bagian atas berupa tabel Daftar Permintaan Bergabung dengan kolom nama project, pengundang, tanggal, status (Pending, Accepted, Rejected), dan aksi, serta kolom pencarian berdasarkan nama project atau Maintainer. Undangan berstatus Pending memiliki tombol Lihat, sedangkan yang sudah ditanggapi hanya menampilkan keterangan "Telah Diterima" atau "Ditolak". Saat sebuah undangan dipilih, panel detail di bawahnya menampilkan nama project, pengundang, tanggal kirim, peran yang diberikan (Reporter), dan deskripsi serta ruang lingkup project. Panel ini dilengkapi

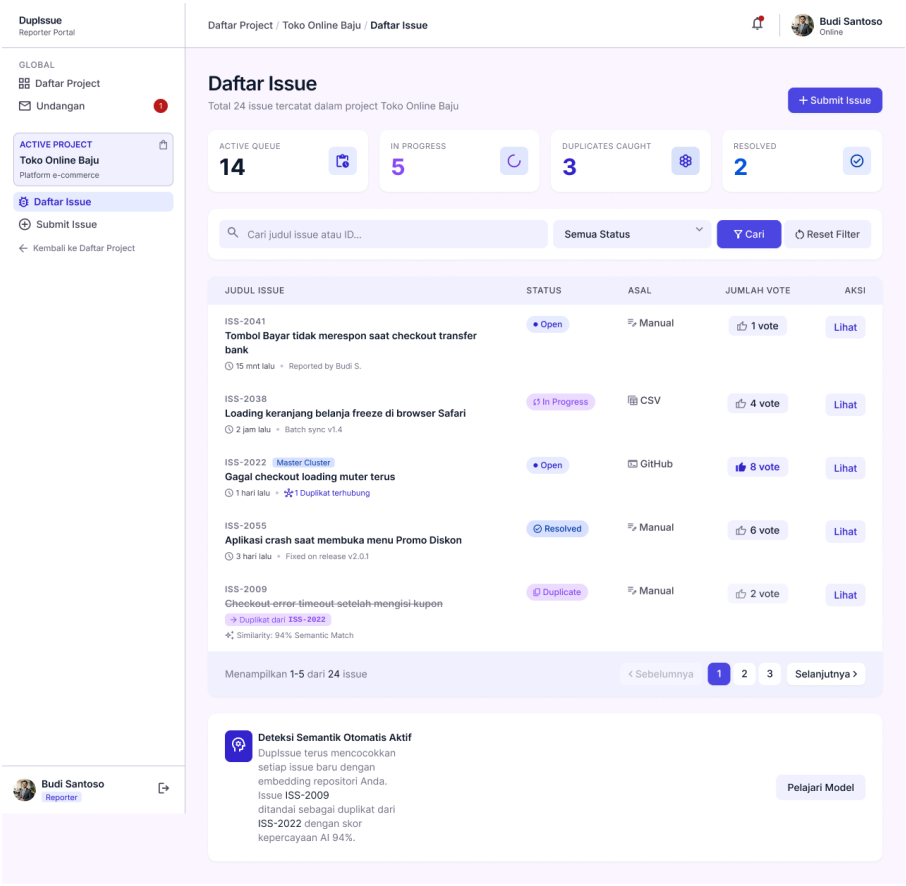
tombol Terima dan Tolak. Reporter baru mendapatkan akses ke project setelah menerima undangan.

- **Daftar Issues**



Halaman ini berfungsi sebagai pusat navigasi satu project. Bagian atas menampilkan nama project, lencana "Akses: Reporter", dan waktu pembaruan terakhir. Tiga kartu ringkasan menampilkan total issue, jumlah laporan milik Reporter beserta dukungan vote yang diterima, dan persentase resolusi pekan ini. Dua kartu aksi utama memandu pengguna, yaitu kartu Daftar Issue untuk menelusuri laporan dan kartu Submit Issue untuk membuat laporan baru dengan dukungan deteksi semantik. Terdapat pula panel informasi modul yang sedang banyak dilaporkan dan panduan peran Reporter dengan tautan Cari Issue Serupa.

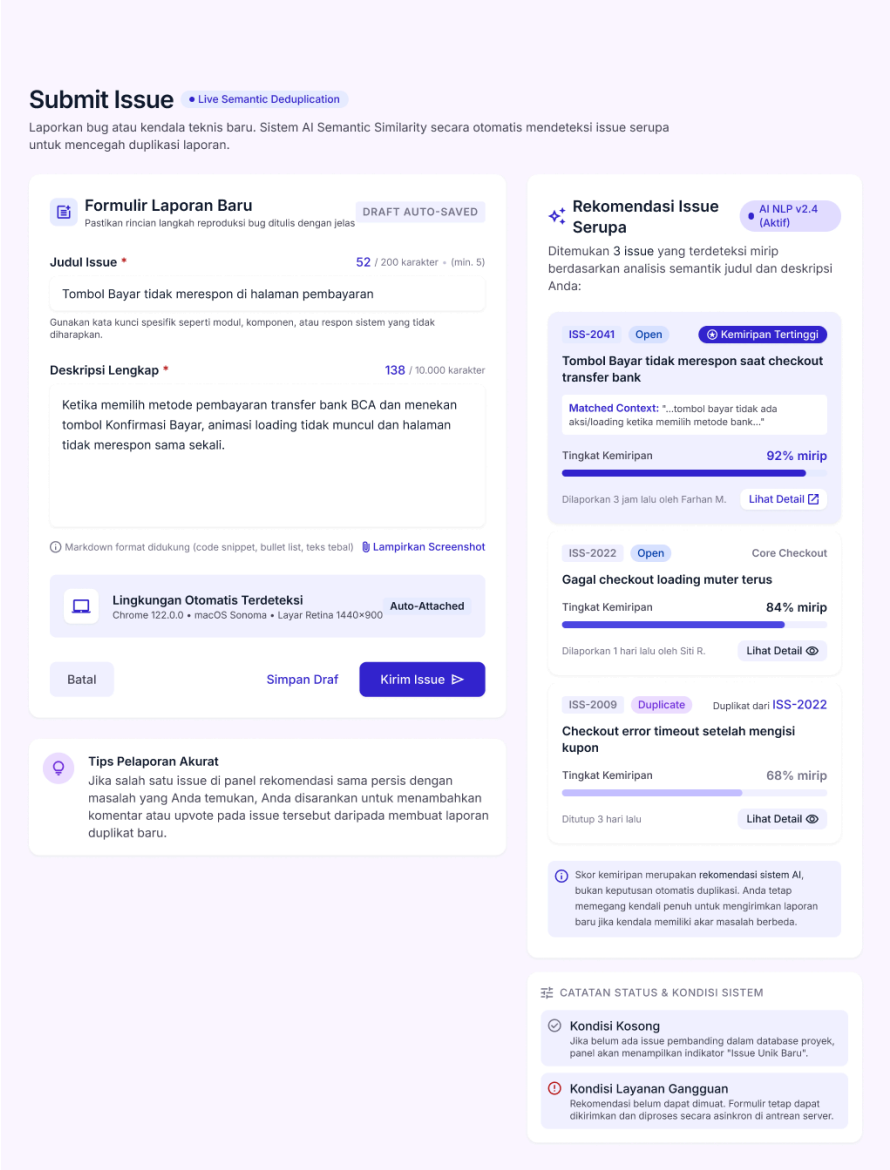
- **Lihat Daftar Issue Project**



Halaman ini menampilkan seluruh issue dalam satu project untuk dipantau Reporter. Bagian atas berisi empat penghitung ringkas (antrean aktif, In Progress, duplikat tertangkap, dan Resolved), disusul kolom pencarian berdasarkan judul atau ID, filter status, tombol Cari, dan Reset Filter. Tabel issue menampilkan ID, judul, lencana status, asal data (Manual, CSV, atau GitHub), jumlah vote, dan tombol Lihat. Issue induk diberi label "Master Cluster" beserta jumlah duplikat terhubung, sedangkan issue duplikat ditampilkan dengan judul tercoret dan keterangan "Duplikat dari" nomor issue induknya. Tersedia penomoran halaman di bawah tabel, dan tombol

Submit Issue berada di pojok kanan atas. Halaman ini bersifat baca saja, sehingga Reporter tidak dapat mengubah status issue.

- Submit Issue dan Rekomendasi NLP

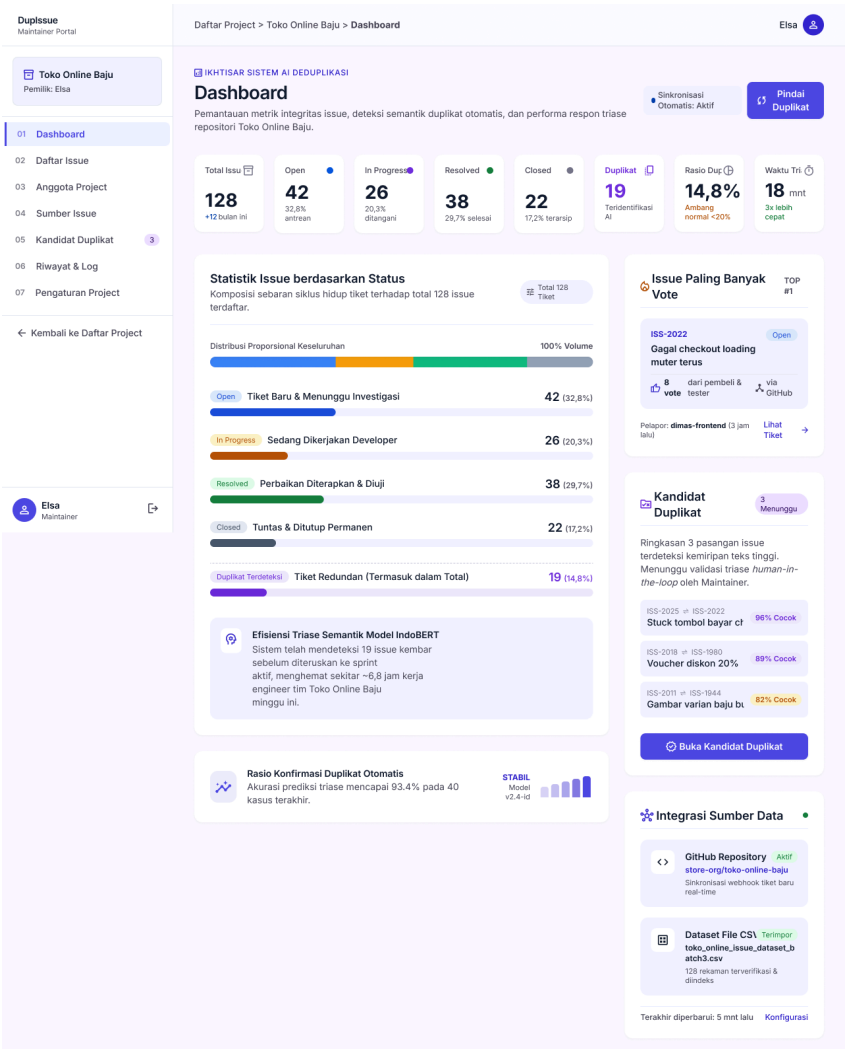


Halaman ini terbagi dua panel. Panel kiri adalah Formulir Laporan Baru dengan kolom Judul Issue dan Deskripsi Lengkap. Penghitung karakter ditampilkan di tiap kolom, sesuai batas judul 5–200 karakter dan deskripsi 10–10.000 karakter. Panel ini juga memuat tombol Batal dan Kirim Issue. Panel kanan, Rekomendasi Issue Serupa, menampilkan hingga tiga issue dari

project yang sama. Setiap kartu berisi ID, status, judul, persentase kemiripan dalam bentuk bar, serta tombol Lihat Detail. Rekomendasi diperbarui ketika Reporter mengetik, dan keterangan di bawahnya menegaskan bahwa skor adalah rekomendasi, bukan keputusan duplikat, sehingga Reporter tetap bebas mengirim laporan. Bagian "Catatan Status & Kondisi Sistem" menjelaskan dua kondisi khusus. Jika belum ada issue pembanding, panel menampilkan indikator "Issue Unik Baru". Jika layanan NLP gangguan, panel menampilkan pesan bahwa rekomendasi belum dapat dimuat, tetapi formulir tetap dapat dikirim dan diproses secara asinkron di antrean.

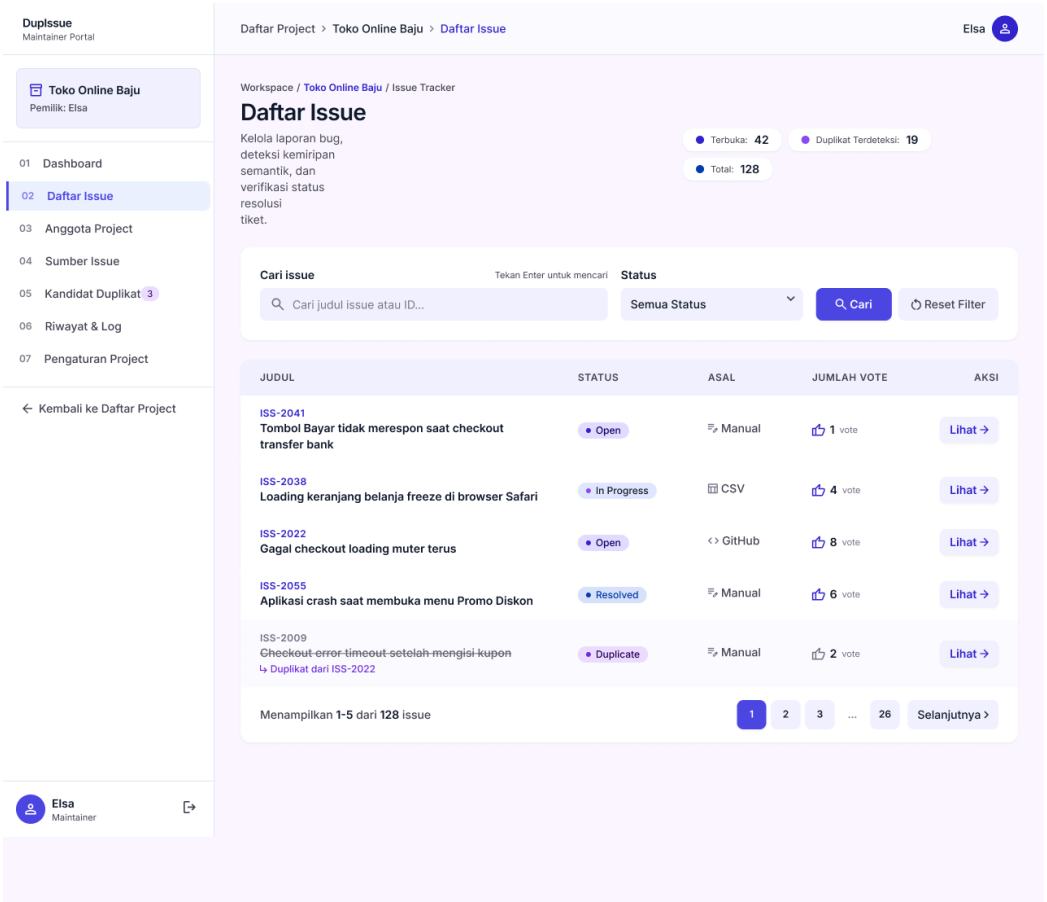
3. Maintener

- **Dashboard**



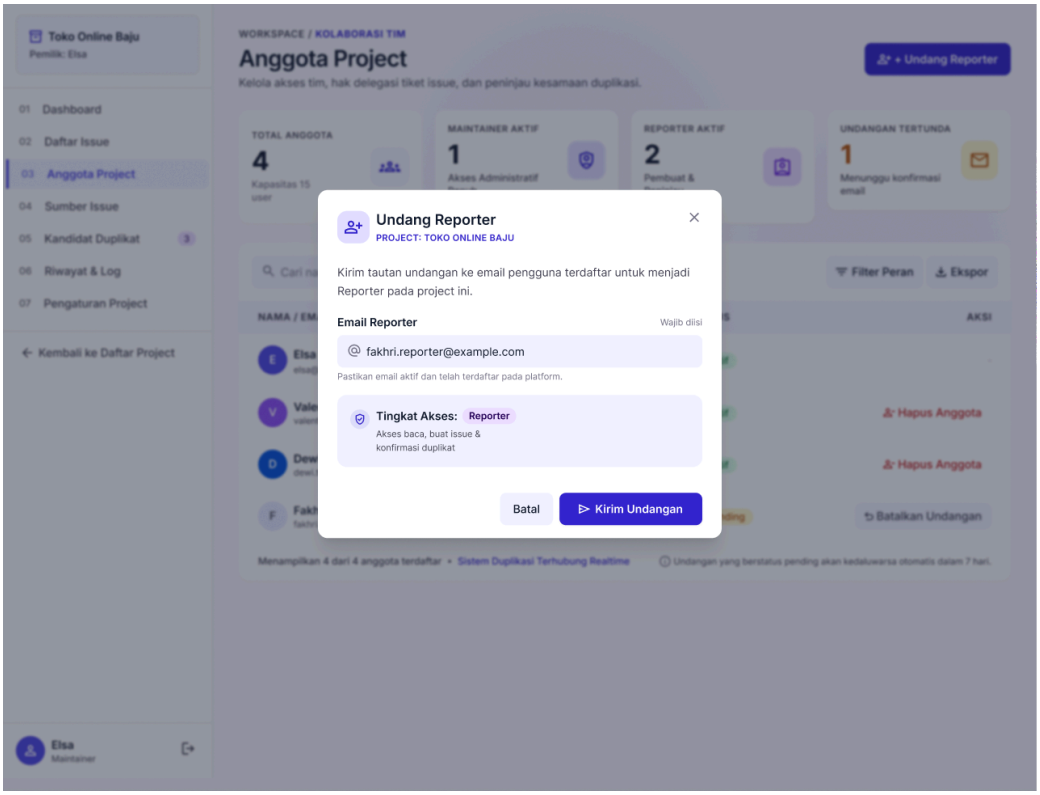
Halaman ini memberi gambaran menyeluruh kondisi project bagi Maintainer. Bagian atas menampilkan deretan metrik: total issue, jumlah per status (Open, In Progress, Resolved, Closed), jumlah duplikat, rasio duplikasi, dan waktu triase. Di bawahnya, panel Statistik Issue berdasarkan Status menampilkan distribusi proporsional tiap status dalam bentuk bar. Kolom kanan memuat issue dengan vote terbanyak, ringkasan kandidat duplikat yang menunggu keputusan beserta tombol Buka Kandidat Duplikat, dan panel Integrasi Sumber Data untuk repository GitHub dan dataset CSV. Menu samping bernomor 01–07 memudahkan perpindahan ke Daftar Issue, Anggota Project, Sumber Issue, Kandidat Duplikat, Riwayat & Log, dan Pengaturan Project. Lencana jumlah pada menu Kandidat Duplikat menandai pekerjaan review yang belum selesai.

- **Daftar Issue**



Halaman ini menampilkan seluruh issue project dengan struktur serupa tampilan Reporter. Bagian atas menampilkan jumlah issue terbuka, duplikat terdeteksi, dan total issue. Pengguna dapat mencari berdasarkan judul atau ID dan memfilter berdasarkan status. Tabel memuat judul, status, asal data, jumlah vote, dan tombol Lihat, sedangkan issue duplikat ditandai dengan judul tercoret beserta tautan ke induknya. Dari tombol Lihat, Maintainer dapat membuka detail issue untuk mengubah status atau meninjau relasi duplikat.

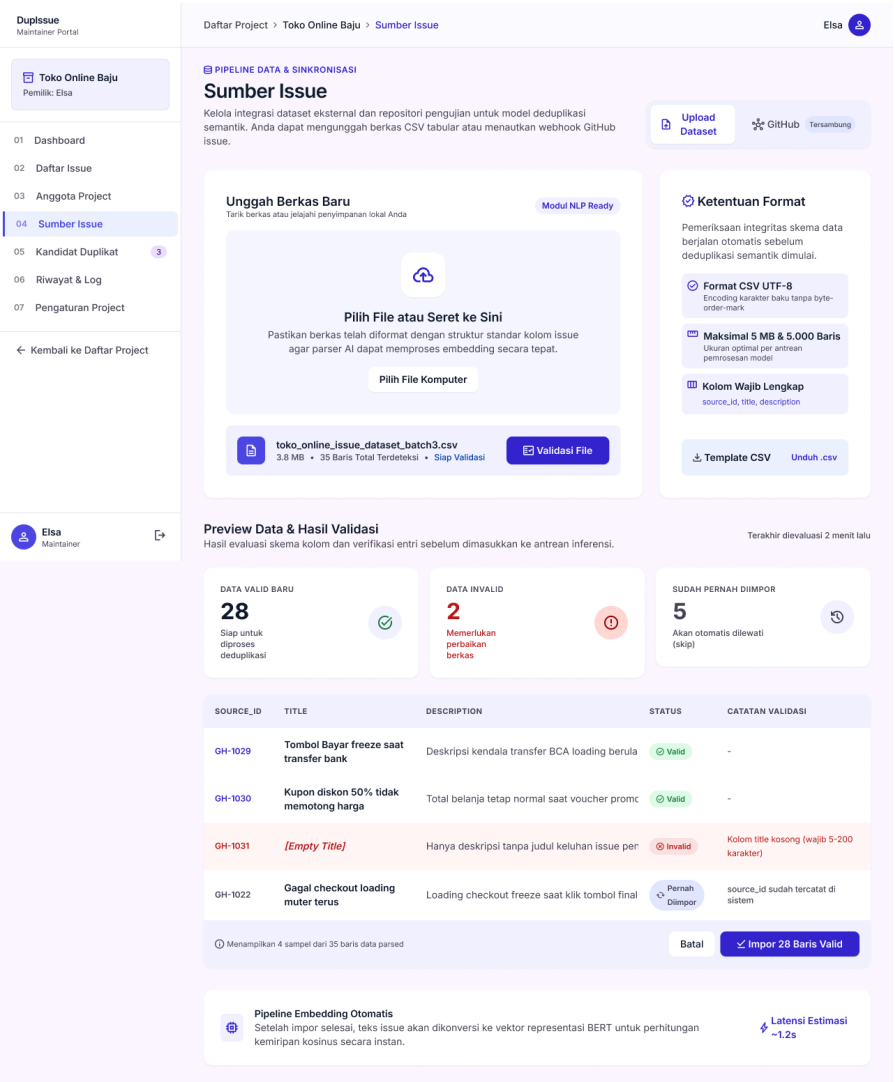
- **Anggota Project**



Halaman ini digunakan Maintainer untuk mengelola akses Reporter. Bagian atas menampilkan ringkasan total anggota, Maintainer aktif, Reporter aktif, dan undangan tertunda, diikuti tabel anggota dengan nama, email, peran, status, dan aksi. Anggota aktif memiliki aksi Hapus Anggota, sedangkan undangan Pending memiliki aksi Batalkan Undangan. Tombol Undang Reporter membuka jendela modal berisi kolom Email Reporter,

keterangan tingkat akses, serta tombol Batal dan Kirim Undangan. Sistem memeriksa bahwa email sudah terdaftar dan belum menjadi anggota atau memiliki undangan Pending.

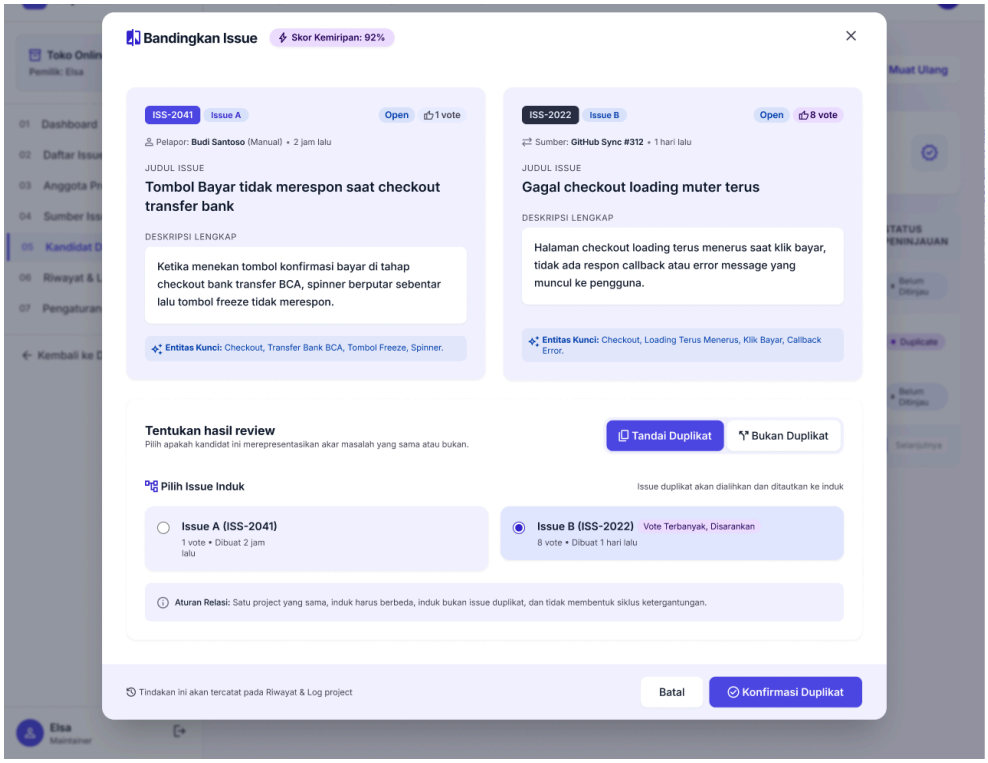
- **Sumber Issue**



Halaman ini memiliki dua tab, Upload Dataset dan GitHub. Pada tab Upload Dataset, Maintainer memilih atau menyeret berkas CSV ke area unggah, lalu menekan Validasi File. Panel Ketentuan Format menjelaskan syarat berkas, yaitu CSV UTF-8, maksimal 5 MB dan 5.000 baris, dengan kolom wajib source_id, title, dan description, serta tautan unduh template CSV. Hasil validasi ditampilkan sebagai tiga penghitung (data valid baru, data invalid, dan sudah pernah diimpor), diikuti tabel pratinjau yang memberi status dan catatan validasi per baris. Hanya baris valid yang dapat diimpor

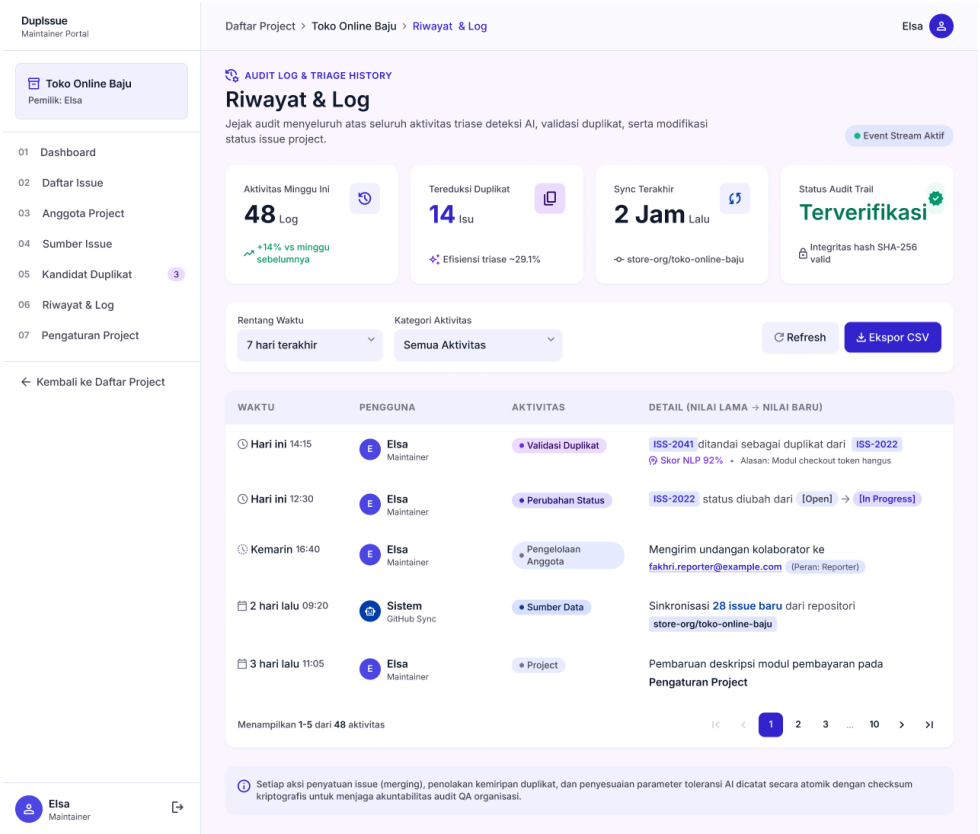
melalui tombol Impor Baris Valid, sedangkan baris invalid dan duplikat sumber dilewati. Tombol Batal menutup pratinjau tanpa menyimpan data. Tab GitHub menyediakan pengisian repository dan token akses, pengujian koneksi, dan pengambilan issue.

- **Kandidat Duplikat**



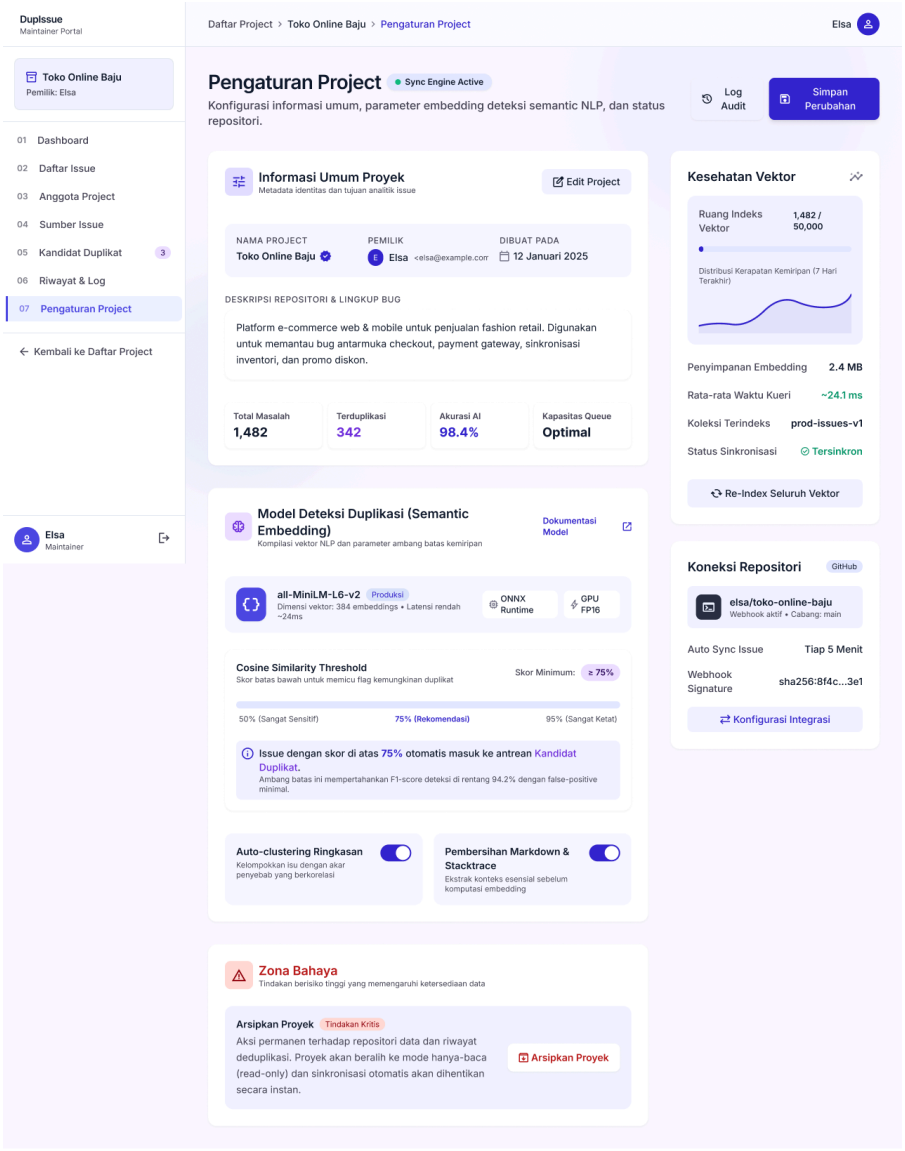
Antarmuka ini berupa modal "Bandingkan Issue" yang menampilkan skor kemiripan di bagian atas. Dua issue ditampilkan berdampingan, masing-masing dengan ID, status, jumlah vote, pelapor atau sumber, waktu pelaporan, judul, deskripsi lengkap, dan entitas kunci. Dengan begitu Maintainer dapat membandingkan isi kedua laporan sekaligus. Di bagian bawah, Maintainer memilih Tandai Duplikat atau Bukan Duplikat. Jika memilih Tandai Duplikat, ia menentukan issue induk melalui pilihan radio, dan sistem menyarankan issue dengan vote terbanyak. Kotak aturan relasi mengingatkan bahwa kedua issue harus berada dalam satu project, induk harus berbeda dari duplikat, induk tidak boleh berstatus duplikat, dan relasi tidak boleh membentuk siklus. Keputusan disimpan setelah Maintainer menekan Konfirmasi Duplikat dan dicatat pada Riwayat & Log.

- **Riwayat dan Log**



Halaman ini menampilkan jejak aktivitas project untuk keperluan audit. Bagian atas menampilkan ringkasan aktivitas minggu ini, jumlah duplikat tereduksi, waktu sinkronisasi terakhir, dan status audit trail. Maintainer dapat memfilter log berdasarkan rentang waktu dan kategori aktivitas, memuat ulang data, atau mengekspor ke CSV. Tabel log memuat waktu, pengguna pelaksana, kategori aktivitas (misalnya Validasi Duplikat, Perubahan Status, Pengelolaan Anggota, dan Sumber Data), serta detail perubahan dari nilai lama ke nilai baru. Halaman ini hanya menampilkan data dan tidak menyediakan aksi pengubahan.

- **Pengaturan Project**



-

Halaman ini menampilkan informasi dan konfigurasi project. Panel Informasi Umum Proyek memuat nama, pemilik, tanggal pembuatan, dan deskripsi, dengan tombol Edit Project untuk mengubah nama atau deskripsi. Panel Model Deteksi Duplikasi menampilkan model yang digunakan dan pengaturan ambang skor cosine similarity, yaitu batas minimum agar pasangan issue masuk ke daftar kandidat. Di kolom kanan terdapat ringkasan kondisi indeks vektor dan koneksi repository GitHub beserta tombol Konfigurasi Integrasi. Bagian Zona Bahaya memuat aksi berisiko tinggi,

yaitu mengarsipkan project. Tombol Simpan Perubahan di bagian atas menyimpan perubahan pengaturan.

3.2 Hardware Interfaces

DupIssue merupakan aplikasi berbasis web yang tidak memerlukan antarmuka perangkat keras khusus untuk menjalankan fungsionalitasnya. Pada sisi klien (*client side*), pengguna berinteraksi dengan sistem melalui perangkat standar seperti laptop atau komputer desktop menggunakan web browser modern. Interaksi dilakukan menggunakan perangkat input berupa keyboard dan mouse atau touchpad, sedangkan hasil pengolahan data dan informasi sistem ditampilkan melalui monitor.

Pada sisi server (*server side*), DupIssue berjalan pada perangkat yang mendukung sistem operasi Ubuntu Server 24.04 LTS serta komponen Laravel, MySQL, dan FastAPI. Lingkungan server pengujian ditargetkan memiliki processor 4 vCPU, RAM 8 GB, dan storage 40 GB SSD. Pemrosesan NLP direncanakan menggunakan CPU tanpa memerlukan GPU khusus. Pertukaran data antara perangkat pengguna dan server dilakukan melalui jaringan menggunakan protokol komunikasi.

3.3 Software Interfaces

DupIssue berinteraksi dengan beberapa komponen perangkat lunak internal dan eksternal yang saling terintegrasi untuk mendukung pengelolaan issue, deteksi duplikasi, dan integrasi sumber data. Backend Laravel berperan sebagai penghubung utama antara antarmuka pengguna, database, layanan NLP, dan GitHub REST API. Rincian antarmuka perangkat lunak yang digunakan dijelaskan pada tabel berikut.

Tabel 3.1. *Software Interfaces DupIssue*

Komponen	Teknologi	Fungsi dan Pertukaran Data

Frontend dan Backend	Laravel Blade, Bootstrap 5.3, JavaScript, Laravel 12	Menampilkan antarmuka pengguna dan mengirimkan data formulir, permintaan rekomendasi, serta tindakan pengguna ke backend. Backend mengembalikan halaman, informasi issue, hasil rekomendasi, dan respons operasi.
Database	MySQL 8.0	Menyimpan dan mengambil data pengguna, project, anggota, undangan, issue, vote, kandidat duplikat, status penanganan, dan log aktivitas melalui backend Laravel.
NLP Service	Python 3.12, FastAPI	Menerima permintaan pemrosesan teks dari Laravel untuk menghasilkan rekomendasi issue serupa dan kandidat duplikat. Hasil pemrosesan dikembalikan dalam format JSON yang memuat identitas kandidat dan skor kemiripan.
NLP Model dan Library	Sentence Transformers, scikit-learn	Menghasilkan representasi semantik teks menggunakan model pretrained paraphrase-multilingual-MiniLM-L12-v2 serta menyediakan metode pembanding TF-IDF dengan cosine similarity.
Background Processing	Laravel Database Queue	Menyimpan pekerjaan deteksi setelah issue berhasil disimpan. Queue worker mengambil pekerjaan dari database dan memanggil layanan NLP untuk memproses kandidat duplikat secara asynchronous.
Integrasi GitHub	GitHub REST API	Mengambil data issue dari repository GitHub menggunakan kredensial yang sesuai. Data yang diterima mencakup informasi seperti nomor issue, judul, dan deskripsi untuk divalidasi dan diimpor ke project DupIssue.
Impor Dataset	CSV UTF-8	Menerima file CSV yang diunggah Maintainer, kemudian memvalidasi kolom source_id, title, dan description sebelum menyimpan baris data valid ke database MySQL.

Integrasi antara Laravel dan FastAPI digunakan untuk dua mekanisme deteksi, yaitu rekomendasi sebelum Reporter mengirim laporan dan deteksi kandidat setelah issue tersimpan. Rekomendasi diproses ketika Reporter mengisi judul dan deskripsi, sedangkan deteksi setelah penyimpanan dijalankan melalui Laravel Database Queue. Seluruh proses pencarian kemiripan dibatasi pada issue dalam project yang sama. Hasil deteksi berupa kandidat dan skor kemiripan digunakan sebagai bahan pertimbangan Maintainer, bukan keputusan duplikasi otomatis.

Integrasi GitHub REST API hanya digunakan untuk mengimpor issue ke dalam DupIssue dan tidak mencakup sinkronisasi dua arah. Selain itu, sistem mendukung unggahan dataset CSV secara langsung tanpa memerlukan layanan eksternal. Apabila layanan NLP atau GitHub mengalami gangguan, sistem harus mempertahankan data issue yang telah tersimpan dan menyediakan penanganan kegagalan sesuai proses terkait.

3.4 Communications Interfaces

DupIssue menggunakan komunikasi berbasis jaringan untuk menghubungkan web browser pengguna dengan backend Laravel, serta mengintegrasikan backend dengan layanan NLP, database, dan GitHub REST API. Pertukaran informasi dilakukan menggunakan protokol dan format data yang disesuaikan dengan kebutuhan masing-masing komponen. Rincian komunikasi sistem dijelaskan pada tabel berikut :

Tabel 3.2. *Communications Interfaces DupIssue*

Komunikasi	Protokol / Format	Deskripsi
Web Browser ↔ Laravel	HTTP/HTTPS, HTML, JSON	Browser mengirim permintaan pengguna ke backend dan menerima halaman HTML atau respons JSON. HTTPS digunakan untuk akses melalui jaringan yang memerlukan perlindungan komunikasi.
Laravel ↔ FastAPI	HTTP internal, REST API, JSON	Backend mengirim data yang diperlukan untuk pemrosesan kemiripan teks dan menerima hasil rekomendasi atau kandidat duplikat beserta skor kemiripan.
Laravel ↔ MySQL	Protokol MySQL melalui koneksi database	Backend melakukan penyimpanan, pembaruan, dan pengambilan data aplikasi serta pekerjaan antrean.
Laravel Queue Worker ↔ MySQL	Protokol MySQL melalui koneksi database	Queue worker mengambil pekerjaan deteksi yang tertunda dan memperbarui informasi pemrosesan sesuai hasil pekerjaan.

Laravel ↔ GitHub REST API	HTTPS, REST API, JSON	Backend mengirim permintaan pengambilan issue ke GitHub menggunakan autentikasi yang sesuai dan menerima data issue dalam format JSON.
Web Browser → Laravel (Upload CSV)	HTTP/HTTPS, multipart/form-data	Maintainer mengunggah dataset CSV melalui formulir aplikasi untuk divalidasi dan diproses oleh backend.

Untuk menjaga keamanan komunikasi, akses aplikasi yang dilakukan melalui jaringan publik harus menggunakan HTTPS. Autentikasi pengguna direncanakan menggunakan mekanisme sesi Laravel, sedangkan hak akses Reporter dan Maintainer diperiksa pada sisi server. Komunikasi internal antara Laravel dan FastAPI dibatasi pada lingkungan server atau jaringan internal yang terkontrol. Kredensial GitHub digunakan oleh backend dan tidak dikirimkan kepada browser dalam respons aplikasi.

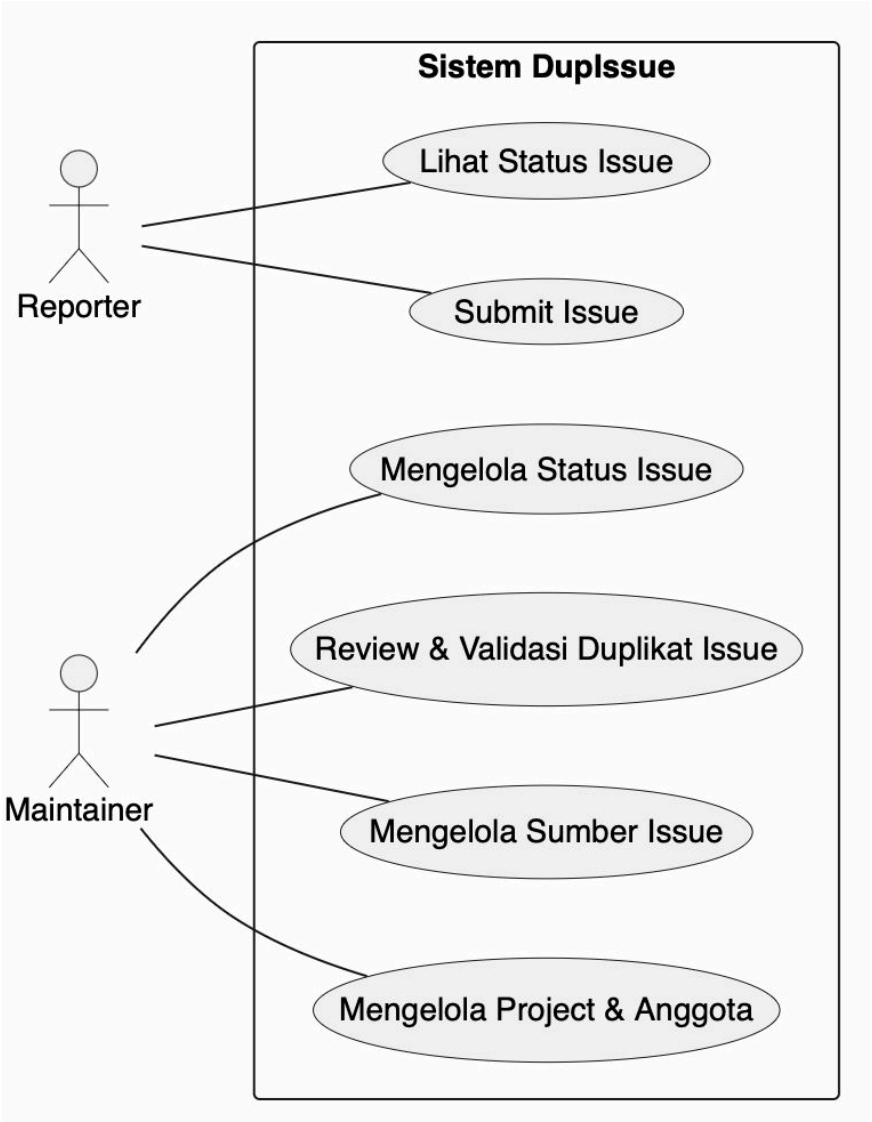
Komunikasi rekomendasi issue dilakukan secara dinamis menggunakan JavaScript dengan jeda setelah pengetikan untuk mengurangi jumlah permintaan ke layanan NLP. Hasil rekomendasi ditampilkan tanpa memuat ulang seluruh halaman, dan respons dari input sebelumnya tidak boleh menggantikan hasil untuk input terbaru. Apabila layanan NLP gagal memberikan respons, sistem menampilkan informasi kegagalan tanpa menghalangi Reporter mengirim laporan yang valid.

Sementara itu, deteksi kandidat setelah penyimpanan issue menggunakan mekanisme asynchronous melalui Laravel Database Queue. Proses ini tidak mengharuskan browser mempertahankan koneksi sampai pemrosesan selesai. Sistem menyimpan status deteksi yang dapat diperiksa kembali melalui aplikasi. Pemberitahuan kepada pengguna disediakan melalui antarmuka DupIssue tanpa mewajibkan integrasi email atau push notification.

4. System Features

4.1 System Feature Overview

DupIssue merupakan aplikasi web pengelolaan bug dan issue report yang dilengkapi rekomendasi serta deteksi kandidat issue duplikat berbasis NLP semantic similarity. Fungsionalitas sistem direpresentasikan dalam use case diagram pada Gambar 4.1.



Gambar 4.1 Use Case Diagram DupIssue

Diagram tersebut menunjukkan dua aktor utama dan enam fungsi utama sistem, dengan penjelasan sebagai berikut.

1. **Reporter**, pengguna yang melaporkan bug atau issue dan memantau penanganannya pada project tempat ia menjadi anggota aktif. Aksi yang dapat dilakukan:
    a) Submit Issue: mengisi judul dan deskripsi, membaca rekomendasi issue serupa, lalu mengirim issue baru atau memberikan vote pada issue yang sama.
    b) Lihat Status Issue: mencari issue, melihat detail, riwayat status, dan relasi issue duplikat dengan issue induk.

2. **Maintainer**, pengguna yang memiliki dan mengelola project. Aksi yang dapat dilakukan:
    c) Mengelola Project & Anggota: membuat dan mengubah project, mengundang Reporter, membatalkan undangan, dan menghapus anggota.
    d) Mengelola Sumber Issue: mengimpor issue dari dataset CSV atau repository GitHub.
    e) Review & Validasi Duplikat Issue: membandingkan kandidat hasil NLP, menetapkan atau menolak relasi duplikat, dan mengoreksi keputusan.
    f) Mengelola Status Issue: memperbarui status penanganan dan menutup duplikat tertaut secara otomatis saat induk Resolved.

Tabel 4.1. Daftar System Feature DupIssue

ID	System Feature	Aktor	Prioritas
REQ-01	Submit Issue	Reporter	High
REQ-02	Lihat Status Issue	Reporter	High
REQ-03	Mengelola Project & Anggota	Maintainer (Reporter pada penerimaan undangan)	High
REQ-04	Mengelola Sumber Issue	Maintainer	High
REQ-05	Review & Validasi Duplikat Issue	Maintainer	High
REQ-06	Mengelola Status Issue	Maintainer	Medium
REQ-07	Fungsi Pendukung (autentikasi, dashboard, log aktivitas)	Reporter, Maintainer	Medium

Setiap fitur pada REQ-01 sampai REQ-06 dijelaskan melalui use case specification, activity diagram, sequence diagram, user story, user story scenario, dan functional requirements. Fungsi pendukung pada REQ-07 hanya dijelaskan melalui functional requirements karena tidak berdiri sebagai use case tersendiri pada use case diagram.

Konvensi. Kata "harus" menyatakan kebutuhan wajib. Istilah status issue (OPEN, IN_PROGRESS, RESOLVED, CLOSED, DUPLICATE) dan status deteksi (PENDING, PROCESSING, COMPLETED, FAILED) mengikuti PDM pada Appendix B. Setiap kebutuhan diberi ID unik dengan format REQ-XX.Y.

4.2 System Feature 1: Submit Issue (REQ-01)

4.2.1 Description and Priority

Reporter mengisi laporan, memeriksa rekomendasi issue serupa dari project yang sama, lalu mengirim issue baru atau memberikan vote pada issue yang sudah ada. Fitur ini merupakan inti tujuan DupIssue, yaitu mencegah laporan duplikat sebelum tersimpan.

4.2.2 Use Case Specification

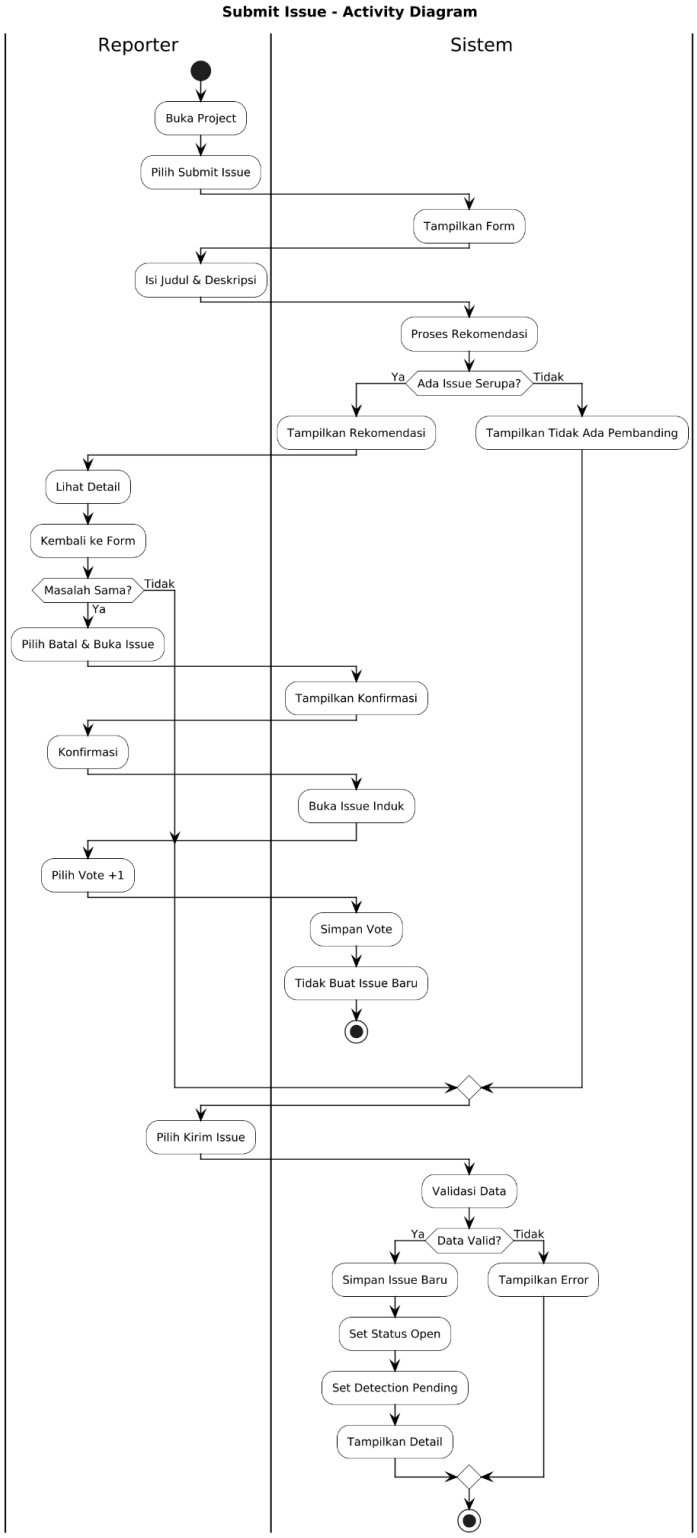
Tabel 4.2. Deskripsi Use Case Specification "Submit Issue"

Nama Use Case	Submit Issue
Aktor	Reporter
Deskripsi	Reporter mengisi laporan, memeriksa rekomendasi issue serupa, lalu mengirim issue baru atau memberikan vote pada issue yang sama
Kondisi Awal	Reporter sudah masuk dan menjadi anggota aktif project serta rekomendasi hanya menggunakan issue dari project yang sama
Kondisi Akhir	Issue baru tersimpan dengan status Open dan deteksi dijadwalkan atau vote tersimpan pada issue induk tanpa membuat issue baru
Skenario Normal	
Aktor	**Sistem**
1. Reporter memilih project dan menekan Submit Issue	
	2. Sistem memeriksa keanggotaan Reporter
	3. Sistem menampilkan formulir Judul dan Deskripsi
4. Reporter mengisi judul dan deskripsi	
	5. Sistem mencari issue serupa setelah jeda pengetikan
	6. Sistem menampilkan maksimal tiga rekomendasi dari project yang sama
7. Reporter membuka salah satu rekomendasi	
	8. Sistem menampilkan detail rekomendasi tanpa menghapus isian
9. Reporter kembali ke formulir	
10. Reporter menekan Kirim Issue	
	11. Sistem memvalidasi judul dan deskripsi
	12. Sistem memeriksa kembali keanggotaan Reporter
	13. Sistem menyimpan issue dengan status Open dan deteksi Pending
	14. Sistem menjadwalkan deteksi duplikasi
	15. Sistem menampilkan detail issue baru
Skenario Alternatif	
A1. Masalah sama dan Reporter memberikan vote	
1.a. Reporter membuka rekomendasi yang sesuai	
	2.a. Sistem menampilkan detail issue rekomendasi
3.a. Reporter menekan Batal Submit dan Buka Issue	

	4.a. Sistem menampilkan konfirmasi pembatalan laporan
5.a. Reporter menekan Ya Buka Issue	
	6.a. Sistem membatalkan laporan baru
	7.a. Sistem membuka issue induk yang sesuai
8.a. Reporter menekan Saya juga mengalami ini +1	
	9.a. Sistem memeriksa keanggotaan Reporter
	10.a. Sistem menyimpan satu vote aktif
	11.a. Sistem memperbarui jumlah vote
A2. Rekomendasi kosong atau Reporter langsung mengirim laporan	
1.b. Reporter mengisi judul dan deskripsi	
	2.b. Sistem menampilkan bahwa rekomendasi tidak tersedia
3.b. Reporter menekan Kirim Issue	
	4.b. Sistem memvalidasi data
	5.b. Sistem menyimpan issue baru
A3. Membatalkan laporan	
1.c. Reporter menekan Batal	
	2.c. Sistem menampilkan konfirmasi pembuangan isian
3.c. Reporter menekan Ya Batalkan	
	4.c. Sistem kembali ke Daftar Issue tanpa menyimpan laporan
A4. Membatalkan vote pada issue	
1.d. Reporter membuka issue yang sudah diberi vote	
2.d. Reporter menekan Batalkan Vote	
	3.d. Sistem menghapus vote aktif milik Reporter
	4.d. Sistem memperbarui jumlah vote
Skenario Exception	
E1. Data laporan tidak valid	
1.e. Reporter menekan Kirim Issue dengan data tidak valid	
	2.e. Sistem menolak pengiriman
	3.e. Sistem menandai kolom yang perlu diperbaiki
	4.e. Sistem menampilkan ketentuan panjang judul dan deskripsi
5.e. Reporter memperbaiki input	
6.e. Reporter menekan Kirim Issue	
	7.e. Sistem memvalidasi ulang data
	8.e. Sistem menyimpan issue setelah data valid
E2. Rekomendasi atau deteksi NLP gagal	
1.f. Reporter mengisi formulir saat layanan rekomendasi tidak tersedia	
	2.f. Sistem menampilkan Rekomendasi belum dapat dimuat
	3.f. Sistem mempertahankan isian formulir
4.f. Reporter menekan Kirim Issue	
	5.f. Sistem menyimpan issue dengan status Open
	6.f. Sistem menetapkan status deteksi Pending atau Failed sesuai kondisi
E3. Issue rekomendasi tidak tersedia atau vote berulang	
1.g. Reporter membuka issue yang tidak tersedia atau memberikan vote ulang	

	2.g. Sistem menolak akses issue yang tidak tersedia
	3.g. Sistem tidak menambahkan vote yang sudah aktif
	4.g. Sistem menampilkan jumlah vote yang tersimpan

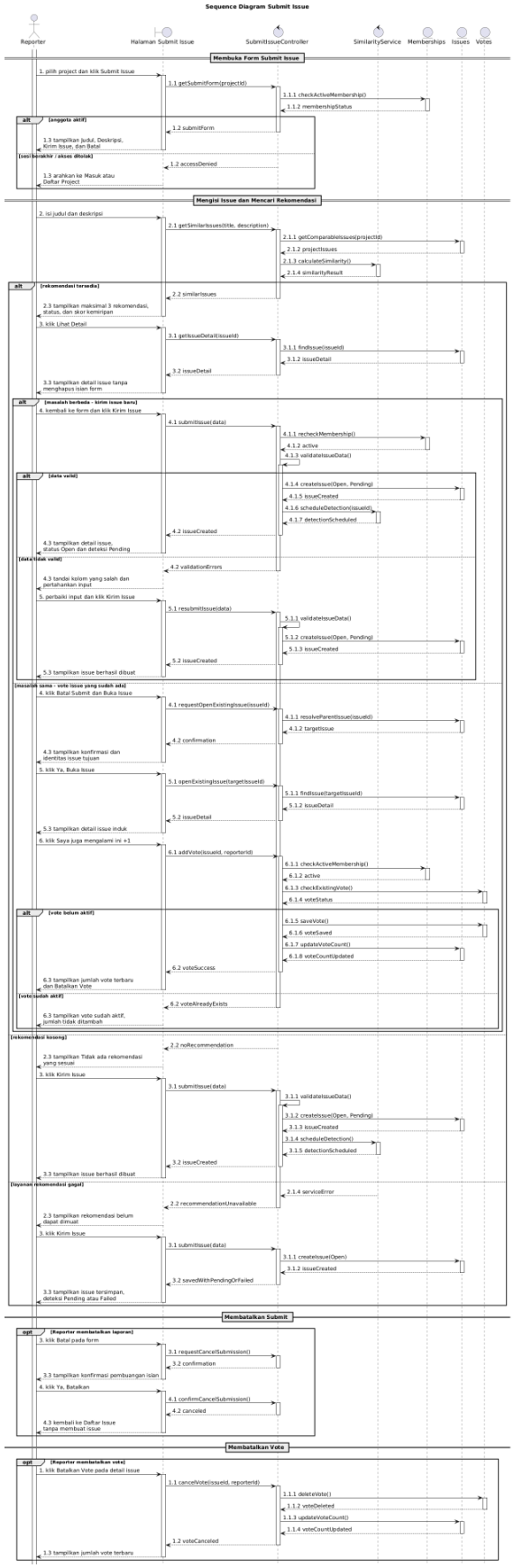
4.2.3 Activity Diagram

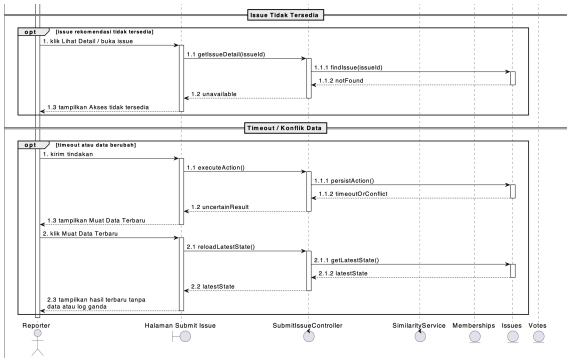


Gambar 4.2 Activity Diagram Submit Issue

Gambar 4.2 menggambarkan alur Submit Issue pada swimlane Reporter dan Sistem. Proses dimulai ketika Reporter membuka project dan memilih Submit Issue. Sistem menampilkan formulir, Reporter mengisi judul dan deskripsi, kemudian Sistem memproses rekomendasi. Jika ada issue serupa, rekomendasi ditampilkan, jika tidak, Sistem menampilkan keterangan tidak ada pembanding. Reporter dapat melihat detail rekomendasi lalu kembali ke formulir. Bila masalah sama, Reporter memilih batal dan buka issue, mengonfirmasi, Sistem membuka issue induk, dan Reporter memilih vote +1 sehingga Sistem menyimpan vote tanpa membuat issue baru. Bila masalah berbeda, Reporter memilih Kirim Issue, Sistem memvalidasi data, dan jika valid Sistem menyimpan issue baru dengan status OPEN dan detection PENDING lalu menampilkan detail. Jika tidak valid, Sistem menampilkan error.

4.2.4 Sequence Diagram


Sequence Diagram Submit Issue
Reporter
Halaman Submit Issue
SubmitIssueController
SimilarityService
Memberships
Issues
Votes
Membuka Form Submit Issue
1. pilih project dan klik Submit Issue
1.1 getSubmitForm(projectId)
1.1.1 checkActiveMembership()
1.1.2 membershipStatus
alt
[anggota aktif]
1.2 submitForm
1.3 tampilkan judul, Deskripsi, Kirim Issue, dan Batal
[sesi berakhir / akses ditolak]
1.2 accessDenied
1.3 arahkan ke Masuk atau Daftar Project
Mengisi Issue dan Mencari Rekomendasi
2. isi judul dan deskripsi
2.1 getSimilarIssues(title, description)
2.1.1 getComparableIssues(projectId)
2.1.2 projectIssues
2.1.3 calculateSimilarity()
2.1.4 similarityResult
alt
[rekomendasi tersedia]
2.2 similarIssues
2.3 tampilkan maksimal 3 rekomendasi, status, dan skor kemiripan
3. klik Lihat Detail
3.1 getIssueDetail(issueId)
3.1.1 findIssue(issueId)
3.1.2 issueDetail
3.2 issueDetail
3.3 tampilkan detail issue tanpa menghapus isian form
alt
[masalah berbeda - kirim issue baru]
4. kembali ke form dan klik Kirim Issue
4.1 submitIssue(data)
4.1.1 recheckMembership()
4.1.2 active
4.1.3 validateIssueData()
alt
[data valid]
4.1.4 createIssue(Open, Pending)
4.1.5 issueCreated
4.1.6 scheduleDetection(issueId)
4.1.7 detectionScheduled
4.2 issueCreated
4.3 tampilkan detail issue, status Open dan deteksi Pending
[data tidak valid]
4.2 validationErrors
4.3 tandai kolom yang salah dan pertahankan input
5. perbaiki input dan klik Kirim Issue
5.1 resubmitIssue(data)
5.1.1 validateIssueData()
5.1.2 createIssue(Open, Pending)
5.1.3 issueCreated
5.2 issueCreated
5.3 tampilkan issue berhasil dibuat
[masalah sama - vote issue yang sudah ada]
4. klik Batal Submit dan Buka Issue
4.1 requestOpenExistingIssue(issueId)
4.1.1 resolveParentIssue(issueId)
4.1.2 targetIssue
4.2 confirmation
4.3 tampilkan konfirmasi dan identitas issue tujuan
5. klik Ya, Buka Issue
5.1 openExistingIssue(targetIssueId)
5.1.1 findIssue(targetIssueId)
5.1.2 issueDetail
5.2 issueDetail
5.3 tampilkan detail issue induk
6. klik Saya juga mengalami ini +1
6.1 addVote(issueId, reporterId)
6.1.1 checkActiveMembership()
6.1.2 active
6.1.3 checkExistingVote()
6.1.4 voteStatus
alt
[vote belum aktif]
6.1.5 saveVote()
6.1.6 voteSaved
6.1.7 updateVoteCount()
6.1.8 voteCountUpdated
6.2 voteSuccess
6.3 tampilkan jumlah vote terbaru dan Batalkan Vote
[vote sudah aktif]
6.2 voteAlreadyExists
6.3 tampilkan vote sudah aktif, jumlah tidak ditambah
[rekomendasi kosong]
2.2 noRecommendation
2.3 tampilkan Tidak ada rekomendasi yang sesuai
3. klik Kirim Issue
3.1 submitIssue(data)
3.1.1 validateIssueData()
3.1.2 createIssue(Open, Pending)
3.1.3 issueCreated
3.1.4 scheduleDetection()
3.1.5 detectionScheduled
3.2 issueCreated
3.3 tampilkan issue berhasil dibuat
[layanan rekomendasi gagal]
2.1.4 serviceError
2.2 recommendationUnavailable
2.3 tampilkan rekomendasi belum dapat dimuat
3. klik Kirim Issue
3.1 submitIssue(data)
3.1.1 createIssue(Open)
3.1.2 issueCreated
3.2 savedWithPendingOrFailed
3.3 tampilkan issue tersimpan, deteksi Pending atau Failed
Membatalkan Submit
opt
[Reporter membatalkan laporan]
3. klik Batal pada form
3.1 requestCancelSubmission()
3.2 confirmation
3.3 tampilkan konfirmasi pembuangan isian
4. klik Ya, Batalkan
4.1 confirmCancelSubmission()
4.2 canceled
4.3 kembali ke Daftar Issue tanpa membuat issue
Membatalkan Vote
opt
[Reporter membatalkan vote]
1. klik Batalkan Vote pada detail issue
1.1 cancelVote(issueId, reporterId)
1.1.1 deleteVote()
1.1.2 voteDeleted
1.1.3 updateVoteCount()
1.1.4 voteCountUpdated
1.2 voteCanceled
1.3 tampilkan jumlah vote terbaru



Gambar 4.3 Sequence Diagram Submit Issue (dibaca dari kiri ke kanan)

Gambar 4.3 menunjukkan interaksi antara Reporter, Halaman Submit Issue, SubmitIssueController, SimilarityService, Memberships, Issues, dan Votes. Controller memeriksa keanggotaan, memanggil SimilarityService untuk rekomendasi, lalu pada pengiriman memvalidasi data, menyimpan issue dengan status OPEN dan deteksi PENDING, dan menjadwalkan deteksi. Fragmen alternatif mencakup pembatalan laporan, pembukaan issue induk, penyimpanan serta pembatalan vote, rekomendasi kosong, validasi gagal, kegagalan layanan rekomendasi, dan issue tidak tersedia.

4.2.5 User Story

Tabel 4.6. User Story Submit Issue

Name (Description)	User Story
Submit Issue	As a Reporter, I want to submit an issue and receive recommendations for similar issues so that I can avoid creating duplicate issues and vote for an existing issue when appropriate.

4.2.6 User Story Scenario

Tabel 4.7. User Story Scenario Submit Issue

```
Feature: Submit Issue dengan Rekomendasi Pre-Submit
  In order to mencegah laporan duplikat
  As a Reporter
  I want to melihat rekomendasi issue serupa sebelum mengirim issue baru

  Background:
    Given I am logged in as "valentino@dupissue.test" with role "Reporter"
    And I am an active member of project "Toko Online Baju" with id "1"
```

Scenario: Reporter mengirim issue setelah memeriksa rekomendasi
Given I am on "/projects/1/submit"
When I fill in "judul" with "Tombol Bayar tidak merespon"
And I fill in "deskripsi" with "Setelah klik Bayar Sekarang halaman tetap berada di checkout"
Then I should see "3 rekomendasi" in the "rekomendasi" element
And I should see "Tombol Bayar" in the "rekomendasi" element
When I follow "Lihat Detail"
Then I should see "detail issue" in the "modal" element
When I press "Kembali ke Formulir"
And I press "Kirim Issue"
Then I should see "Issue berhasil dibuat"
And I should see "Open" in the "status" element
And I should see "Pending" in the "detection_status" element

Scenario: Reporter membatalkan submit dan memberikan vote pada issue yang sama
Given I am on "/projects/1/submit"
When I fill in "judul" with "Tombol Bayar tidak merespon"
And I fill in "deskripsi" with "Masalah yang sama terjadi saat proses checkout"
Then I should see "3 rekomendasi" in the "rekomendasi" element
When I follow "Lihat Detail"
And I press "Batal Submit dan Buka Issue"
And I press "Ya, Buka Issue"
Then I should be on "/projects/1/issues/10"
When I press "Saya juga mengalami ini +1"
Then I should see "Vote berhasil"
And I should see "1" in the "vote_count" element
And I should not see "Issue berhasil dibuat"

Scenario: Reporter mengirim issue saat rekomendasi kosong
Given I am on "/projects/1/submit"
When I fill in "judul" with "Error upload foto profil"
And I fill in "deskripsi" with "Upload foto profil menyebabkan aplikasi berhenti"
Then I should see "Belum ada issue pembanding" in the "rekomendasi" element
When I press "Kirim Issue"
Then I should see "Issue berhasil dibuat"
And I should see "Open" in the "status" element

Scenario: Reporter membatalkan laporan
Given I am on "/projects/1/submit"
When I fill in "judul" with "Test batal"
And I press "Batal"
Then I should see "Konfirmasi pembuangan isian"
When I press "Ya, Batalkan"
Then I should be on "/projects/1/issues"
And I should not see "Issue berhasil dibuat"

Scenario: Reporter membatalkan vote
Given I am on "/projects/1/issues/10"
And I should see "Batalkan Vote"
When I press "Batalkan Vote"
Then I should see "Vote dibatalkan"
And I should see "0" in the "vote_count" element

```
Scenario Outline: Validasi judul dan deskripsi tidak valid
  Given I am on "/projects/1/submit"
  When I fill in "judul" with "<judul>"
  And I fill in "deskripsi" with "<deskripsi>"
  And I press "Kirim Issue"
  Then I should see "<pesan_error>"
  And I should see "<field>" in the "error" element

  Examples:
    | judul       | deskripsi                    | pesan_error                        | field     |
    |             | Deskripsi valid lebih dari 10  | Judul wajib 5-200 karakter         | judul     |
    | ab          | Deskripsi valid lebih dari 10  | Judul wajib 5-200 karakter         | judul     |
    | Judul valid | pendek                       | Deskripsi wajib 10-10000 karakter  | deskripsi |
    | Judul valid |                              | Deskripsi wajib 10-10000 karakter  | deskripsi |

Scenario: Layanan rekomendasi gagal tetapi issue tetap dapat dikirim
  Given I am on "/projects/1/submit"
  And I should see "Rekomendasi belum dapat dimuat"
  When I fill in "judul" with "Error rekomendasi"
  And I fill in "deskripsi" with "Deskripsi valid lebih dari sepuluh karakter"
  And I press "Kirim Issue"
  Then I should see "Issue berhasil dibuat"
  And I should see "Open" in the "status" element
  And I should see "Failed" in the "detection_status" element

Scenario: Vote berulang tidak menambah jumlah vote
  Given I am on "/projects/1/issues/10"
  And I should see "Batalkan Vote"
  When I press "Saya juga mengalami ini +1"
  Then I should see "Vote sudah aktif"
  And I should see "1" in the "vote_count" element

Scenario: Rekomendasi yang dipilih tidak tersedia
  Given I am on "/projects/1/submit"
  When I follow "Lihat Detail"
  Then I should see "Akses tidak tersedia"
```

4.2.7 Functional Requirements

- **REQ-01.1:** Sistem harus memeriksa keanggotaan aktif Reporter pada project sebelum menampilkan formulir dan sekali lagi sebelum menyimpan issue atau vote.
- **REQ-01.2:** Formulir harus memuat kolom Judul (5 sampai 200 karakter) dan Deskripsi (10 sampai 10.000 karakter).
- **REQ-01.3:** Sistem harus meminta rekomendasi ke NLP Service setelah jeda pengetikan (nilai awal 500 ms, lihat Appendix C) dan menampilkannya tanpa memuat ulang halaman.
- **REQ-01.4:** Sistem harus menampilkan maksimal tiga rekomendasi yang hanya berasal dari project yang sama, lengkap dengan skor kemiripan.

- **REQ-01.5:** Sistem tidak boleh menampilkan respons rekomendasi dari input lama sebagai pengganti hasil input terbaru.
- **REQ-01.6:** Rekomendasi yang berstatus DUPLICATE harus diarahkan ke issue induknya sebelum Reporter memberikan vote.
- **REQ-01.7:** Reporter harus dapat membuka detail rekomendasi dan kembali ke formulir tanpa kehilangan isian.
- **REQ-01.8:** Bila rekomendasi kosong, sistem harus menampilkan "Belum ada issue pembanding" dan tetap mengizinkan pengiriman.
- **REQ-01.9:** Bila layanan NLP gagal, sistem harus menampilkan "Rekomendasi belum dapat dimuat", mempertahankan isian, dan tetap mengizinkan pengiriman.
- **REQ-01.10:** Bila data tidak valid, sistem harus menolak pengiriman, menandai kolom yang salah, dan menampilkan ketentuan panjang judul dan deskripsi.
- **REQ-01.11:** Issue valid harus disimpan dengan source_type FORM, status OPEN, detection_status PENDING, vote_count 0, dan author_id Reporter.
- **REQ-01.12:** Sistem harus menjadwalkan deteksi duplikasi melalui Laravel Database Queue. Kegagalan penjadwalan atau NLP tidak boleh menghapus issue yang tersimpan.
- **REQ-01.13:** Reporter harus dapat membatalkan submit untuk membuka issue induk, dengan konfirmasi, tanpa menyimpan issue baru.
- **REQ-01.14:** Reporter harus dapat membatalkan pengisian dengan konfirmasi pembuangan isian dan kembali ke Daftar Issue.
- **REQ-01.15:** Sistem harus menyimpan paling banyak satu vote aktif untuk setiap kombinasi issue dan Reporter, dan menolak vote berulang dengan pesan "Vote sudah aktif".
- **REQ-01.16:** Pembatalan vote harus menghapus catatan vote milik Reporter dan memperbarui vote_count agar konsisten dengan tabel votes.
- **REQ-01.17:** Akses ke issue yang tidak tersedia atau di luar project Reporter harus ditolak dengan pesan "Akses tidak tersedia".
- **REQ-01.18:** Pembuatan issue, vote, dan pembatalan vote harus dicatat pada log aktivitas project.

4.3 System Feature 2: Lihat Status Issue (REQ-02)

4.3.1 Description and Priority

Reporter mencari issue, melihat daftar, detail, riwayat status, jumlah vote, serta menelusuri issue induk dari issue duplikat pada project yang diikuti. Fitur ini hanya membaca data dan tidak mengubahnya.

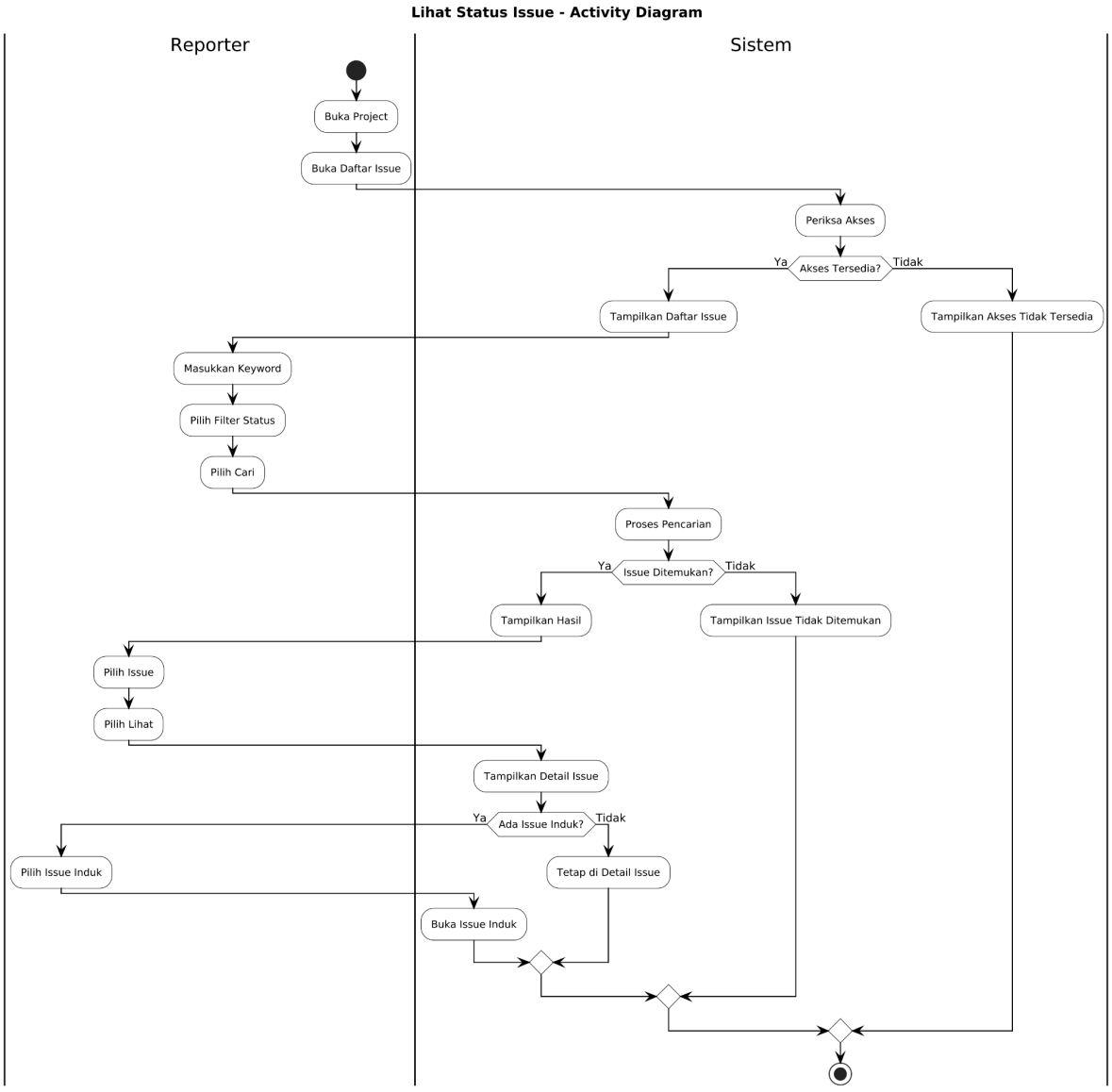
4.3.2 Use Case Specification

Tabel 4.8. Deskripsi Use Case Specification "Lihat Status Issue"

Nama Use Case	Lihat Status Issue
Aktor	Reporter
Deskripsi	Reporter mencari issue dan melihat perkembangan penanganan, detail laporan, serta relasi issue induk pada project yang diikuti
Kondisi Awal	Reporter sudah masuk dan menjadi anggota aktif project
Kondisi Akhir	Daftar atau detail issue tampil dengan status terbaru tanpa mengubah data issue
Skenario Normal	
Aktor	**Sistem**
1. Reporter memilih project dan membuka Daftar Issue	
	2. Sistem memeriksa keanggotaan Reporter
	3. Sistem menampilkan daftar issue project
4. Reporter memasukkan kata kunci atau filter status	
5. Reporter menekan Cari	
	6. Sistem menampilkan issue yang sesuai
7. Reporter memilih issue yang ingin diperiksa	
	8. Sistem menampilkan detail dan riwayat status issue
9. Reporter menekan Muat Ulang	
	10. Sistem menampilkan status terbaru yang dapat diakses
Skenario Alternatif	
A1. Tidak ada data atau hasil pencarian	
1.a. Reporter menekan Cari pada project kosong atau pencarian tanpa hasil	
	2.a. Sistem menampilkan bahwa issue tidak tersedia
	3.a. Sistem menyediakan Reset Filter
4.a. Reporter menekan Reset Filter	
	5.a. Sistem menghapus kata kunci dan filter
	6.a. Sistem menampilkan seluruh issue yang tersedia
A2. Menelusuri issue induk	
1.b. Reporter membuka issue yang ditandai sebagai duplikat	
	2.b. Sistem menampilkan tautan Issue Induk
3.b. Reporter menekan tautan Issue Induk	
	4.b. Sistem memeriksa hak akses Reporter
	5.b. Sistem menampilkan detail issue induk
Skenario Exception	
E1. Issue tidak tersedia	
1.c. Reporter membuka issue yang tidak ditemukan atau tidak dapat diakses	
	2.c. Sistem menampilkan Issue tidak tersedia
	3.c. Sistem menyediakan Kembali ke Daftar
4.c. Reporter menekan Kembali ke Daftar	
	5.c. Sistem menampilkan daftar issue yang masih dapat diakses
E2. Status gagal diperbarui	

1.d. Reporter menekan Muat Ulang saat layanan data bermasalah	
	2.d. Sistem menampilkan Gagal memuat status
	3.d. Sistem menandai data lama sebagai belum diperbarui
4.d. Reporter menekan Coba Lagi	
	5.d. Sistem memeriksa kembali hak akses
	6.d. Sistem mengambil ulang detail issue

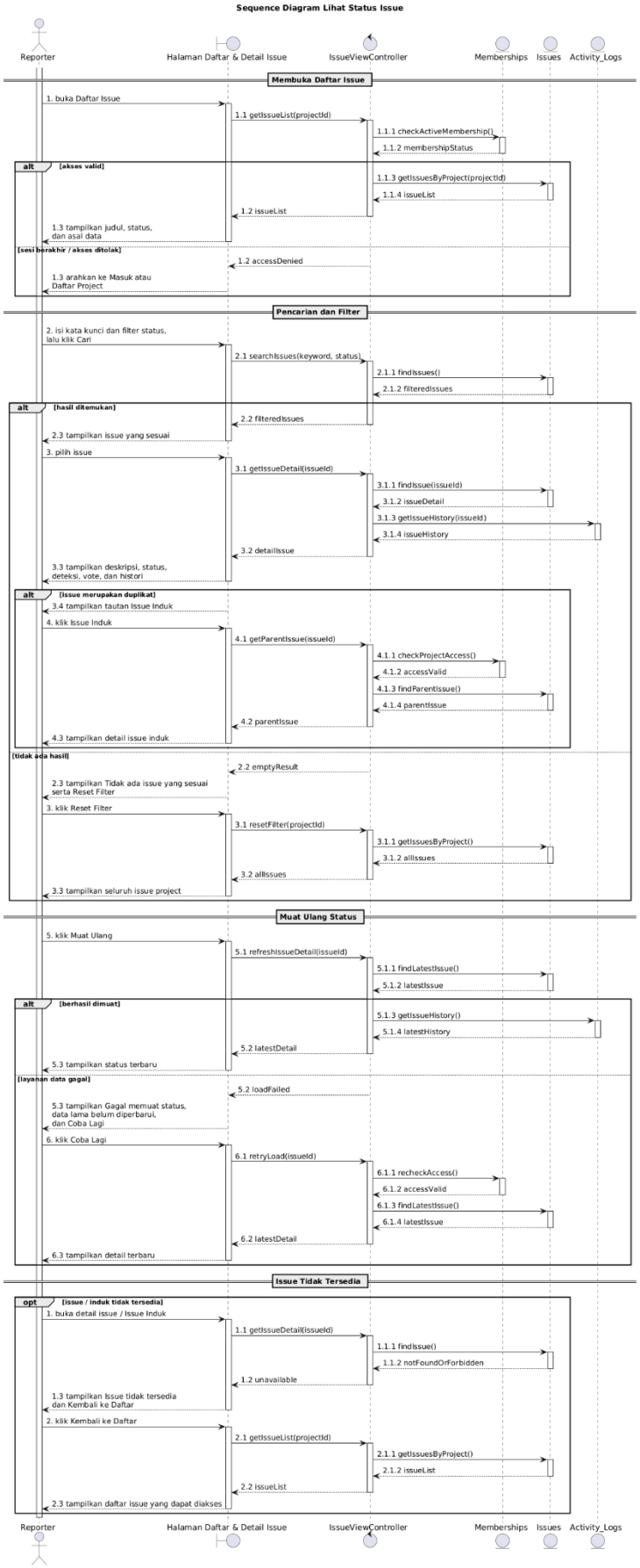
4.3.3 Activity Diagram



Gambar 4.4 Activity Diagram Lihat Status Issue

Gambar 4.4 menggambarkan alur Reporter membuka project dan Daftar Issue. Sistem memeriksa akses: jika tidak tersedia, Sistem menampilkan pesan akses tidak tersedia, jika tersedia Sistem menampilkan daftar. Reporter memasukkan kata kunci, memilih filter status, dan menekan Cari. Bila issue ditemukan, hasil ditampilkan dan Reporter dapat memilih issue serta menekan Lihat untuk membuka detail. Jika issue memiliki induk, Reporter dapat membuka issue induk, jika tidak ia tetap di detail. Bila issue tidak ditemukan, Sistem menampilkan keterangan issue tidak ditemukan.

4.3.4 Sequence Diagram


Sequence Diagram Lihat Status Issue
Reporter
Halaman Daftar & Detail Issue
IssueViewController
Memberships
Issues
Activity_Logs
Membuka Daftar Issue
1. buka Daftar Issue
1.1 getIssueList(projectId)
1.1.1 checkActiveMembership()
1.1.2 membershipStatus
alt [akses valid]
1.1.3 getIssuesByProject(projectId)
1.1.4 issueList
1.2 issueList
1.3 tampilkan judul, status, dan asal data
[sesi berakhir / akses ditolak]
1.2 accessDenied
1.3 arahkan ke Masuk atau Daftar Project
Pencarian dan Filter
2. isi kata kunci dan filter status, lalu klik Cari
2.1 searchIssues(keyword, status)
2.1.1 findIssues()
2.1.2 filteredIssues
alt [hasil ditemukan]
2.2 filteredIssues
2.3 tampilkan issue yang sesuai
3. pilih issue
3.1 getIssueDetail(issueId)
3.1.1 findIssue(issueId)
3.1.2 issueDetail
3.1.3 getIssueHistory(issueId)
3.1.4 issueHistory
3.2 detailIssue
3.3 tampilkan deskripsi, status, deteksi, vote, dan histori
alt [issue merupakan duplikat]
3.4 tampilkan tautan Issue Induk
4. klik Issue Induk
4.1 getParentIssue(issueId)
4.1.1 checkProjectAccess()
4.1.2 accessValid
4.1.3 findParentIssue()
4.1.4 parentIssue
4.2 parentIssue
4.3 tampilkan detail issue induk
[tidak ada hasil]
2.2 emptyResult
2.3 tampilkan Tidak ada issue yang sesuai serta Reset Filter
3. klik Reset Filter
3.1 resetFilter(projectId)
3.1.1 getIssuesByProject()
3.1.2 allIssues
3.2 allIssues
3.3 tampilkan seluruh issue project
Muat Ulang Status
5. klik Muat Ulang
5.1 refreshIssueDetail(issueId)
5.1.1 findLatestIssue()
5.1.2 latestIssue
alt [berhasil dimuat]
5.1.3 getIssueHistory()
5.1.4 latestHistory
5.2 latestDetail
5.3 tampilkan status terbaru
[layanan data gagal]
5.2 loadFailed
5.3 tampilkan Gagal memuat status, data lama belum diperbarui, dan Coba Lagi
6. klik Coba Lagi
6.1 retryLoad(issueId)
6.1.1 recheckAccess()
6.1.2 accessValid
6.1.3 findLatestIssue()
6.1.4 latestIssue
6.2 latestDetail
6.3 tampilkan detail terbaru
Issue Tidak Tersedia
opt [issue / induk tidak tersedia]
1. buka detail issue / Issue Induk
1.1 getIssueDetail(issueId)
1.1.1 findIssue()
1.1.2 notFoundOrForbidden
1.2 unavailable
1.3 tampilkan Issue tidak tersedia dan Kembali ke Daftar
2. klik Kembali ke Daftar
2.1 getIssueList(projectId)
2.1.1 getIssuesByProject()
2.1.2 issueList
2.2 issueList
2.3 tampilkan daftar issue yang dapat diakses

Gambar 4.5 Sequence Diagram Lihat Status Issue

Gambar 4.5 menunjukkan interaksi Reporter, Halaman Daftar & Detail Issue, IssueViewController, Memberships, Issues, dan Activity_Logs. Controller memeriksa keanggotaan aktif sebelum mengambil daftar issue, menjalankan pencarian dan filter, mengambil detail beserta riwayat status, menelusuri issue induk dengan pemeriksaan akses project, memuat ulang status, serta menangani kondisi issue tidak tersedia dan kegagalan layanan data.

4.3.5 User Story

Tabel 4.12. User Story Lihat Status Issue

Name (Description)	User Story
Lihat Status Issue	As a Reporter, I want to view the status of submitted issues so that I can monitor the progress and handling of reported issues.

4.3.6 User Story Scenario

Tabel 4.13. User Story Scenario Lihat Status Issue

```
Feature: Melihat Status Issue
  In order to memantau perkembangan penanganan issue
  As a Reporter
  I want to mencari issue dan melihat status terbaru pada project yang diikuti

  Background:
    Given I am logged in as "valentino@dupissue.test" with role "Reporter"
    And I am an active member of project "Toko Online Baju" with id "1"

  Scenario: Reporter mencari issue dan melihat detail status
    Given I am on "/projects/1/issues"
    When I fill in "cari" with "Bayar"
    And I select "Open" from "status"
    And I press "Cari"
    Then I should see "Bayar" in the "issue_list" element
    When I follow "Lihat"
    Then I should be on "/projects/1/issues/10"
    And I should see "Open" in the "status" element
    And I should see "riwayat perubahan status" in the "history" element
    When I press "Muat Ulang"
    Then I should see "Open" in the "status" element
```

```
Scenario: Pencarian tidak menghasilkan data
  Given I am on "/projects/1/issues"
  When I fill in "cari" with "issue-tidak-ada"
  And I press "Cari"
  Then I should see "Issue tidak tersedia"
  And I should see "Reset Filter"
  When I press "Reset Filter"
  Then I should see "Daftar Issue"

Scenario: Reporter menelusuri issue induk dari issue duplikat
  Given I am on "/projects/1/issues/11"
  And I should see "Issue Induk"
  When I follow "Issue Induk"
  Then I should be on "/projects/1/issues/10"
  And I should see "detail issue" in the "detail" element

Scenario: Issue tidak tersedia
  Given I am on "/projects/1/issues/999"
  Then I should see "Issue tidak tersedia"
  And I should see "Kembali ke Daftar"
  When I press "Kembali ke Daftar"
  Then I should be on "/projects/1/issues"

Scenario: Status gagal dimuat ulang
  Given I am on "/projects/1/issues/10"
  And I am viewing the issue while the status service is unavailable
  When I press "Muat Ulang"
  Then I should see "Gagal memuat status"
  And I should see "Coba Lagi"
  When I press "Coba Lagi"
  Then I should see "detail issue" in the "detail" element
```

4.3.7 Functional Requirements

- **REQ-02.1:** Sistem harus memeriksa keanggotaan aktif Reporter sebelum menampilkan daftar maupun detail issue, dan hanya menampilkan issue dari project yang diikuti.
- **REQ-02.2:** Daftar issue harus menampilkan judul, status, asal data (Manual, CSV, atau GitHub), dan jumlah vote.
- **REQ-02.3:** Sistem harus menyediakan pencarian berdasarkan kata kunci pada judul atau deskripsi dan filter berdasarkan status issue.
- **REQ-02.4:** Bila tidak ada hasil, sistem harus menampilkan "Issue tidak tersedia" dan tombol "Reset Filter" yang mengembalikan seluruh daftar issue.
- **REQ-02.5:** Detail issue harus menampilkan judul, deskripsi, status penanganan, jumlah vote, dan riwayat perubahan status.

- **REQ-02.6:** Untuk issue berstatus DUPLICATE, sistem harus menampilkan tautan "Issue Induk" yang membuka detail issue induk setelah pemeriksaan hak akses.
- **REQ-02.7:** Tombol "Muat Ulang" harus mengambil status terbaru tanpa mengubah data issue.
- **REQ-02.8:** Bila pengambilan status gagal, sistem harus menampilkan "Gagal memuat status", menandai data lama sebagai belum diperbarui, dan menyediakan "Coba Lagi" yang memeriksa ulang hak akses.
- **REQ-02.9:** Issue yang tidak ditemukan atau di luar akses harus ditolak dengan "Issue tidak tersedia" dan tombol "Kembali ke Daftar".
- **REQ-02.10:** Fitur ini harus bersifat hanya-baca bagi Reporter: tidak ada perubahan status, relasi, atau konten issue.

4.4 System Feature 3: Mengelola Project & Anggota (REQ-03)

4.4.1 Description and Priority

Maintainer membuat dan mengubah project, mengundang Reporter melalui email, membatalkan undangan, dan mencabut akses anggota. Reporter menanggapi undangan (terima atau tolak) melalui halaman Undangan. Project menjadi batas akses, impor, dan perbandingan issue, sehingga fitur ini menjadi prasyarat bagi REQ-01, REQ-02, REQ-04, dan REQ-05.

4.4.2 Use Case Specification

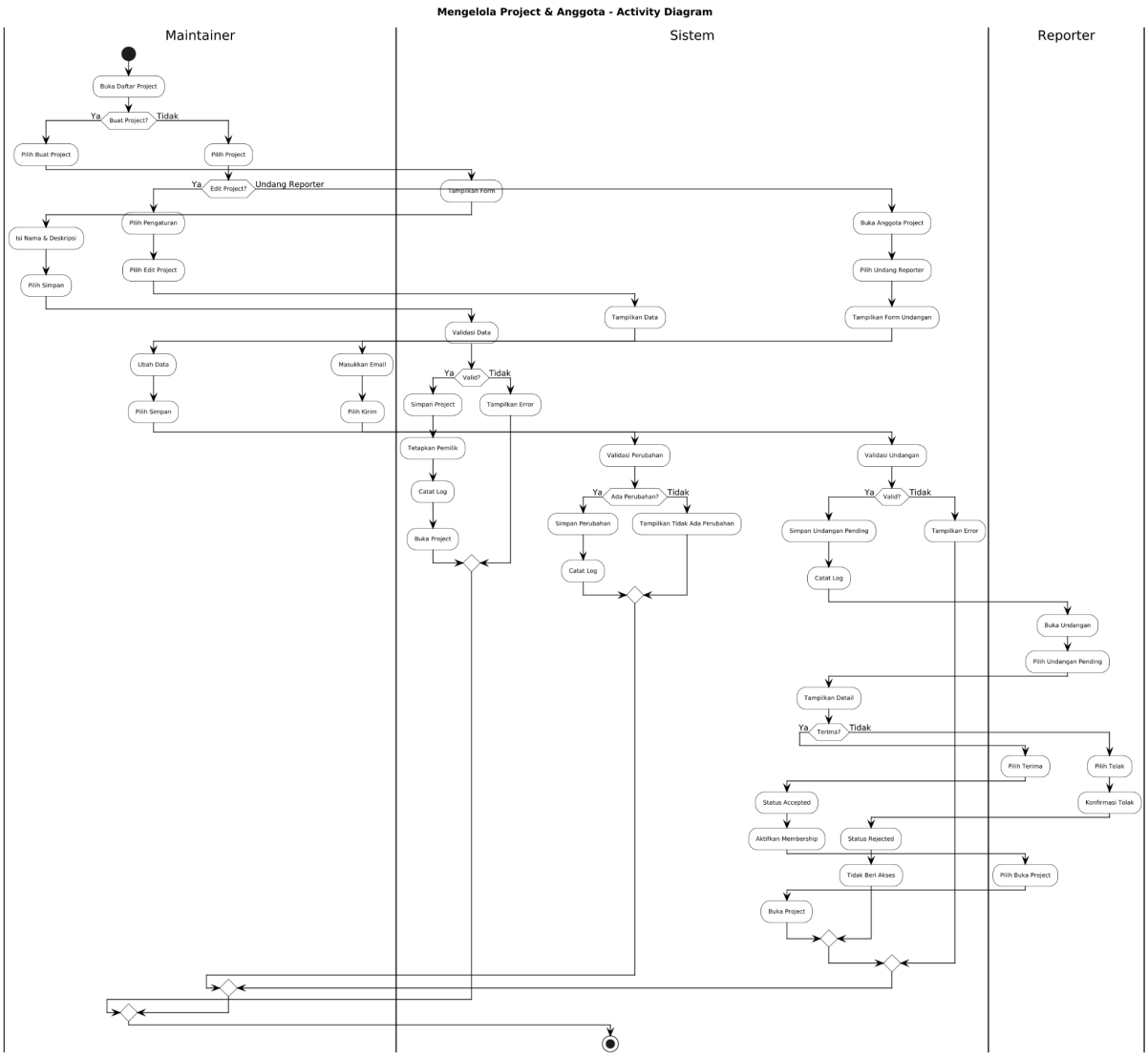
Tabel 4.14. Deskripsi Use Case Specification "Mengelola Project & Anggota"

Nama Use Case	Mengelola Project & Anggota
Aktor	Maintainer
Deskripsi	Maintainer membuat dan mengubah project, mengundang anggota, membatalkan undangan, serta mencabut akses anggota
Kondisi Awal	Maintainer sudah masuk dan hanya dapat mengelola project miliknya serta akun tujuan undangan sudah terdaftar
Kondisi Akhir	Project, undangan, atau keanggotaan tersimpan sesuai tindakan tanpa menghapus laporan dan histori lama
Skenario Normal	
Aktor	**Sistem**
1. Maintainer membuka Daftar Project dan menekan Buat Project	
	2. Sistem memeriksa peran Maintainer

	3. Sistem menampilkan formulir Nama Project dan Deskripsi
4. Maintainer mengisi nama dan deskripsi project	
5. Maintainer menekan Simpan	
	6. Sistem memvalidasi data project
	7. Sistem menyimpan project baru
	8. Sistem menetapkan Maintainer sebagai pemilik project
	9. Sistem mencatat log pembuatan project
	10. Sistem menampilkan halaman project
11. Maintainer membuka menu Anggota Project	
12. Maintainer menekan Undang Anggota	
	13. Sistem menampilkan kolom Email Anggota
14. Maintainer memasukkan email anggota	
15. Maintainer menekan Kirim Undangan	
	16. Sistem memvalidasi email dan status keanggotaan
	17. Sistem menyimpan undangan berstatus Pending
	18. Sistem mencatat log undangan
	19. Sistem menampilkan status undangan
Skenario Alternatif	
A1. Mengubah informasi project	
1.a. Maintainer membuka project miliknya	
2.a. Maintainer menekan Edit Project	
	3.a. Sistem menampilkan data project saat ini
4.a. Maintainer mengubah nama atau deskripsi	
5.a. Maintainer menekan Simpan Perubahan	
	6.a. Sistem memvalidasi data terbaru
	7.a. Sistem menyimpan perubahan project
	8.a. Sistem mencatat log perubahan
A2. Membatalkan undangan	
1.b. Maintainer membuka menu Anggota Project	
2.b. Maintainer memilih undangan berstatus Pending	
3.b. Maintainer menekan Batalkan Undangan	
	4.b. Sistem menampilkan konfirmasi pembatalan
5.b. Maintainer menekan Konfirmasi	
	6.b. Sistem mengubah status undangan menjadi Canceled
	7.b. Sistem mencatat log pembatalan
A3. Menghapus anggota	
1.c. Maintainer memilih anggota aktif	
2.c. Maintainer menekan Hapus Anggota	
	3.c. Sistem menampilkan dampak pencabutan akses
4.c. Maintainer menekan Konfirmasi	
	5.c. Sistem menonaktifkan keanggotaan
	6.c. Sistem mencatat log penghapusan anggota
	7.c. Sistem mempertahankan laporan dan histori lama
A4. Mengundang kembali anggota	
1.d. Maintainer memasukkan email anggota yang pernah dikeluarkan atau dibatalkan undangannya	
2.d. Maintainer menekan Kirim Undangan	
	3.d. Sistem memeriksa bahwa tidak ada keanggotaan aktif atau undangan Pending
	4.d. Sistem membuat undangan baru berstatus Pending

Skenario Exception	
E1. Informasi project tidak valid	
1.f. Maintainer menekan Simpan dengan data project tidak valid	
	2.f. Sistem menolak penyimpanan
	3.f. Sistem menandai kolom yang perlu diperbaiki
4.f. Maintainer memperbaiki data project	
5.f. Maintainer menekan Simpan	
	6.f. Sistem memvalidasi ulang data
	7.f. Sistem menyimpan project setelah data valid
E2. Undangan tidak dapat dibuat	
1.g. Maintainer mengirim undangan dengan data tujuan tidak valid	
	2.g. Sistem menolak pembuatan undangan
	3.g. Sistem menampilkan alasan penolakan
	4.g. Sistem tidak menggandakan undangan atau keanggotaan

4.4.3 Activity Diagram

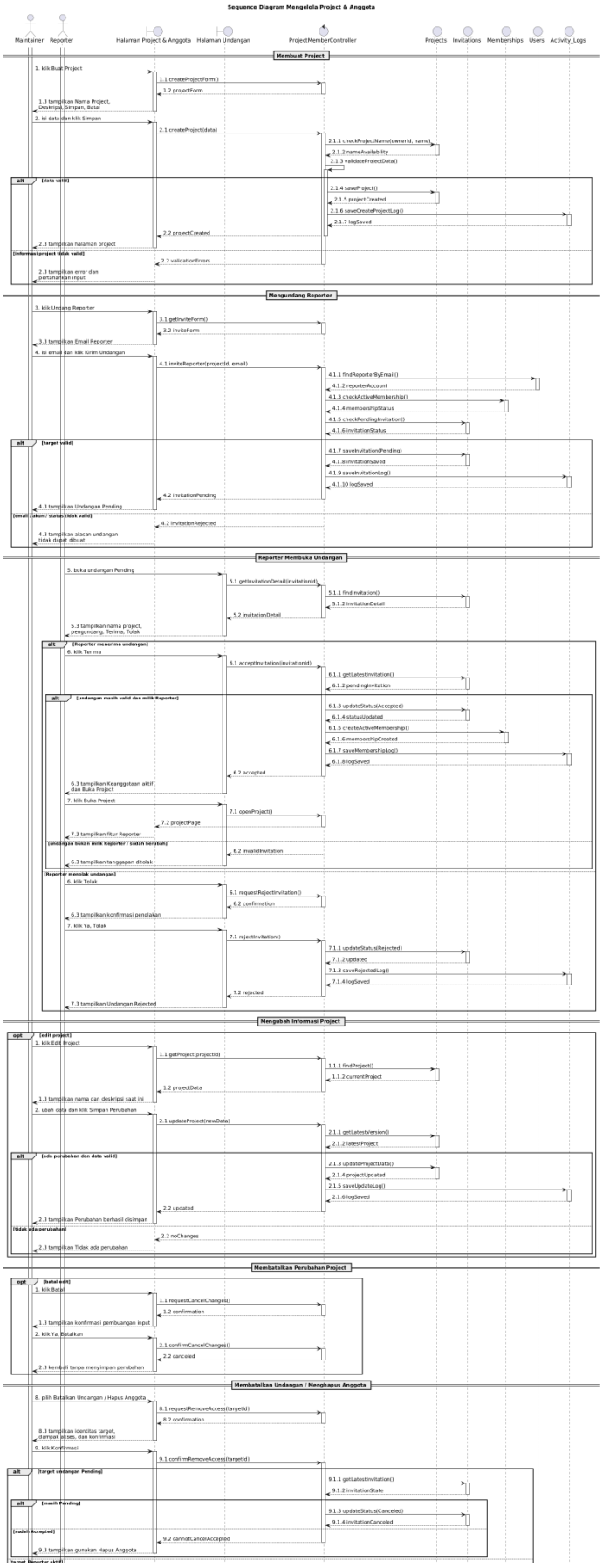


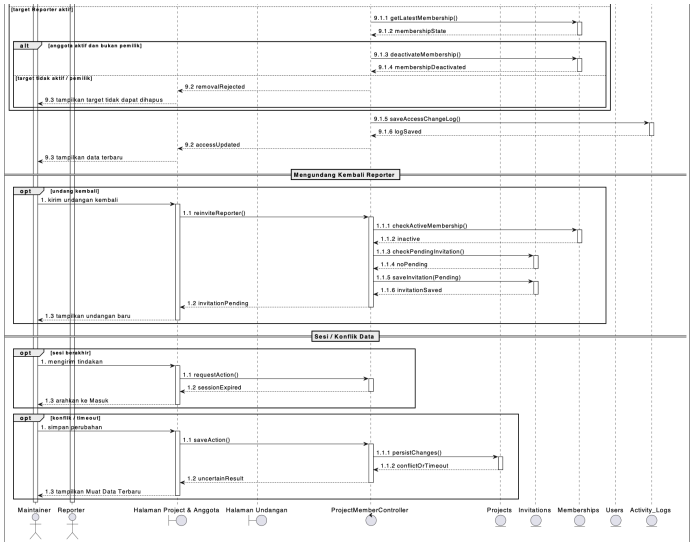
Gambar 4.6 Activity Diagram Mengelola Project & Anggota

Gambar 4.6 menggambarkan tiga swimlane: Maintainer, Sistem, dan Reporter. Maintainer memilih membuat project baru atau mengedit project yang ada. Pada kedua jalur Sistem menampilkan formulir, memvalidasi data, lalu menyimpan, mencatat log, dan membuka halaman project, atau menampilkan error bila tidak valid. Pada jalur undangan, Maintainer membuka Anggota Project, memilih Undang Reporter, dan Sistem memvalidasi undangan. Undangan valid disimpan sebagai PENDING dan dicatat. Reporter kemudian membuka undangan, dan bila menerima, Sistem mengaktifkan keanggotaan sehingga Reporter dapat membuka project. Bila menolak, status menjadi REJECTED dan tidak ada

akses. Maintainer juga dapat menghapus anggota, yang dinilai Sistem sebelum menonaktifkan keanggotaan dan mencatat log.

4.4.4 Sequence Diagram


Sequence Diagram Mengelola Project & Anggota
Maintainer
Reporter
Halaman Project & Anggota
Halaman Undangan
ProjectMemberController
Projects
Invitations
Memberships
Users
Activity_Logs
Membuat Project
1. klik Buat Project
1.1 createProjectForm()
1.2 projectForm
1.3 tampilkan Nama Project, Deskripsi, Simpan, Batal
2. isi data dan klik Simpan
2.1 createProject(data)
2.1.1 checkProjectName(ownerId, name)
2.1.2 nameAvailability
2.1.3 validateProjectData()
alt
[data valid]
2.1.4 saveProject()
2.1.5 projectCreated
2.1.6 saveCreateProjectLog()
2.1.7 logSaved
2.2 projectCreated
2.3 tampilkan halaman project
[informasi project tidak valid]
2.2 validationErrors
2.3 tampilkan error dan pertahankan input
Mengundang Reporter
3. klik Undang Reporter
3.1 getInviteForm()
3.2 inviteForm
3.3 tampilkan Email Reporter
4. isi email dan klik Kirim Undangan
4.1 inviteReporter(projectId, email)
4.1.1 findReporterByEmail()
4.1.2 reporterAccount
4.1.3 checkActiveMembership()
4.1.4 membershipStatus
4.1.5 checkPendingInvitation()
4.1.6 invitationStatus
alt
[target valid]
4.1.7 saveInvitation(Pending)
4.1.8 invitationSaved
4.1.9 saveInvitationLog()
4.1.10 logSaved
4.2 invitationPending
4.3 tampilkan Undangan Pending
[email / akun / status tidak valid]
4.2 invitationRejected
4.3 tampilkan alasan undangan tidak dapat dibuat
Reporter Membuka Undangan
5. buka undangan Pending
5.1 getInvitationDetail(invitationId)
5.1.1 findInvitation()
5.1.2 invitationDetail
5.2 invitationDetail
5.3 tampilkan nama project, pengundang, Terima, Tolak
alt
[Reporter menerima undangan]
6. klik Terima
6.1 acceptInvitation(invitationId)
6.1.1 getLatestInvitation()
6.1.2 pendingInvitation
alt
[undangan masih valid dan milik Reporter]
6.1.3 updateStatus(Accepted)
6.1.4 status(updated
6.1.5 createActiveMembership()
6.1.6 membershipCreated
6.1.7 saveMembershipLog()
6.1.8 logSaved
6.2 accepted
6.3 tampilkan Keanggotaan aktif dan Buka Project
7. klik Buka Project
7.1 openProject()
7.2 projectPage
7.3 tampilkan fitur Reporter
[undangan bukan milik Reporter / sudah berubah]
6.2 InvalidInvitation
6.3 tampilkan tanggapan ditolak
[Reporter menolak undangan]
6. klik Tolak
6.1 requestRejectInvitation()
6.2 confirmation
6.3 tampilkan konfirmasi penolakan
7. klik Ya, Tolak
7.1 rejectInvitation()
7.1.1 updateStatus(Rejected)
7.1.2 updated
7.1.3 saveRejectedLog()
7.1.4 logSaved
7.2 rejected
7.3 tampilkan Undangan Rejected
Mengubah Informasi Project
opt
[edit project]
1. klik Edit Project
1.1 getProject(projectId)
1.1.1 findProject()
1.1.2 currentProject
1.2 projectData
1.3 tampilkan nama dan deskripsi saat ini
2. ubah data dan klik Simpan Perubahan
2.1 updateProject(newData)
2.1.1 getLatestVersion()
2.1.2 latestProject
alt
[ada perubahan dan data valid]
2.1.3 updateProjectData()
2.1.4 projectUpdated
2.1.5 saveUpdateLog()
2.1.6 logSaved
2.2 updated
2.3 tampilkan Perubahan berhasil disimpan
[tidak ada perubahan]
2.2 noChanges
2.3 tampilkan Tidak ada perubahan
Membatalkan Perubahan Project
opt
[batal edit]
1. klik Batal
1.1 requestCancelChanges()
1.2 confirmation
1.3 tampilkan konfirmasi pembuangan input
2. klik Ya, Batalkan
2.1 confirmCancelChanges()
2.2 canceled
2.3 kembali tanpa menyimpan perubahan
Membatalkan Undangan / Menghapus Anggota
8. pilih Batalkan Undangan / Hapus Anggota
8.1 requestRemoveAccess(targetId)
8.2 confirmation
8.3 tampilkan identitas target, dampak akses, dan konfirmasi
9. klik Konfirmasi
9.1 confirmRemoveAccess(targetId)
alt
[target undangan Pending]
9.1.1 getLatestInvitation()
9.1.2 invitationState
alt
[masih Pending]
9.1.3 updateStatus(Canceled)
9.1.4 invitationCanceled
[sudah Accepted]
9.2 cannotCancelAccepted
9.3 tampilkan gunakan Hapus Anggota
[target bukan/ tidak]



Gambar 4.7 Sequence Diagram Mengelola Project & Anggota (dibaca dari kiri ke kanan)

Gambar 4.7 menunjukkan interaksi Maintainer, Reporter, Halaman Project & Anggota, Halaman Undangan, ProjectMemberController, Projects, Invitations, Memberships, Users, dan Activity_Logs. Bagian yang digambarkan adalah pembuatan project, pengundangan Reporter (validasi email, peran, keanggotaan aktif, dan undangan PENDING), Reporter menanggapi undangan, pengubahan informasi project, pembatalan undangan, penghapusan anggota, serta kondisi gagal atau data basi.

4.4.5 User Story

Tabel 4.18. User Story Mengelola Project & Anggota

Name (Description)	User Story
Mengelola Project & Anggota	As a Maintainer, I want to manage projects and their members so that I can control project organization and user access.

4.4.6 User Story Scenario

Tabel 4.19. User Story Scenario Mengelola Project & Anggota

Feature: Mengelola Project dan Anggota

In order to mengelola workspace project
As a Maintainer
I want to membuat project dan mengelola anggota pada project milik saya

Background:
Given I am logged in as "elsa@dupissue.test" with role "Maintainer"

Scenario: Maintainer membuat project dan mengundang anggota
Given I am on "/projects"
When I press "Buat Project"
Then I should see "Nama Project"
And I should see "Deskripsi"
When I fill in "Nama Project" with "Toko Online Baju"
And I fill in "Deskripsi" with "Aplikasi toko baju online"
And I press "Simpan"
Then I should see "Project berhasil dibuat"
And I should be on "/projects/1"
When I follow "Anggota Project"
And I press "Undang Anggota"
Then I should see "Email Anggota"
When I fill in "Email Anggota" with "valentino@dupissue.test"
And I press "Kirim Undangan"
Then I should see "Pending" in the "invitation_status" element

Scenario: Maintainer mengubah informasi project
Given I am on "/projects/1/settings"
When I press "Edit Project"
Then I should see "Toko Online Baju"
When I fill in "Nama Project" with "Toko Online Baju V2"
And I press "Simpan Perubahan"
Then I should see "Perubahan berhasil disimpan"
And I should see "Toko Online Baju V2"

Scenario: Maintainer membatalkan undangan Pending
Given I am on "/projects/1/members"
And I should see "Pending" in the "invitation_status" element
When I press "Batalkan Undangan"
Then I should see "Konfirmasi pembatalan undangan"
When I press "Konfirmasi"
Then I should see "Canceled" in the "invitation_status" element

Scenario: Maintainer menghapus anggota aktif
Given I am on "/projects/1/members"
When I press "Hapus Anggota"
Then I should see "dampak pencabutan akses"
When I press "Konfirmasi"
Then I should see "Keanggotaan dinonaktifkan"
And I should see "riwayat tetap tersimpan"

Scenario: Maintainer mengundang kembali anggota
Given I am on "/projects/1/members"
When I press "Undang Anggota"
And I fill in "Email Anggota" with "fakhri@dupissue.test"

```
  And I press "Kirim Undangan"
  Then I should see "Pending" in the "invitation_status" element

Scenario Outline: Informasi project tidak valid
  Given I am on "/projects/new"
  When I fill in "Nama Project" with "<nama>"
  And I fill in "Deskripsi" with "<deskripsi>"
  And I press "Simpan"
  Then I should see "<pesan>"
  And I should see "Nama Project" in the "error" element

  Examples:
    | nama           | deskripsi         | pesan                    |
    |                | Deskripsi valid   | Nama wajib 3-100 karakter  |
    | ab             | Deskripsi valid   | Nama wajib 3-100 karakter  |
    | Toko Online Baju | Deskripsi valid | Nama sudah digunakan       |

Scenario Outline: Undangan tidak dapat dibuat
  Given I am on "/projects/1/members"
  When I press "Undang Anggota"
  And I fill in "Email Anggota" with "<email>"
  And I press "Kirim Undangan"
  Then I should see "<pesan>"

  Examples:
    | email                  | pesan                                          |
    | bukan-email            | Format email tidak valid                       |
    | joshua@dupissue.test   | Akun bukan Reporter                            |
    | valentino@dupissue.test | Sudah menjadi anggota atau memiliki undangan Pending |
```

4.4.7 Functional Requirements

- **REQ-03.1:** Sistem harus memeriksa bahwa pengguna berperan Maintainer sebelum menampilkan formulir pembuatan project dan sebelum menyimpan project.
- **REQ-03.2:** Sistem harus memvalidasi nama project (3 sampai 100 karakter dan tidak duplikat, cakupan keunikan lihat Appendix C) dan deskripsi (maksimal 1.000 karakter).
- **REQ-03.3:** Sistem harus menetapkan Maintainer pembuat sebagai pemilik project (projects.owner_id) dan mencatat log pembuatan project.
- **REQ-03.4:** Maintainer hanya boleh mengubah informasi project miliknya. Setiap perubahan harus divalidasi ulang dan dicatat pada log aktivitas.
- **REQ-03.5:** Undangan harus dikirim berdasarkan email dan hanya diterima bila format email valid, akun terdaftar dengan peran Reporter, serta belum ada keanggotaan aktif atau undangan PENDING pada project yang sama.
- **REQ-03.6:** Undangan baru harus berstatus PENDING dan belum memberikan akses ke project sebelum berstatus ACCEPTED.

- **REQ-03.7:** Maintainer harus dapat membatalkan undangan PENDING dengan konfirmasi, dan status berubah menjadi CANCELED.
- **REQ-03.8:** Maintainer harus dapat menghapus anggota aktif dengan konfirmasi yang menjelaskan dampak pencabutan akses. Sistem menonaktifkan keanggotaan (is_active 0 dan removed_at terisi) tanpa menghapus issue, vote, atau histori anggota tersebut.
- **REQ-03.9:** Maintainer harus dapat mengundang kembali email yang pernah dikeluarkan atau undangannya dibatalkan melalui pembuatan undangan baru.
- **REQ-03.10:** Reporter harus dapat melihat undangan masuk, serta menerima atau menolak undangan PENDING. Penerimaan mengaktifkan keanggotaan dan penolakan mengubah status menjadi REJECTED. Waktu tanggapan dicatat pada responded_at.
- **REQ-03.11:** Sistem tidak boleh menggandakan undangan atau keanggotaan, dan harus menampilkan alasan penolakan bila undangan tidak dapat dibuat.
- **REQ-03.12:** Pembuatan project, perubahan project, undangan, pembatalan, dan penghapusan anggota harus dicatat pada activity_logs.

4.5 System Feature 4: Mengelola Sumber Issue (REQ-04)

4.5.1 Description and Priority

Maintainer mengimpor issue ke project miliknya dari dataset CSV atau repository GitHub. Data divalidasi dan ditampilkan sebagai pratinjau sebelum disimpan, hanya baris valid yang belum pernah diimpor yang disimpan, dan deteksi duplikasi dijadwalkan setelahnya.

4.5.2 Use Case Specification

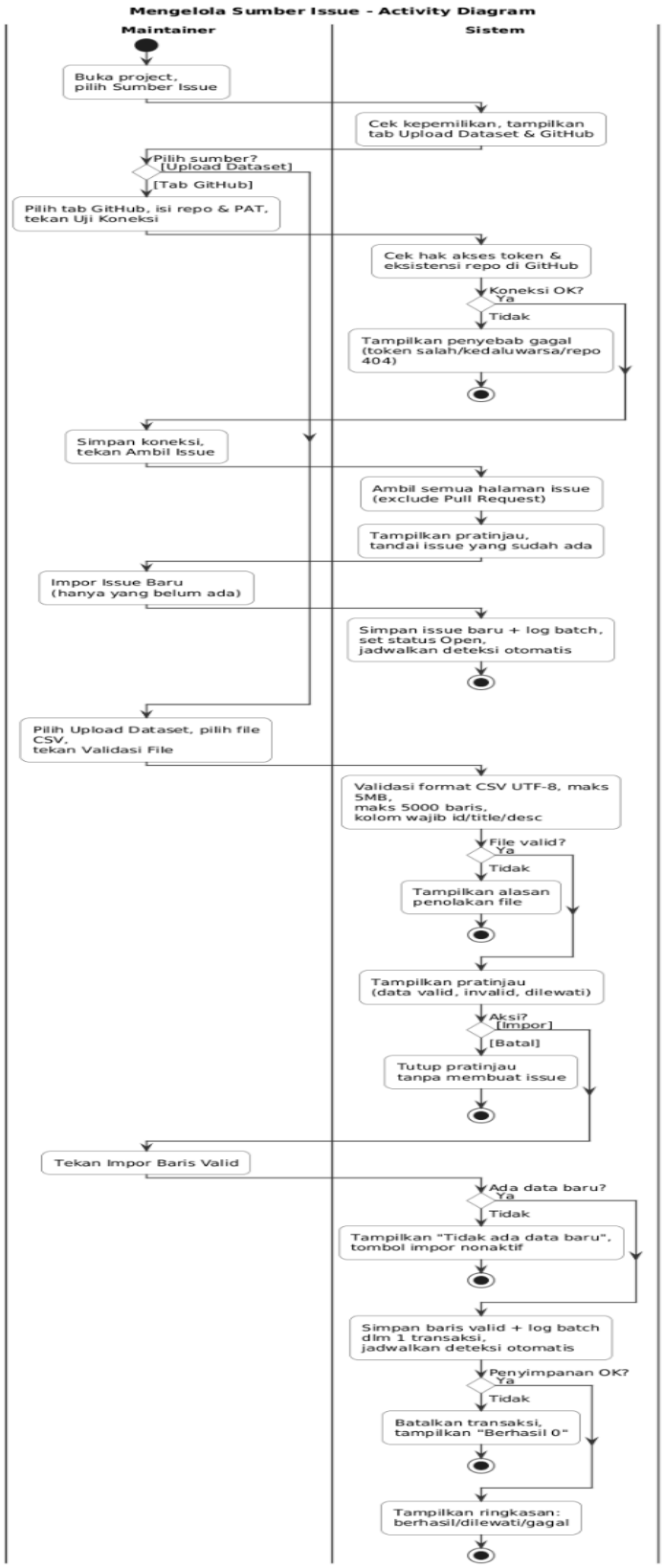
Tabel 4.20. Deskripsi Use Case Specification "Mengelola Sumber Issue"

Nama Use Case	Mengelola Sumber Issue
Aktor	Maintainer
Deskripsi	Maintainer mengimpor issue dari dataset CSV atau repository GitHub ke project yang dikelola
Kondisi Awal	Maintainer sudah masuk dan merupakan pemilik project serta sumber data yang diperlukan tersedia
Kondisi Akhir	Issue valid baru tersimpan satu kali dengan asal data yang sesuai dan sistem menampilkan hasil impor
Skenario Normal	
Aktor	**Sistem**
1. Maintainer membuka project dan memilih Sumber Issue	

	2. Sistem memeriksa kepemilikan project
	3. Sistem menampilkan tab Upload Dataset dan GitHub
4. Maintainer memilih Upload Dataset	
5. Maintainer memilih file CSV	
6. Maintainer menekan Validasi File	
	7. Sistem memeriksa format CSV UTF-8
	8. Sistem memeriksa ukuran file maksimal 5 MB
	9. Sistem memeriksa maksimal 5000 baris data
	10. Sistem memeriksa kolom source_id title dan description
	11. Sistem menampilkan pratinjau hasil validasi
	12. Sistem menampilkan jumlah data valid invalid dan pernah diimpor
13. Maintainer menekan Impor Baris Valid	
	14. Sistem memeriksa kembali akses dan identitas sumber
	15. Sistem menyimpan baris valid baru dalam satu transaksi
	16. Sistem mencatat log batch impor
	17. Sistem menjadwalkan deteksi duplikasi
	18. Sistem menampilkan ringkasan hasil impor
Skenario Alternatif	
A1. Menghubungkan GitHub dan mengimpor issue	
1.a. Maintainer memilih tab GitHub	
	2.a. Sistem menampilkan kolom Repository dan PAT
3.a. Maintainer mengisi repository dan PAT	
4.a. Maintainer menekan Uji Koneksi	
	5.a. Sistem menguji hak baca repository
	6.a. Sistem menampilkan hasil koneksi berhasil
7.a. Maintainer menekan Simpan Koneksi	
	8.a. Sistem menyimpan konfigurasi secara terlindungi
	9.a. Sistem menampilkan tombol Ambil Issue
10.a. Maintainer menekan Ambil Issue	
	11.a. Sistem mengambil issue dan mengecualikan pull request
	12.a. Sistem menampilkan pratinjau issue baru
13.a. Maintainer menekan Impor Issue Baru	
	14.a. Sistem menyimpan issue valid baru
	15.a. Sistem mencatat URL dan nomor sumber
	16.a. Sistem menjadwalkan deteksi duplikasi
	17.a. Sistem menampilkan hasil impor
A2. Sebagian data tidak valid atau seluruhnya sudah ada	
1.b. Maintainer memeriksa pratinjau data	
	2.b. Sistem menampilkan alasan data yang dilewati
	3.b. Sistem mengaktifkan impor hanya untuk data valid baru
4.b. Maintainer mengganti file CSV	
5.b. Maintainer menekan Validasi File	
	6.b. Sistem mengganti pratinjau dengan hasil validasi baru
A3. Memperbarui koneksi GitHub	
1.c. Maintainer menekan Edit Koneksi	

2.c. Maintainer mengisi repository atau PAT baru	
3.c. Maintainer menekan Uji Koneksi	
	4.c. Sistem menguji konfigurasi baru tanpa mengganti konfigurasi lama
5.c. Maintainer menekan Simpan Koneksi	
	6.c. Sistem mengganti konfigurasi aktif
Skenario Exception	
E1. File tidak memenuhi ketentuan	
1.e. Maintainer menekan Validasi File untuk file yang tidak sesuai	
	2.e. Sistem menolak file sebelum proses impor
	3.e. Sistem menampilkan alasan penolakan
4.e. Maintainer memilih file yang sesuai	
5.e. Maintainer menekan Validasi File	
	6.e. Sistem menampilkan pratinjau setelah file valid
E2. Koneksi atau pengambilan GitHub gagal	
1.f. Maintainer menekan Uji Koneksi atau Ambil Issue saat konfigurasi bermasalah	
	2.f. Sistem menampilkan penyebab kegagalan
	3.f. Sistem membatalkan pratinjau yang belum lengkap
4.f. Maintainer memperbaiki konfigurasi atau mengulangi pengambilan	
	5.f. Sistem mengulangi pengujian atau pengambilan data
	6.f. Sistem tidak menyimpan issue sebelum impor dikonfirmasi
E3. Impor atau deteksi gagal	
1.g. Maintainer mengonfirmasi impor saat penyimpanan batch gagal	
	2.g. Sistem membatalkan seluruh penyimpanan batch
	3.g. Sistem menampilkan jumlah berhasil nol
4.g. Maintainer membuka hasil impor saat deteksi gagal	
	5.g. Sistem mempertahankan issue yang sudah tersimpan
	6.g. Sistem menetapkan status deteksi Failed

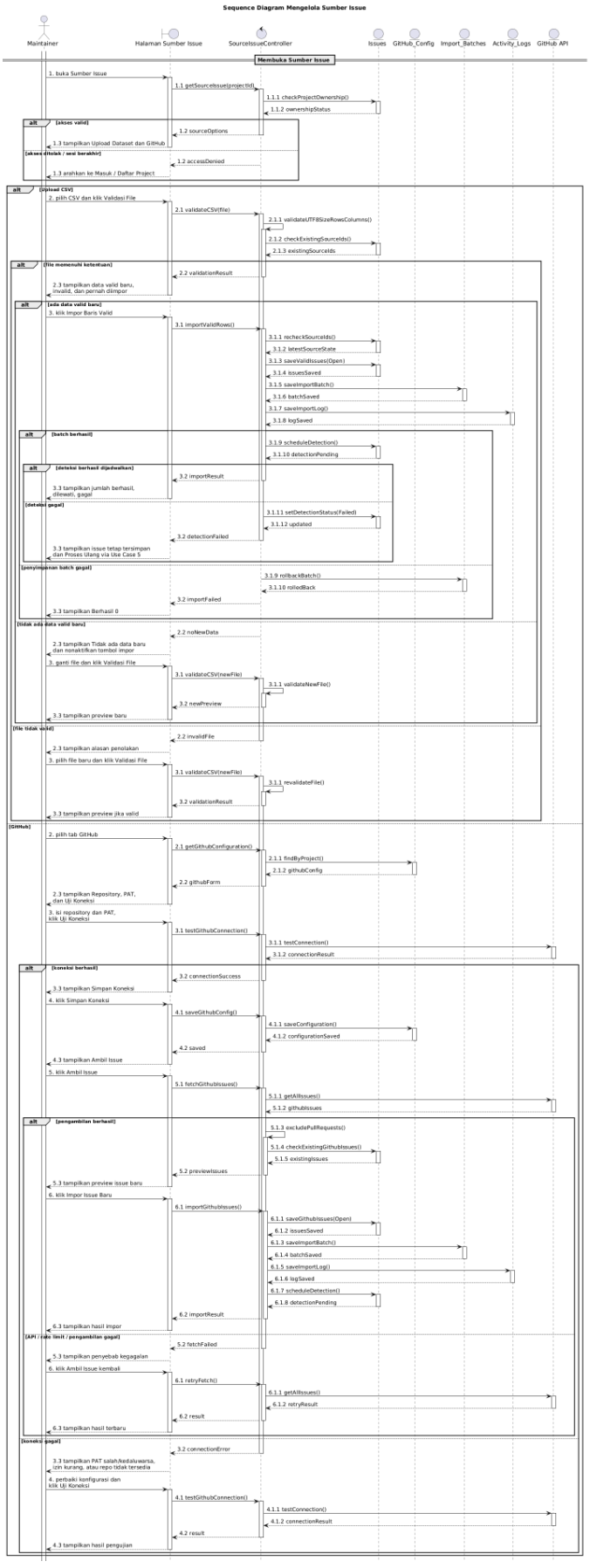
4.5.3 Activity Diagram

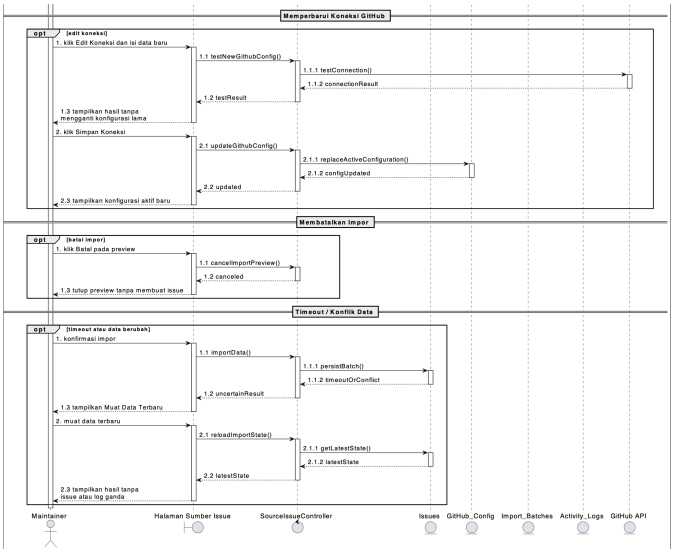

Mengelola Sumber Issue - Activity Diagram
Maintainer
Sistem
Buka project, pilih Sumber Issue
Cek kepemilikan, tampilkan tab Upload Dataset & GitHub
Pilih sumber?
[Upload Dataset]
[Tab GitHub]
Pilih tab GitHub, isi repo & PAT, tekan Uji Koneksi
Cek hak akses token & eksistensi repo di GitHub
Koneksi OK?
Ya
Tidak
Tampilkan penyebab gagal (token salah/kedaluwarsa/repo 404)
Simpan koneksi, tekan Ambil Issue
Ambil semua halaman issue (exclude Pull Request)
Tampilkan pratinjau, tandai issue yang sudah ada
Impor Issue Baru (hanya yang belum ada)
Simpan issue baru + log batch, set status Open, jadwalkan deteksi otomatis
Pilih Upload Dataset, pilih file CSV, tekan Validasi File
Validasi format CSV UTF-8, maks 5MB, maks 5000 baris, kolom wajib id/title/desc
File valid?
Ya
Tidak
Tampilkan alasan penolakan file
Tampilkan pratinjau (data valid, invalid, dilewati)
Aksi?
[Impor]
[Batal]
Tutup pratinjau tanpa membuat issue
Tekan Impor Baris Valid
Ada data baru?
Ya
Tidak
Tampilkan "Tidak ada data baru", tombol impor nonaktif
Simpan baris valid + log batch dlm 1 transaksi, jadwalkan deteksi otomatis
Penyimpanan OK?
Ya
Tidak
Batalkan transaksi, tampilkan "Berhasil 0"
Tampilkan ringkasan: berhasil/dilewati/gagal

Gambar 4.8 Activity Diagram Mengelola Sumber Issue

Gambar 4.8 menggambarkan alur Maintainer membuka Sumber Issue dan memilih sumber. Pada jalur GitHub, Maintainer mengisi repository dan PAT lalu menguji koneksi. Sistem memeriksa hak akses token dan keberadaan repository, dan bila gagal menampilkan penyebab. Bila berhasil, koneksi disimpan, issue diambil dari semua halaman (kecuali pull request), pratinjau menandai issue yang sudah ada, dan impor hanya menyimpan issue baru. Pada jalur Upload Dataset, Sistem memvalidasi format UTF-8, ukuran, jumlah baris, dan kolom wajib, menampilkan pratinjau, lalu Maintainer memilih impor atau batal. Sistem memeriksa ketersediaan data baru, menyimpan baris valid dalam satu transaksi beserta log batch, menjadwalkan deteksi otomatis, dan menampilkan ringkasan. Jika penyimpanan gagal, transaksi dibatalkan dan ditampilkan "Berhasil 0".

4.5.4 Sequence Diagram


Sequence Diagram Mengelola Sumber Issue
Maintainer
Halaman Sumber Issue
SourceIssueController
Issues
GitHub_Config
Import_Batches
Activity_Logs
GitHub API
Membuka Sumber Issue
1. buka Sumber Issue
1.1 getSourceIssue(projectId)
1.1.1 checkProjectOwnership()
1.1.2 ownershipStatus
alt [akses valid]
1.2 sourceOptions
1.3 tampilkan Upload Dataset dan GitHub
[akses ditolak / sesi berakhir]
1.2 accessDenied
1.3 arahkan ke Masuk / Daftar Project
alt [Upload CSV]
2. pilih CSV dan klik Validasi File
2.1 validateCSV(file)
2.1.1 validateUTF8SizeRowsColumns()
2.1.2 checkExistingSourceIds()
2.1.3 existingSourceIds
alt [file memenuhi ketentuan]
2.2 validationResult
2.3 tampilkan data valid baru, invalid, dan pernah diimpor
alt [ada data valid baru]
3. klik Impor Baris Valid
3.1 importValidRows()
3.1.1 recheckSourceIds()
3.1.2 latestSourceState
3.1.3 saveValidIssues(Open)
3.1.4 issuesSaved
3.1.5 saveImportBatch()
3.1.6 batchSaved
3.1.7 saveImportLog()
3.1.8 logSaved
alt [batch berhasil]
3.1.9 scheduleDetection()
3.1.10 detectionPending
alt [deteksi berhasil dijadwalkan]
3.2 importResult
3.3 tampilkan jumlah berhasil, dilewati, gagal
[deteksi gagal]
3.1.11 setDetectionStatus(Failed)
3.1.12 updated
3.2 detectionFailed
3.3 tampilkan issue tetap tersimpan dan Proses Ulang via Use Case 5
[penyimpanan batch gagal]
3.1.9 rollbackBatch()
3.1.10 rolledBack
3.2 importFailed
3.3 tampilkan Berhasil 0
[tidak ada data valid baru]
2.2 noNewData
2.3 tampilkan Tidak ada data baru dan nonaktifkan tombol impor
3. ganti file dan klik Validasi File
3.1 validateCSV(newFile)
3.1.1 validateNewFile()
3.2 newPreview
3.3 tampilkan preview baru
[file tidak valid]
2.2 invalidFile
2.3 tampilkan alasan penolakan
3. pilih file baru dan klik Validasi File
3.1 validateCSV(newFile)
3.1.1 revalidateFile()
3.2 validationResult
3.3 tampilkan preview jika valid
[GitHub]
2. pilih tab GitHub
2.1 getGithubConfiguration()
2.1.1 findByProject()
2.1.2 githubConfig
2.2 githubForm
2.3 tampilkan Repository, PAT, dan Uji Koneksi
3. isi repository dan PAT, klik Uji Koneksi
3.1 testGithubConnection()
3.1.1 testConnection()
3.1.2 connectionResult
alt [koneksi berhasil]
3.2 connectionSuccess
3.3 tampilkan Simpan Koneksi
4. klik Simpan Koneksi
4.1 saveGithubConfig()
4.1.1 saveConfiguration()
4.1.2 configurationSaved
4.2 saved
4.3 tampilkan Ambil Issue
5. klik Ambil Issue
5.1 fetchGithubIssues()
5.1.1 getAllIssues()
5.1.2 githubIssues
alt [pengambilan berhasil]
5.1.3 excludePullRequests()
5.1.4 checkExistingGithubIssues()
5.1.5 existingIssues
5.2 previewIssues
5.3 tampilkan preview issue baru
6. klik Impor Issue Baru
6.1 importGithubIssues()
6.1.1 saveGithubIssues(Open)
6.1.2 issuesSaved
6.1.3 saveImportBatch()
6.1.4 batchSaved
6.1.5 saveImportLog()
6.1.6 logSaved
6.1.7 scheduleDetection()
6.1.8 detectionPending
6.2 importResult
6.3 tampilkan hasil impor
[API / rate limit / pengambilan gagal]
5.2 fetchFailed
5.3 tampilkan penyebab kegagalan
6. klik Ambil Issue kembali
6.1 retryFetch()
6.1.1 getAllIssues()
6.1.2 retryResult
6.2 result
6.3 tampilkan hasil terbaru
[koneksi gagal]
3.2 connectionError
3.3 tampilkan PAT salah/kedaluwarsa, izin kurang, atau repo tidak tersedia
4. perbaiki konfigurasi dan klik Uji Koneksi
4.1 testGithubConnection()
4.1.1 testConnection()
4.1.2 connectionResult
4.2 result
4.3 tampilkan hasil pengujian



Gambar 4.9 Sequence Diagram Mengelola Sumber Issue (dibaca dari kiri ke kanan)

Gambar 4.9 menunjukkan interaksi Maintainer, Halaman Sumber Issue, SourceIssueController, Issues, GitHub_Config, Import_Batches, Activity_Logs, dan GitHub API. Controller memeriksa kepemilikan project, memvalidasi file CSV, menghitung baris valid dan duplikat, menyimpan batch dalam transaksi, menguji koneksi GitHub, menyimpan konfigurasi, mengambil issue, serta menangani pembatalan impor, pembaruan koneksi, dan kegagalan penyimpanan.

4.5.5 User Story

Tabel 4.24. User Story Mengelola Sumber Issue

Name (Description)	User Story
Mengelola Sumber Issue	As a Maintainer, I want to manage issue sources by uploading datasets or integrating GitHub repositories so that issue data can be added to the project efficiently.

4.5.6 User Story Scenario

Tabel 4.25. User Story Scenario Mengelola Sumber Issue

```
Feature: Mengelola Sumber Issue
  In order to memuat issue dari sumber eksternal
  As a Maintainer
  I want to mengimpor issue dari CSV dan GitHub ke project milik saya

  Background:
    Given I am logged in as "elsa@dupissue.test" with role "Maintainer"
    And I own project "Toko Online Baju" with id "1"
    And I am on "/projects/1/sources"

  Scenario: Maintainer mengimpor CSV valid
    When I press "Upload Dataset"
    Then I should see "Pilih file CSV"
    When I attach the file "features/fixtures/valid_100_rows.csv" to "file"
    And I press "Validasi File"
    Then I should see "Pratinjau" in the "preview" element
    And I should see "Jumlah dilewati: 0"
    When I press "Impor Baris Valid"
    Then I should see "Impor berhasil"
    And I should see "Open" in the "status" element
    And I should see "Pending" in the "detection_status" element

  Scenario: Maintainer menghubungkan GitHub dan mengimpor issue
    When I press "GitHub"
    Then I should see "Repository"
    And I should see "PAT"
    When I fill in "repository" with "kelompok4/toko-baju"
    And I fill in "PAT" with "ghp_valid_token"
    And I press "Uji Koneksi"
    Then I should see "Koneksi berhasil"
    When I press "Simpan Koneksi"
    Then I should see "Konfigurasi tersimpan"
    When I press "Ambil Issue"
    Then I should see "Pratinjau GitHub"
    When I press "Impor Issue Baru"
    Then I should see "Impor GitHub berhasil"
    And I should see "Pending" in the "detection_status" element

  Scenario: Maintainer mengganti file CSV sebelum impor
    When I attach the file "features/fixtures/valid.csv" to "file"
    And I press "Validasi File"
    Then I should see "Pratinjau" in the "preview" element
    When I attach the file "features/fixtures/valid_baru.csv" to "file"
    And I press "Validasi File"
    Then I should see "Pratinjau baru"
    And I should not see "Impor berhasil"

  Scenario: Maintainer memperbarui koneksi GitHub
    When I press "GitHub"
    And I press "Edit Koneksi"
    And I fill in "repository" with "kelompok4/toko-baju-v2"
    And I press "Uji Koneksi"
    Then I should see "Koneksi berhasil"
```

```
  When I press "Simpan Koneksi"
  Then I should see "Konfigurasi aktif diganti"

Scenario Outline: File CSV tidak memenuhi ketentuan
  When I attach the file "<file>" to "file"
  And I press "Validasi File"
  Then I should see "<pesan>"
  And I should not see "Pratinjau" in the "preview" element

  Examples:
    | file                          | pesan                                  |
    | features/fixtures/rusak.csv   | File rusak                             |
    | features/fixtures/bukan_utf8.csv | Bukan CSV UTF-8                     |
    | features/fixtures/kosong.csv  | File kosong                            |
    | features/fixtures/besar_6mb.csv | Melebihi batas 5MB                   |
    | features/fixtures/tanpa_kolom.csv | Tanpa kolom wajib source_id,title,description |
    | features/fixtures/6000_baris.csv | Melebihi 5000 baris                 |

Scenario Outline: Koneksi GitHub gagal
  When I press "GitHub"
  And I fill in "repository" with "<repo>"
  And I fill in "PAT" with "<pat>"
  And I press "Uji Koneksi"
  Then I should see "<pesan>"

  Examples:
    | repo                | pat          | pesan                    |
    | kelompok4/toko-baju | salah_token  | PAT salah                |
    | kelompok4/toko-baju | expired_token | Token kedaluwarsa       |
    | kelompok4/private   | valid_token  | Izin kurang              |
    | tidak_ada/repo      | valid_token  | Repository tidak tersedia |

Scenario: Penyimpanan batch gagal
  When I attach the file "features/fixtures/valid_100_rows.csv" to "file"
  And I press "Validasi File"
  And I press "Impor Baris Valid"
  Then I should see "Berhasil 0"
  And I should not see "Impor berhasil"

Scenario: Deteksi gagal setelah impor berhasil
  Given I am on "/projects/1/sources/import-result/1"
  And I should see "Impor berhasil"
  Then I should see "Failed" in the "detection_status" element
  And I should see "Data tidak perlu diimpor ulang"
```

4.5.7 Functional Requirements

- **REQ-04.1:** Sistem harus memeriksa bahwa pengguna adalah Maintainer pemilik project sebelum menampilkan halaman Sumber Issue dan sekali lagi sebelum menyimpan data impor.

- **REQ-04.2:** Halaman Sumber Issue harus menyediakan dua tab, yaitu Upload Dataset dan GitHub.
- **REQ-04.3:** Sistem harus menolak file CSV yang rusak, kosong, bukan UTF-8, lebih dari 5 MB, lebih dari 5.000 baris, atau tidak memiliki kolom source_id, title, dan description, dengan menampilkan alasan penolakan.
- **REQ-04.4:** Sistem harus menampilkan pratinjau hasil validasi berisi jumlah data valid, invalid, dan sudah pernah diimpor, beserta alasan setiap baris yang dilewati.
- **REQ-04.5:** Sistem harus mencegah penyimpanan issue yang sama akibat impor berulang dengan mengenali identitas sumber (source_id untuk CSV, repository dan nomor issue untuk GitHub) pada project yang sama.
- **REQ-04.6:** Impor harus menyimpan seluruh baris valid baru dalam satu transaksi. Bila penyimpanan gagal, seluruh batch harus dibatalkan dan sistem menampilkan jumlah berhasil nol.
- **REQ-04.7:** Issue hasil impor harus disimpan dengan source_type CSV atau GITHUB, status OPEN (kecuali kolom status opsional disepakati, lihat Appendix C), detection_status PENDING, dan author_id kosong.
- **REQ-04.8:** Setiap kegiatan impor harus dicatat pada import_batches (pelaksana, jumlah total, valid, dilewati, dan status) serta activity_logs.
- **REQ-04.9:** Sistem harus menjadwalkan deteksi duplikasi melalui Laravel Database Queue setelah impor berhasil. Kegagalan deteksi tidak boleh menghapus issue dan menetapkan detection_status FAILED tanpa perlu impor ulang.
- **REQ-04.10:** Uji Koneksi GitHub harus memeriksa hak baca repository dan membedakan kegagalan: PAT salah, token kedaluwarsa, izin kurang, dan repository tidak tersedia.
- **REQ-04.11:** Sistem hanya boleh menyimpan satu konfigurasi GitHub aktif per project. PAT harus disimpan terenkripsi, tidak ditampilkan kembali kepada browser, dan tidak dicatat pada log.
- **REQ-04.12:** Pembaruan koneksi harus diuji terlebih dahulu tanpa mengganti konfigurasi lama sampai Maintainer menekan "Simpan Koneksi".
- **REQ-04.13:** Pengambilan issue GitHub harus mencakup seluruh halaman hasil, mengecualikan pull request, dan menampilkan pratinjau yang menandai issue yang sudah ada.
- **REQ-04.14:** Issue GitHub yang diimpor harus menyimpan nama repository, nomor issue, dan URL sumber, serta memperbarui last_synced_at.
- **REQ-04.15:** Sistem tidak boleh menyimpan issue apa pun sebelum Maintainer mengonfirmasi impor. Pembatalan pratinjau tidak boleh membuat issue.

- **REQ-04.16:** Gangguan pada GitHub tidak boleh menghentikan impor CSV maupun pengelolaan issue lokal.

4.6 System Feature 5: Review & Validasi Duplikat Issue (REQ-05)

4.6.1 Description and Priority

Maintainer membandingkan issue dengan kandidat hasil NLP secara berdampingan, lalu menetapkan pasangan sebagai duplikat (dengan memilih issue induk) atau bukan duplikat. Maintainer juga dapat memproses ulang deteksi yang gagal, mengoreksi penandaan duplikat, dan meninjau kembali pasangan yang pernah ditolak. Skor NLP hanya rekomendasi, keputusan akhir ada pada Maintainer (human-in-the-loop).

4.6.2 Use Case Specification

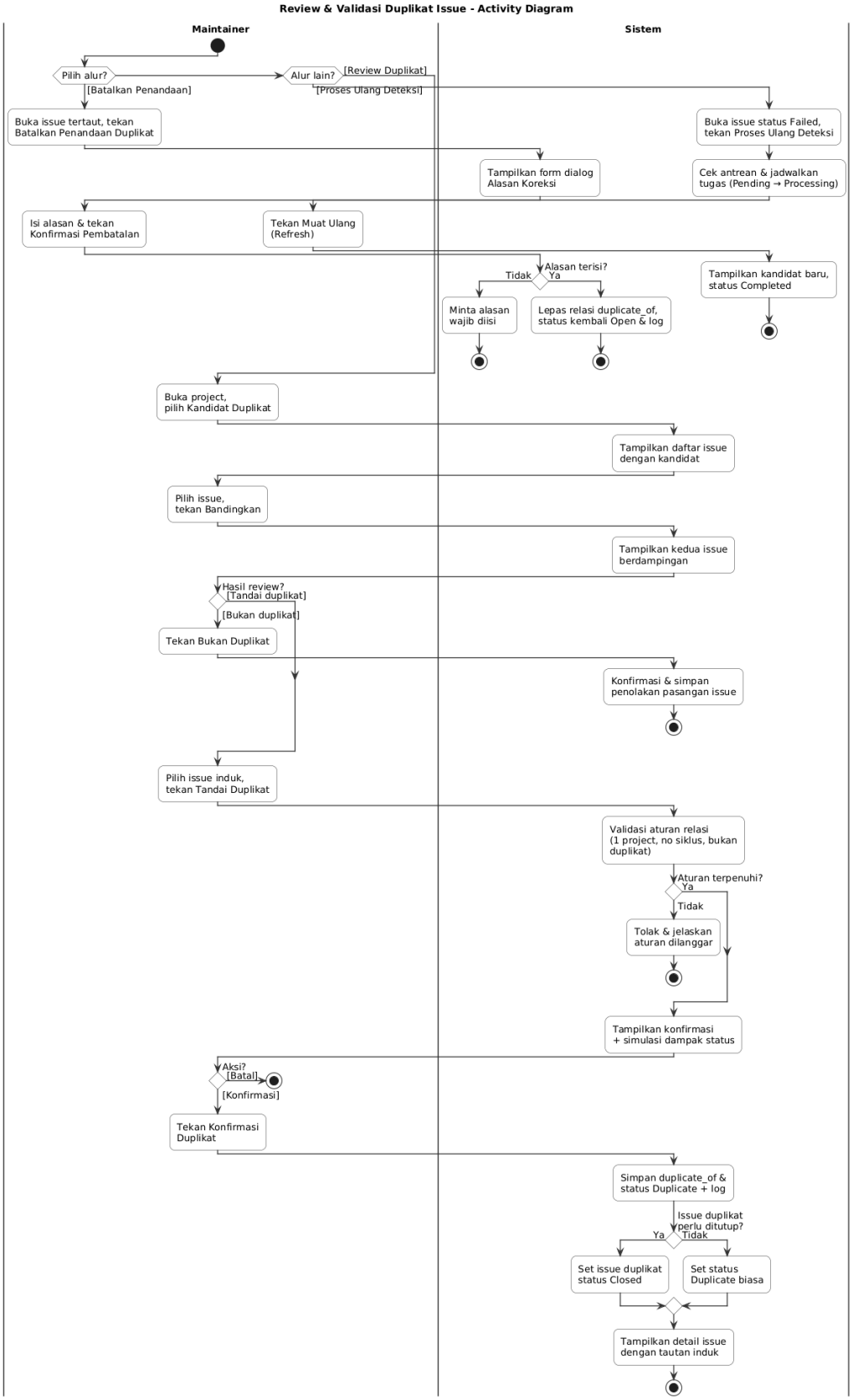
Tabel 4.26. Deskripsi Use Case Specification "Review & Validasi Duplikat Issue"

Nama Use Case	Review & Validasi Duplikat Issue
Aktor	Maintainer
Deskripsi	Maintainer membandingkan kandidat, menentukan duplikat atau bukan duplikat, menetapkan induk, dan mengoreksi keputusan
Kondisi Awal	Maintainer sudah masuk dan merupakan pemilik project serta kandidat duplikat sudah tersedia untuk ditinjau
Kondisi Akhir	Keputusan pasangan tersimpan dan relasi duplikat diperbarui tanpa menggabungkan konten atau histori issue
Skenario Normal	
Aktor	**Sistem**
1. Maintainer membuka project dan memilih Kandidat Duplikat	
	2. Sistem menampilkan daftar issue yang mempunyai kandidat
3. Maintainer memilih issue	
4. Maintainer menekan Bandingkan	
	5. Sistem menampilkan detail kedua issue
6. Maintainer menekan Tandai Duplikat	
7. Maintainer memilih issue induk	
	8. Sistem memvalidasi relasi kedua issue
	9. Sistem menampilkan konfirmasi penandaan duplikat
10. Maintainer menekan Konfirmasi Duplikat	
	11. Sistem memeriksa versi data terbaru
	12. Sistem menyimpan keputusan dan duplicate_of
	13. Sistem mengubah status issue menjadi Duplicate
	14. Sistem mencatat log keputusan

	15. Sistem menampilkan detail issue dengan tautan induk
Skenario Alternatif	
A1. Kandidat bukan duplikat	
1.a. Maintainer menekan Bukan Duplikat	
	2.a. Sistem menampilkan konfirmasi penolakan pasangan
3.a. Maintainer menekan Konfirmasi Bukan Duplikat	
	4.a. Sistem menyimpan keputusan pasangan
	5.a. Sistem mencatat log keputusan
	6.a. Sistem mempertahankan status penanganan issue
	7.a. Sistem menampilkan kembali daftar kandidat
A2. Kandidat belum tersedia atau deteksi gagal	
1.b. Maintainer membuka daftar kandidat issue	
	2.b. Sistem menampilkan status deteksi terbaru
3.b. Maintainer menekan Proses Ulang pada status Failed	
	4.b. Sistem memeriksa bahwa tidak ada proses aktif
	5.b. Sistem menjadwalkan deteksi ulang
	6.b. Sistem mengubah status deteksi menjadi Pending
7.b. Maintainer menekan Muat Ulang setelah proses selesai	
	8.b. Sistem menampilkan kandidat terbaru
	9.b. Sistem mempertahankan keputusan manual yang sudah tersimpan
A3. Induk sudah Resolved atau Closed	
1.c. Maintainer mengonfirmasi duplikat pada induk Resolved atau Closed	
	2.c. Sistem menyimpan relasi duplicate_of
	3.c. Sistem mengubah issue duplikat menjadi Closed
A4. Mengoreksi penandaan duplikat	
1.d. Maintainer membuka issue yang sudah tertaut	
2.d. Maintainer menekan Batalkan Penandaan Duplikat	
	3.d. Sistem menampilkan induk aktif dan kolom Alasan Koreksi
4.d. Maintainer mengisi alasan koreksi	
5.d. Maintainer menekan Konfirmasi Pembatalan	
	6.d. Sistem memeriksa relasi terbaru
	7.d. Sistem menghapus relasi duplicate_of
	8.d. Sistem mengubah status issue menjadi Open
	9.d. Sistem mencatat alasan dan keputusan lama
A5. Meninjau kembali pasangan yang pernah ditolak	
1.e. Maintainer membuka pasangan yang pernah dinyatakan Bukan Duplikat	
2.e. Maintainer menekan Bandingkan	
	3.e. Sistem menampilkan pasangan dan histori keputusan
4.e. Maintainer melakukan validasi baru	
Skenario Exception	
E1. Relasi atau status issue tidak valid	
1.g. Maintainer memilih induk atau calon duplikat yang tidak valid	

	2.g. Sistem menolak tindakan
	3.g. Sistem menampilkan aturan relasi yang harus dipenuhi
4.g. Maintainer memilih relasi yang valid	
	5.g. Sistem memvalidasi ulang relasi
	6.g. Sistem menampilkan konfirmasi validasi
E2. Alasan koreksi kosong	
1.h. Maintainer menekan Konfirmasi Pembatalan tanpa mengisi alasan koreksi	
	2.h. Sistem menolak penyimpanan koreksi
	3.h. Sistem meminta alasan koreksi
4.h. Maintainer mengisi alasan koreksi	
5.h. Maintainer menekan Konfirmasi Pembatalan	
	6.h. Sistem memvalidasi dan menyimpan koreksi
E3. Pemrosesan ulang tidak tersedia atau gagal	
1.i. Maintainer menekan Proses Ulang saat deteksi masih aktif atau sudah Completed	
	2.i. Sistem menampilkan proses atau hasil yang sudah tersedia
3.i. Maintainer menekan Muat Ulang setelah proses ulang gagal	
	4.i. Sistem menampilkan status Failed
	5.i. Sistem menyediakan Proses Ulang
	6.i. Sistem menyaring kandidat yang tidak valid

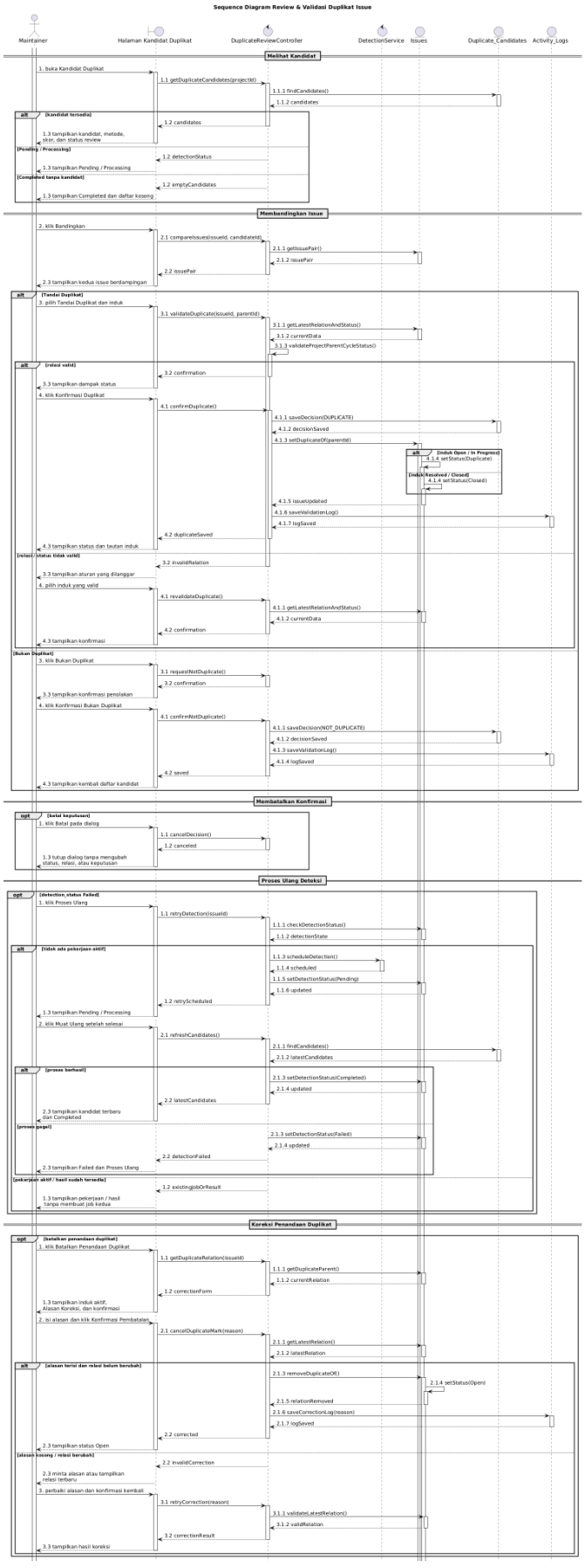
4.6.3 Activity Diagram

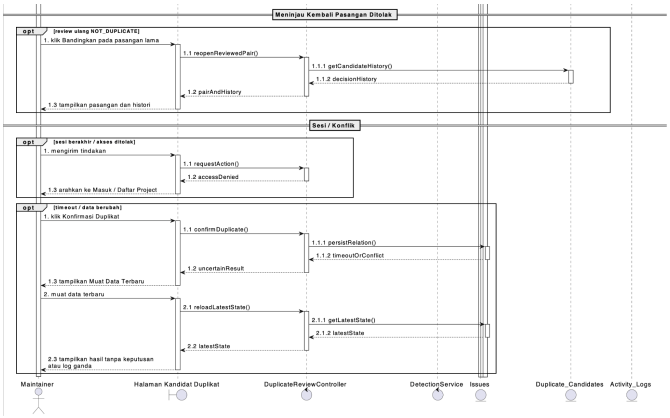

Review & Validasi Duplikat Issue - Activity Diagram
Maintainer
Sistem
Pilih alur?
Alur lain?
[Review Duplikat]
[Batalkan Penandaan]
[Proses Ulang Deteksi]
Buka issue tertaut, tekan Batalkan Penandaan Duplikat
Buka issue status Failed, tekan Proses Ulang Deteksi
Tampilkan form dialog Alasan Koreksi
Cek antrean & jadwalkan tugas (Pending → Processing)
Isi alasan & tekan Konfirmasi Pembatalan
Tekan Muat Ulang (Refresh)
Alasan terisi?
Tidak
Ya
Tampilkan kandidat baru, status Completed
Minta alasan wajib diisi
Lepas relasi duplicate_of, status kembali Open & log
Buka project, pilih Kandidat Duplikat
Tampilkan daftar issue dengan kandidat
Pilih issue, tekan Bandingkan
Tampilkan kedua issue berdampingan
Hasil review?
[Tandai duplikat]
[Bukan duplikat]
Tekan Bukan Duplikat
Konfirmasi & simpan penolakan pasangan issue
Pilih issue induk, tekan Tandai Duplikat
Validasi aturan relasi (1 project, no siklus, bukan duplikat)
Aturan terpenuhi?
Ya
Tidak
Tolak & jelaskan aturan dilanggar
Tampilkan konfirmasi + simulasi dampak status
Aksi?
[Batal]
[Konfirmasi]
Tekan Konfirmasi Duplikat
Simpan duplicate_of & status Duplicate + log
Issue duplikat perlu ditutup?
Ya
Tidak
Set issue duplikat status Closed
Set status Duplicate biasa
Tampilkan detail issue dengan tautan induk

Gambar 4.10 Activity Diagram Review & Validasi Duplikat Issue

Gambar 4.10 memiliki dua alur dari Maintainer. Pada alur Review Duplikat, Maintainer membuka Kandidat Duplikat, memilih issue lalu Bandingkan, dan Sistem menampilkan kedua issue berdampingan. Bila hasil review bukan duplikat, Maintainer menekan Bukan Duplikat dan Sistem mengonfirmasi serta menyimpan penolakan pasangan. Bila duplikat, Maintainer memilih issue induk dan menekan Tandai Duplikat, Sistem memvalidasi aturan relasi (satu project, tanpa siklus, induk bukan duplikat), menampilkan konfirmasi beserta simulasi dampak status, dan setelah Maintainer mengonfirmasi, Sistem menyimpan duplicate_of dan log lalu menetapkan status Closed bila perlu ditutup atau Duplicate bila tidak. Pada alur lain, Maintainer dapat membatalkan penandaan (Sistem meminta alasan wajib, melepas relasi, mengembalikan status Open, dan mencatat log) atau memproses ulang deteksi berstatus Failed (Sistem memeriksa antrean dan menjadwalkan tugas Pending lalu Processing sampai kandidat baru berstatus Completed).

4.6.4 Sequence Diagram


Sequence Diagram Review & Validasi Duplikat Issue
Maintainer
Halaman Kandidat Duplikat
DuplicateReviewController
DetectionService
Issues
Duplicate_Candidates
Activity_Logs
Melihat Kandidat
1. buka Kandidat Duplikat
1.1 getDuplicateCandidates(projectId)
1.1.1 findCandidates()
1.1.2 candidates
alt
[kandidat tersedia]
1.2 candidates
1.3 tampilkan kandidat, metode, skor, dan status review
[Pending / Processing]
1.2 detectionStatus
1.3 tampilkan Pending / Processing
[Completed tanpa kandidat]
1.2 emptyCandidates
1.3 tampilkan Completed dan daftar kosong
Membandingkan Issue
2. klik Bandingkan
2.1 compareIssues(issueId, candidateId)
2.1.1 getIssuePair()
2.1.2 issuePair
2.2 issuePair
2.3 tampilkan kedua issue berdampingan
alt
[Tandai Duplikat]
3. pilih Tandai Duplikat dan induk
3.1 validateDuplicate(issueId, parentId)
3.1.1 getLatestRelationAndStatus()
3.1.2 currentData
3.1.3 validateProjectParentCycleStatus()
alt
[relasi valid]
3.2 confirmation
3.3 tampilkan dampak status
4. klik Konfirmasi Duplikat
4.1 confirmDuplicate()
4.1.1 saveDecision(DUPLICATE)
4.1.2 decisionSaved
4.1.3 setDuplicateOf(parentId)
alt
[induk Open / In Progress]
4.1.4 setStatus(Duplicate)
[induk Resolved / Closed]
4.1.4 setStatus(Closed)
4.1.5 issueUpdated
4.1.6 saveValidationLog()
4.1.7 logSaved
4.2 duplicateSaved
4.3 tampilkan status dan tautan induk
[relasi / status tidak valid]
3.2 invalidRelation
3.3 tampilkan aturan yang dilanggar
4. pilih induk yang valid
4.1 revalidateDuplicate()
4.1.1 getLatestRelationAndStatus()
4.1.2 currentData
4.2 confirmation
4.3 tampilkan konfirmasi
[Bukan Duplikat]
3. klik Bukan Duplikat
3.1 requestNotDuplicate()
3.2 confirmation
3.3 tampilkan konfirmasi penolakan
4. klik Konfirmasi Bukan Duplikat
4.1 confirmNotDuplicate()
4.1.1 saveDecision(NOT_DUPLICATE)
4.1.2 decisionSaved
4.1.3 saveValidationLog()
4.1.4 logSaved
4.2 saved
4.3 tampilkan kembali daftar kandidat
Membatalkan Konfirmasi
opt
[batal keputusan]
1. klik Batal pada dialog
1.1 cancelDecision()
1.2 canceled
1.3 tutup dialog tanpa mengubah status, relasi, atau keputusan
Proses Ulang Deteksi
opt
[detection_status Failed]
1. klik Proses Ulang
1.1 retryDetection(issueId)
1.1.1 checkDetectionStatus()
1.1.2 detectionState
alt
[tidak ada pekerjaan aktif]
1.1.3 scheduleDetection()
1.1.4 scheduled
1.1.5 setDetectionStatus(Pending)
1.1.6 updated
1.2 retryScheduled
1.3 tampilkan Pending / Processing
2. klik Muat Ulang setelah selesai
2.1 refreshCandidates()
2.1.1 findCandidates()
2.1.2 latestCandidates
alt
[proses berhasil]
2.1.3 setDetectionStatus(Completed)
2.1.4 updated
2.2 latestCandidates
2.3 tampilkan kandidat terbaru dan Completed
[proses gagal]
2.1.3 setDetectionStatus(Failed)
2.1.4 updated
2.2 detectionFailed
2.3 tampilkan Failed dan Proses Ulang
[pekerjaan aktif / hasil sudah tersedia]
1.2 existingJobOrResult
1.3 tampilkan pekerjaan / hasil tanpa membuat job kedua
Koreksi Penandaan Duplikat
opt
[batalkan penandaan duplikat]
1. klik Batalkan Penandaan Duplikat
1.1 getDuplicateRelation(issueId)
1.1.1 getDuplicateParent()
1.1.2 currentRelation
1.2 correctionForm
1.3 tampilkan induk aktif, Alasan Koreksi, dan konfirmasi
2. isi alasan dan klik Konfirmasi Pembatalan
2.1 cancelDuplicateMark(reason)
2.1.1 getLatestRelation()
2.1.2 latestRelation
alt
[alasan terisi dan relasi belum berubah]
2.1.3 removeDuplicateOf()
2.1.4 setStatus(Open)
2.1.5 relationRemoved
2.1.6 saveCorrectionLog(reason)
2.1.7 logSaved
2.2 corrected
2.3 tampilkan status Open
[alasan kosong / relasi berubah]
2.2 invalidCorrection
2.3 minta alasan atau tampilkan relasi terbaru
3. perbaiki alasan dan konfirmasi kembali
3.1 retryCorrection(reason)
3.1.1 validateLatestRelation()
3.1.2 validRelation
3.2 correctionResult
3.3 tampilkan hasil koreksi



Gambar 4.11 Sequence Diagram Review & Validasi Duplikat Issue (dibaca dari kiri ke kanan)

Gambar 4.11 menunjukkan interaksi Maintainer, Halaman Kandidat Duplikat, DuplicateReviewController, DetectionService, Issues, Duplicate_Candidates, dan Activity_Logs. Bagian yang digambarkan adalah melihat kandidat (termasuk status deteksi Pending atau Processing), membandingkan issue, menandai duplikat dengan validasi relasi dan pemeriksaan versi data, menandai bukan duplikat, membatalkan penandaan dengan alasan koreksi, memproses ulang deteksi, meninjau kembali pasangan yang ditolak, serta kondisi gagal atau data basi.

4.6.5 User Story

Tabel 4.27. User Story Review & Validasi Duplikat Issue

Name (Description)	User Story
Review & Validasi Duplikat Issue	As a Maintainer, I want to review and validate potential duplicate issues so that duplicate relationships can be confirmed accurately before they are applied.

4.6.6 User Story Scenario

Tabel 4.28. User Story Scenario Review & Validasi Duplikat Issue

Feature: Review dan Validasi Duplikat Issue In order to menetapkan duplikat dengan human-in-the-loop

As a Maintainer
I want to membandingkan kandidat dan memvalidasi relasi duplikat

Background:
Given I am logged in as "elsa@dupissue.test" with role "Maintainer"
And I own project "Toko Online Baju" with id "1"

Scenario: Maintainer menandai issue sebagai duplikat
Given I am on "/projects/1/candidates"
Then I should see "Daftar issue yang mempunyai kandidat"
And I should see "skor kemiripan"
When I press "Bandingkan"
Then I should see "judul" in the "comparison" element
And I should see "deskripsi" in the "comparison" element
When I press "Tandai Duplikat"
And I select "10" from "issue_induk"
And I press "Konfirmasi Duplikat"
Then I should see "Keputusan tersimpan"
And I should see "duplicate_of: 10"
And I should see "Duplicate" in the "status" element
And I should see "Issue Induk"

Scenario: Maintainer menetapkan kandidat sebagai bukan duplikat
Given I am on "/projects/1/candidates"
When I press "Bandingkan"
And I press "Bukan Duplikat"
Then I should see "Konfirmasi penolakan pasangan"
When I press "Konfirmasi Bukan Duplikat"
Then I should see "Keputusan tersimpan"
And I should see "Daftar issue yang mempunyai kandidat"

Scenario: Maintainer memproses ulang deteksi yang gagal
Given I am on "/projects/1/issues/14"
And I should see "Failed" in the "detection_status" element
When I press "Proses Ulang"
Then I should see "Pending" in the "detection_status" element
When I press "Muat Ulang"
Then I should see "Completed" in the "detection_status" element
And I should see "kandidat terbaru"

Scenario: Maintainer menandai duplikat pada induk yang sudah Resolved
Given I am on "/projects/1/candidates"
When I press "Bandingkan"
And I press "Tandai Duplikat"
And I select "10" from "issue_induk"
And I press "Konfirmasi Duplikat"
Then I should see "Closed" in the "status" element
And I should see "Issue Induk"

Scenario: Maintainer membatalkan penandaan duplikat
Given I am on "/projects/1/issues/11"
When I press "Batalkan Penandaan Duplikat"
Then I should see "Alasan Koreksi"

```
  When I fill in "Alasan Koreksi" with "Salah link ternyata berbeda bug"
  And I press "Konfirmasi Pembatalan"
  Then I should see "duplicate_of dilepas"
  And I should see "Open" in the "status" element
  And I should see "alasan serta keputusan lama tercatat"

Scenario: Maintainer meninjau kembali pasangan yang pernah ditolak
  Given I am on "/projects/1/candidates"
  And I should see "Bukan Duplikat" in the "decision_history" element
  When I press "Bandingkan"
  Then I should see "histori keputusan"
  When I press "Tandai Duplikat"
  And I select "10" from "issue_induk"
  And I press "Konfirmasi Duplikat"
  Then I should see "Keputusan tersimpan"

Scenario Outline: Relasi duplikat tidak valid
  Given I am on "/projects/1/candidates"
  When I press "Tandai Duplikat"
  And I select "<induk>" from "issue_induk"
  And I press "Konfirmasi Duplikat"
  Then I should see "<pesan>"

  Examples:
    | induk | pesan                          |
    | 11    | Induk harus berbeda            |
    | 12    | Harus satu project             |
    | 13    | Induk bukan duplikat           |
    | 14    | Tidak membentuk siklus         |

Scenario: Alasan koreksi wajib diisi
  Given I am on "/projects/1/issues/11"
  When I press "Batalkan Penandaan Duplikat"
  And I press "Konfirmasi Pembatalan"
  Then I should see "Alasan Koreksi wajib diisi"
  And I should see "Duplicate" in the "status" element

Scenario: Proses ulang tidak membuat pekerjaan kedua
  Given I am on "/projects/1/issues/11"
  And I should see "Processing" in the "detection_status" element
  When I press "Proses Ulang"
  Then I should see "Pekerjaan sedang berjalan"
```

4.6.7 Functional Requirements

- **REQ-05.1:** Sistem harus memeriksa bahwa pengguna adalah Maintainer pemilik project sebelum menampilkan kandidat dan menyimpan keputusan.
- **REQ-05.2:** Sistem harus menampilkan daftar issue yang memiliki kandidat beserta skor kemiripan, status deteksi, dan riwayat keputusan pasangan.

- **REQ-05.3:** Sistem harus menampilkan perbandingan berdampingan yang memuat judul, deskripsi, status, jumlah vote, dan asal data kedua issue.
- **REQ-05.4:** Pencarian kandidat harus dibatasi pada issue dalam project yang sama, dan skor kemiripan tidak boleh menetapkan status DUPLICATE secara otomatis.
- **REQ-05.5:** Sistem harus memvalidasi relasi duplikat sebelum konfirmasi: induk berbeda dari duplikat, berada pada project yang sama, bukan issue yang sudah berstatus duplikat, dan tidak membentuk siklus. Pelanggaran ditolak dengan pesan aturan yang sesuai.
- **REQ-05.6:** Konfirmasi duplikat harus menampilkan ringkasan dampak status dan memeriksa versi data terbaru sebelum menyimpan.
- **REQ-05.7:** Saat konfirmasi duplikat, sistem harus menyimpan keputusan DUPLICATE (decided_by dan decided_at), mengisi duplicate_of, menetapkan status DUPLICATE, dan mencatat log, tanpa menggabungkan konten atau histori issue.
- **REQ-05.8:** Bila issue induk berstatus RESOLVED atau CLOSED, sistem harus menetapkan issue duplikat menjadi CLOSED.
- **REQ-05.9:** Sistem harus menyediakan aksi "Bukan Duplikat" dengan konfirmasi, menyimpan keputusan NOT_DUPLICATE, dan mempertahankan status penanganan issue.
- **REQ-05.10:** Pasangan yang pernah ditolak harus tetap dapat ditinjau ulang dengan menampilkan histori keputusan, dan keputusan baru tidak menghapus histori sebelumnya.
- **REQ-05.11:** Pembatalan penandaan duplikat harus mewajibkan alasan koreksi, menghapus duplicate_of, mengembalikan status menjadi OPEN, dan mencatat alasan serta keputusan lama.
- **REQ-05.12:** Sistem harus menyediakan "Proses Ulang" hanya pada deteksi berstatus FAILED, mengubah status menjadi PENDING, dan tidak membuat pekerjaan kedua bila deteksi masih PENDING, PROCESSING, atau sudah COMPLETED.
- **REQ-05.13:** Pemrosesan ulang harus mempertahankan keputusan manual yang sudah tersimpan dan menyaring kandidat yang tidak valid.
- **REQ-05.14:** Kegagalan transaksi saat menyimpan keputusan harus membatalkan seluruh perubahan dan menampilkan pesan yang dapat diulang.

4.7 System Feature 6: Mengelola Status Issue (REQ-06)

4.7.1 Description and Priority

Maintainer memperbarui status penanganan issue (OPEN, IN_PROGRESS, RESOLVED, CLOSED) sesuai transisi yang diizinkan. Ketika issue induk menjadi

RESOLVED, seluruh duplikat tertaut ditutup otomatis (auto-close). Issue yang masih berelasi sebagai duplikat tidak memakai dropdown status biasa.

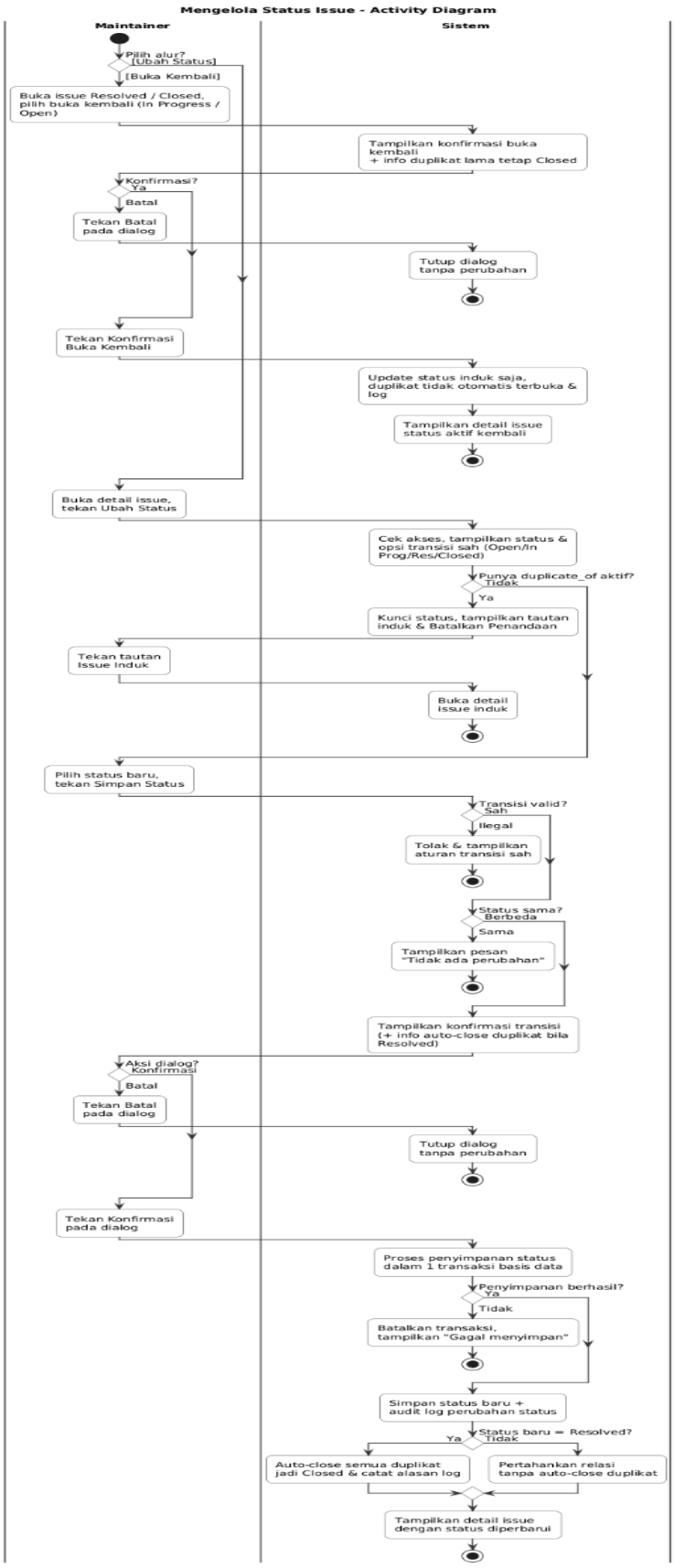
4.7.2 Use Case Specification

Tabel 4.32. Deskripsi Use Case Specification "Mengelola Status Issue"

Nama Use Case	Mengelola Status Issue
Aktor	Maintainer
Deskripsi	Maintainer memperbarui status penanganan issue dan sistem menutup duplikat tertaut ketika induk menjadi Resolved
Kondisi Awal	Maintainer sudah masuk dan merupakan pemilik project serta perubahan status biasa dilakukan pada issue tanpa duplicate of aktif
Kondisi Akhir	Status dan log tersimpan serta duplikat tertaut menjadi Closed ketika induk berubah menjadi Resolved
Skenario Normal	
Aktor	**Sistem**
1. Maintainer membuka detail issue dan menekan Ubah Status	
	2. Sistem memeriksa akses Maintainer
	3. Sistem menampilkan status issue saat ini
	4. Sistem menampilkan transisi status yang diizinkan
5. Maintainer memilih Resolved pada issue In Progress	
6. Maintainer menekan Simpan Status	
	7. Sistem memvalidasi transisi status
	8. Sistem menampilkan konfirmasi perubahan
	9. Sistem menampilkan jumlah duplikat yang akan ditutup
10. Maintainer menekan Konfirmasi	
	11. Sistem memeriksa versi data terbaru
	12. Sistem menyimpan status induk
	13. Sistem menutup seluruh duplikat tertaut
	14. Sistem mencatat log perubahan
	15. Sistem memperbarui detail issue
16. Maintainer membuka salah satu issue duplikat	
	17. Sistem menampilkan status Closed dan tautan issue induk
Skenario Alternatif	
A1. Status tidak berubah	
1.a. Maintainer memilih status yang sama	
2.a. Maintainer menekan Simpan Status	
	3.a. Sistem menampilkan Tidak ada perubahan
	4.a. Sistem mempertahankan status sebelumnya
A2. Issue tidak mempunyai duplikat	
1.b. Maintainer mengonfirmasi perubahan status pada issue tanpa duplikat	
	2.b. Sistem memperbarui status issue
	3.b. Sistem mencatat log perubahan

	4.b. Sistem tidak mengubah issue lain
A3. Membuka kembali issue yang selesai	
1.c. Maintainer memilih In Progress pada issue Resolved atau Open pada issue Closed	
2.c. Maintainer menekan Simpan Status	
	3.c. Sistem menampilkan konfirmasi pembukaan kembali
	4.c. Sistem menjelaskan bahwa duplikat lama tetap Closed
5.c. Maintainer menekan Konfirmasi	
	6.c. Sistem memperbarui status induk
	7.c. Sistem mencatat log perubahan
A4. Issue masih berelasi sebagai duplikat	
1.d. Maintainer membuka Ubah Status pada issue dengan duplicate of aktif	
	2.d. Sistem tidak menampilkan pilihan status biasa
	3.d. Sistem menampilkan tautan Issue Induk
4.d. Maintainer menekan tautan Issue Induk	
	5.d. Sistem menampilkan detail issue induk
Skenario Exception	
E1. Transisi tidak diizinkan	
1.e. Maintainer mengirim perubahan status yang tidak diizinkan	
	2.e. Sistem menolak perubahan status
	3.e. Sistem menampilkan transisi yang sah
4.e. Maintainer memilih status yang diizinkan	
5.e. Maintainer menekan Simpan Status	
	6.e. Sistem memvalidasi ulang transisi
	7.e. Sistem menampilkan konfirmasi perubahan
E2. Penyimpanan status atau auto close gagal	
1.f. Maintainer mengonfirmasi perubahan saat penyimpanan gagal	
	2.f. Sistem membatalkan seluruh transaksi
	3.f. Sistem mempertahankan status sebelum tindakan
	4.f. Sistem menampilkan Gagal menyimpan dan Coba Lagi
5.f. Maintainer memuat data terbaru	
6.f. Maintainer mengulangi penyimpanan status	
	7.f. Sistem memvalidasi ulang transisi dan keadaan data
	8.f. Sistem menjalankan transaksi tanpa menggandakan log

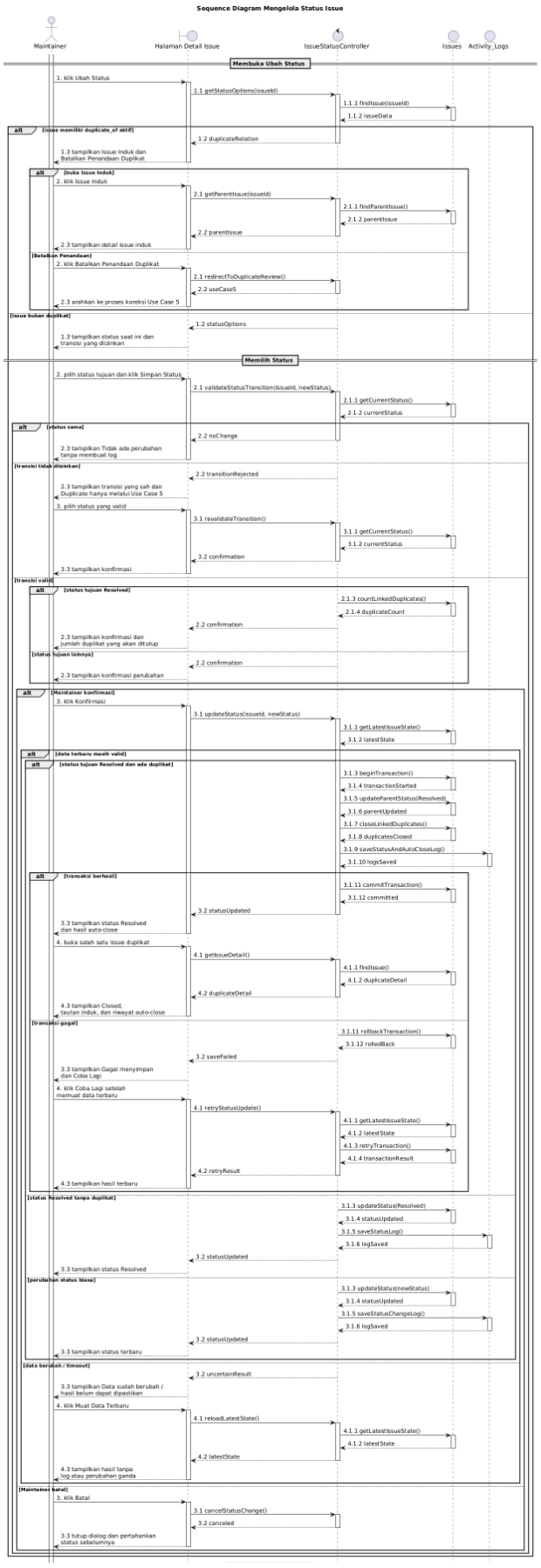
4.7.3 Activity Diagram

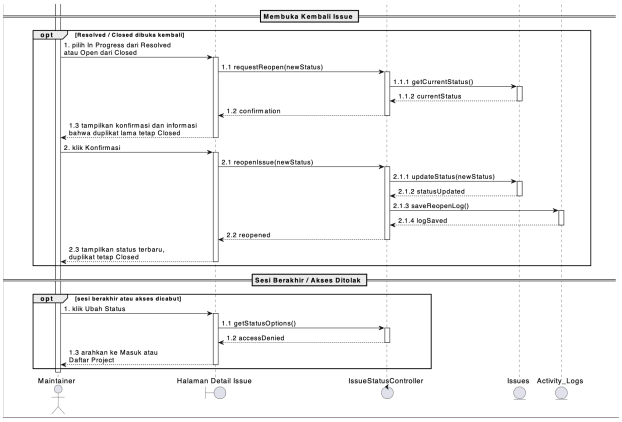

Mengelola Status Issue - Activity Diagram
Maintainer
Sistem
Pilih alur?
[Ubah Status]
[Buka Kembali]
Buka issue Resolved / Closed, pilih buka kembali (In Progress / Open)
Tampilkan konfirmasi buka kembali + info duplikat lama tetap Closed
Konfirmasi?
Ya
Batal
Tekan Batal pada dialog
Tutup dialog tanpa perubahan
Tekan Konfirmasi Buka Kembali
Update status induk saja, duplikat tidak otomatis terbuka & log
Tampilkan detail issue status aktif kembali
Buka detail issue, tekan Ubah Status
Cek akses, tampilkan status & opsi transisi sah (Open/In Prog/Res/Closed)
Punya duplicate_of aktif?
Tidak
Ya
Kunci status, tampilkan tautan induk & Batalkan Penandaan
Tekan tautan Issue Induk
Buka detail Issue induk
Pilih status baru, tekan Simpan Status
Transisi valid?
Sah
Ilegal
Tolak & tampilkan aturan transisi sah
Status sama?
Berbeda
Sama
Tampilkan pesan "Tidak ada perubahan"
Tampilkan konfirmasi transisi (+ info auto-close duplikat bila Resolved)
Aksi dialog?
Konfirmasi
Batal
Tekan Batal pada dialog
Tutup dialog tanpa perubahan
Tekan Konfirmasi pada dialog
Proses penyimpanan status dalam 1 transaksi basis data
Penyimpanan berhasil?
Ya
Tidak
Batalkan transaksi, tampilkan "Gagal menyimpan"
Simpan status baru + audit log perubahan status
Status baru = Resolved?
Ya
Tidak
Auto-close semua duplikat jadi Closed & catat alasan log
Pertahankan relasi tanpa auto-close duplikat
Tampilkan detail issue dengan status diperbarui

Gambar 4.12 Activity Diagram Mengelola Status Issue

Gambar 4.12 memiliki alur Ubah Status dan alur Buka Kembali. Pada Ubah Status, Sistem memeriksa akses dan menampilkan status serta opsi transisi sah. Bila issue memiliki duplicate_of aktif, status dikunci dan Sistem menampilkan tautan induk serta Batalkan Penandaan. Bila tidak, Maintainer memilih status baru dan menekan Simpan Status. Sistem memvalidasi transisi (ilegal ditolak) dan membandingkan status (sama menampilkan "Tidak ada perubahan"), menampilkan konfirmasi, lalu setelah konfirmasi menyimpan status dan audit log dalam satu transaksi. Bila status baru Resolved, seluruh duplikat otomatis menjadi Closed dengan catatan alasan, bila bukan, relasi duplikat dipertahankan. Pada alur Buka Kembali, Maintainer membuka issue Resolved atau Closed, Sistem menampilkan konfirmasi beserta informasi duplikat lama tetap Closed, dan hanya status induk yang diperbarui.

4.7.4 Sequence Diagram


Sequence Diagram Mengelola Status Issue
Maintainer
Halaman Detail Issue
IssueStatusController
Issues
Activity_Logs
Membuka Ubah Status
1. klik Ubah Status
1.1 getStatusOptions(issueId)
1.1.1 findIssue(issueId)
1.1.2 issueData
alt
[issue memiliki duplicate_of aktif]
1.2 duplicateRelation
1.3 tampilkan Issue Induk dan Batalkan Penandaan Duplikat
alt
[buka Issue Induk]
2. klik Issue Induk
2.1 getParentIssue(issueId)
2.1.1 findParentIssue()
2.1.2 parentIssue
2.2 parentIssue
2.3 tampilkan detail issue induk
[Batalkan Penandaan]
2. klik Batalkan Penandaan Duplikat
2.1 redirectToDuplicateReview()
2.2 useCase5
2.3 arahkan ke proses koreksi Use Case 5
[issue bukan duplikat]
1.2 statusOptions
1.3 tampilkan status saat ini dan transisi yang diizinkan
Memilih Status
2. pilih status tujuan dan klik Simpan Status
2.1 validateStatusTransition(issueId, newStatus)
2.1.1 getCurrentStatus()
2.1.2 currentStatus
alt
[status sama]
2.2 noChange
2.3 tampilkan Tidak ada perubahan tanpa membuat log
[transisi tidak diizinkan]
2.2 transitionRejected
2.3 tampilkan transisi yang sah dan Duplicate hanya melalui Use Case 5
3. pilih status yang valid
3.1 revalidateTransition()
3.1.1 getCurrentStatus()
3.1.2 currentStatus
3.2 confirmation
3.3 tampilkan konfirmasi
[transisi valid]
alt
[status tujuan Resolved]
2.1.3 countLinkedDuplicates()
2.1.4 duplicateCount
2.2 confirmation
2.3 tampilkan konfirmasi dan jumlah duplikat yang akan ditutup
[status tujuan lainnya]
2.2 confirmation
2.3 tampilkan konfirmasi perubahan
alt
[Maintainer konfirmasi]
3. klik Konfirmasi
3.1 updateStatus(issueId, newStatus)
3.1.1 getLatestIssueState()
3.1.2 latestState
alt
[data terbaru masih valid]
alt
[status tujuan Resolved dan ada duplikat]
3.1.3 beginTransaction()
3.1.4 transactionStarted
3.1.5 updateParentStatus(Resolved)
3.1.6 parentUpdated
3.1.7 closeLinkedDuplicates()
3.1.8 duplicatesClosed
3.1.9 saveStatusAndAutoCloseLog()
3.1.10 logsSaved
alt
[transaksi berhasil]
3.1.11 commitTransaction()
3.1.12 committed
3.2 statusUpdated
3.3 tampilkan status Resolved dan hasil auto-close
4. buka salah satu issue duplikat
4.1 getIssueDetail()
4.1.1 findIssue()
4.1.2 duplicateDetail
4.2 duplicateDetail
4.3 tampilkan Closed, tautan induk, dan riwayat auto-close
[transaksi gagal]
3.1.11 rollbackTransaction()
3.1.12 rolledBack
3.2 saveFailed
3.3 tampilkan Gagal menyimpan dan Coba Lagi
4. klik Coba Lagi setelah memuat data terbaru
4.1 retryStatusUpdate()
4.1.1 getLatestIssueState()
4.1.2 latestState
4.1.3 retryTransaction()
4.1.4 transactionResult
4.2 retryResult
4.3 tampilkan hasil terbaru
[status Resolved tanpa duplikat]
3.1.3 updateStatus(Resolved)
3.1.4 statusUpdated
3.1.5 saveStatusLog()
3.1.6 logSaved
3.2 statusUpdated
3.3 tampilkan status Resolved
[perubahan status biasa]
3.1.3 updateStatus(newStatus)
3.1.4 statusUpdated
3.1.5 saveStatusChangeLog()
3.1.6 logSaved
3.2 statusUpdated
3.3 tampilkan status terbaru
[data berubah / timeout]
3.2 uncertainResult
3.3 tampilkan Data sudah berubah / hasil belum dapat dipastikan
4. klik Muat Data Terbaru
4.1 reloadLatestState()
4.1.1 getLatestIssueState()
4.1.2 latestState
4.2 latestState
4.3 tampilkan hasil tanpa log atau perubahan ganda
[Maintainer batal]
3. klik Batal
3.1 cancelStatusChange()
3.2 canceled
3.3 tutup dialog dan pertahankan status sebelumnya



Gambar 4.13 Sequence Diagram Mengelola Status Issue (dibaca dari kiri ke kanan)

Gambar 4.13 menunjukkan interaksi Maintainer, Halaman Detail Issue, IssueStatusController, Issues, dan Activity_Logs. Controller mengambil opsi status, menangani issue ber-duplicate_of aktif, memvalidasi transisi, menghitung duplikat yang akan ditutup, lalu pada konfirmasi memeriksa versi data, memperbarui status induk, menutup duplikat, mencatat log, dan melakukan commit atau rollback. Bagian lain menggambarkan pembukaan kembali issue serta kondisi gagal atau data basi.

4.7.5 User Story

Tabel 4.36. User Story Mengelola Status Issue

Name (Description)	User Story
Mengelola Status Issue	As a Maintainer, I want to manage issue statuses so that the progress and resolution of each issue can be tracked consistently.

4.7.6 User Story Scenario

Tabel 4.37. User Story Scenario Mengelola Status Issue

Feature: Mengelola Status Issue
In order to melacak penanganan dan menutup duplikat secara otomatis
As a Maintainer
I want to memperbarui status penanganan issue

Background:

```
  Given I am logged in as "elsa@dupissue.test" with role "Maintainer"
  And I own project "Toko Online Baju" with id "1"

Scenario: Maintainer mengubah status menjadi Resolved dan menutup duplikat
  Given I am on "/projects/1/issues/10"
  When I press "Ubah Status"
  Then I should see "In Progress" in the "status" element
  And I should see "pilihan transisi yang diizinkan"
  When I select "Resolved" from "status"
  And I press "Simpan Status"
  Then I should see "Konfirmasi"
  And I should see "jumlah duplikat tertaut yang akan ditutup: 1"
  When I press "Konfirmasi"
  Then I should see "Resolved" in the "status" element
  And I should see "log tercatat"
  When I go to "/projects/1/issues/11"
  Then I should see "Closed" in the "status" element
  And I should see "Issue Induk"
  And I should see "riwayat auto-close"

Scenario: Maintainer memilih status yang sama
  Given I am on "/projects/1/issues/10"
  When I press "Ubah Status"
  And I select "In Progress" from "status"
  And I press "Simpan Status"
  Then I should see "Tidak ada perubahan"
  And I should not see "log perubahan"

Scenario: Issue tidak mempunyai duplikat tertaut
  Given I am on "/projects/1/issues/12"
  When I press "Ubah Status"
  And I select "Resolved" from "status"
  And I press "Simpan Status"
  And I press "Konfirmasi"
  Then I should see "Resolved" in the "status" element
  And I should see "log tercatat"

Scenario: Maintainer membuka kembali issue yang selesai
  Given I am on "/projects/1/issues/10"
  And I should see "Resolved" in the "status" element
  When I press "Ubah Status"
  And I select "In Progress" from "status"
  And I press "Simpan Status"
  And I press "Konfirmasi"
  Then I should see "In Progress" in the "status" element
  And I should see "duplikat lama tetap Closed"

Scenario: Issue duplikat tidak menggunakan dropdown status biasa
  Given I am on "/projects/1/issues/11"
  When I press "Ubah Status"
  Then I should not see "status" in the "status_form" element
  And I should see "Issue Induk"
  And I should see "Batalkan Penandaan Duplikat"
```

```
  When I follow "Issue Induk"
  Then I should be on "/projects/1/issues/10"

Scenario Outline: Transisi status tidak diizinkan
  Given I am on "/projects/1/issues/<issue>"
  When I press "Ubah Status"
  And I select "<tujuan>" from "status"
  And I press "Simpan Status"
  Then I should see "Transisi tidak diizinkan"
  And I should see "Duplicate hanya ditetapkan melalui validasi"

  Examples:
    | issue | tujuan    |
    | 20    | Duplicate |
    | 20    | Closed    |
    | 21    | Duplicate |

Scenario: Penyimpanan status gagal
  Given I am on "/projects/1/issues/10"
  When I press "Ubah Status"
  And I select "Resolved" from "status"
  And I press "Simpan Status"
  And I press "Konfirmasi"
  Then I should see "Gagal menyimpan"
  And I should see "Coba Lagi"
  And I should see "In Progress" in the "status" element
  When I press "Coba Lagi"
  Then I should see "Resolved" in the "status" element
```

4.7.7 Functional Requirements

Transisi status yang diizinkan diringkas pada Tabel 4.38. Matriks ini merupakan usulan berdasarkan use case dan activity diagram, dan perlu disepakati kelompok (lihat Appendix C, TBD-02).

Tabel 4.38. Transisi Status Penanganan Issue (Usulan)

Status Awal	Status Tujuan yang Diizinkan	Pemicu
OPEN	IN_PROGRESS	Maintainer (Ubah Status)
IN_PROGRESS	RESOLVED	Maintainer, memicu auto-close duplikat tertaut
RESOLVED	IN_PROGRESS	Maintainer (buka kembali), duplikat lama tetap CLOSED

CLOSED	OPEN	Maintainer (buka kembali), duplikat lama tetap CLOSED
OPEN atau IN_PROGRESS	DUPLICATE	Hanya melalui Review & Validasi (REQ-05), bukan Ubah Status
Duplikat tertaut	CLOSED	Sistem (auto-close) atau REQ-05.8

- **REQ-06.1:** Sistem harus memeriksa bahwa pengguna adalah Maintainer pemilik project sebelum menampilkan dan menyimpan perubahan status.
- **REQ-06.2:** Sistem harus menampilkan status saat ini dan hanya transisi status yang diizinkan (Tabel 4.38).
- **REQ-06.3:** Sistem harus menolak transisi yang tidak diizinkan, termasuk penetapan DUPLICATE melalui Ubah Status, dengan pesan "Transisi tidak diizinkan" dan menampilkan transisi yang sah.
- **REQ-06.4:** Bila status yang dipilih sama dengan status saat ini, sistem harus menampilkan "Tidak ada perubahan" tanpa menyimpan data maupun mencatat log.
- **REQ-06.5:** Sebelum menyimpan, sistem harus menampilkan konfirmasi dan, bila status baru RESOLVED, jumlah duplikat tertaut yang akan ditutup.
- **REQ-06.6:** Sistem harus memeriksa versi data terbaru sebelum menyimpan agar perubahan oleh pengguna lain tidak tertimpa.
- **REQ-06.7:** Saat issue induk menjadi RESOLVED, sistem harus mengubah seluruh duplikat tertaut menjadi CLOSED dalam satu transaksi dengan penyimpanan status induk, disertai catatan alasan auto-close pada log.
- **REQ-06.8:** Pembukaan kembali issue RESOLVED atau CLOSED harus meminta konfirmasi, hanya mengubah status induk, dan mempertahankan duplikat lama pada status CLOSED.
- **REQ-06.9:** Untuk issue dengan duplicate_of aktif, sistem tidak boleh menampilkan pilihan status biasa dan harus menampilkan tautan "Issue Induk" serta aksi "Batalkan Penandaan Duplikat".
- **REQ-06.10:** Bila penyimpanan status atau auto-close gagal, sistem harus membatalkan seluruh transaksi, mempertahankan status sebelumnya, menampilkan "Gagal menyimpan" dengan "Coba Lagi", dan tidak menggandakan log pada percobaan ulang.
- **REQ-06.11:** Setiap perubahan status harus dicatat pada activity_logs dengan nilai lama dan nilai baru.

4.8 Fungsi Pendukung (REQ-07)

Fungsi pendukung berikut tidak berdiri sebagai use case tersendiri tetapi diperlukan agar enam fitur utama dapat berjalan. Kebutuhannya konsisten dengan 2.2 (Product Functions) dan 2.3 (User Classes).

- **REQ-07.1 (Autentikasi):** Sistem harus menyediakan halaman login dengan email dan kata sandi, serta menggunakan sesi Laravel. Setelah login, pengguna diarahkan sesuai perannya (Reporter ke Daftar Project dan Undangan, Maintainer ke Dashboard). Kegagalan login menampilkan pesan kredensial salah tanpa menyebutkan bagian mana yang salah.
- **REQ-07.2 (Logout dan sesi):** Sistem harus menyediakan logout dan menampilkan pesan sesi berakhir ketika sesi kedaluwarsa tanpa kehilangan data yang sudah tersimpan.
- **REQ-07.3 (RBAC):** Setiap halaman dan operasi yang dilindungi harus memeriksa peran dan keanggotaan atau kepemilikan project pada sisi server. Akses ditolak dengan pesan "Akses tidak tersedia".
- **REQ-07.4 (Daftar Project):** Reporter harus melihat daftar project aktif miliknya dan undangan masuk. Maintainer harus melihat daftar project miliknya.
- **REQ-07.5 (Dashboard Maintainer):** Sistem harus menampilkan ringkasan project berupa total issue, jumlah per status (OPEN, IN_PROGRESS, RESOLVED, CLOSED), jumlah dan persentase issue duplikat, issue dengan vote terbanyak, serta jumlah kandidat duplikat yang menunggu keputusan. Metrik rata-rata waktu triase dikonfirmasi pada Appendix C (TBD-04).
- **REQ-07.6 (Empty state):** Project tanpa issue harus menampilkan keadaan kosong dengan tombol Impor File CSV atau Hubungkan GitHub.
- **REQ-07.7 (Riwayat aktivitas):** Sistem harus mencatat aktivitas penting project (pembuatan, perubahan status, validasi duplikat, impor, pengelolaan anggota) pada activity_logs yang dapat diakses Maintainer.
- **REQ-07.8 (Pemberitahuan dalam aplikasi):** Sistem harus menampilkan penanda undangan baru dan status deteksi pada antarmuka tanpa mewajibkan email atau push notification.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Kebutuhan performa menetapkan respons sistem agar pelaporan dan peninjauan issue berjalan lancar. Seluruh angka di bawah ini adalah target awal untuk lingkungan pengujian (4 vCPU, 8 GB RAM, tanpa GPU) dan akan divalidasi pada pengujian performa.

- **REQ-NF-01:** Halaman utama aplikasi (Daftar Project, Daftar Issue, Detail Issue, Dashboard) harus selesai dimuat dalam waktu kurang dari 3 detik pada jaringan normal untuk project hingga 5.000 issue.
- **REQ-NF-02:** Rekomendasi issue serupa pada formulir Submit Issue harus muncul dalam waktu maksimal 3 detik setelah jeda pengetikan pada 95% permintaan, untuk project hingga 5.000 issue. Sistem harus menampilkan indikator pemuatan selama proses berlangsung.
- **REQ-NF-03:** Pengiriman issue (Kirim Issue) harus selesai menyimpan data dan menampilkan detail issue dalam waktu maksimal 3 detik, karena deteksi kandidat dijalankan secara asynchronous melalui antrean dan tidak boleh menahan respons kepada Reporter.
- **REQ-NF-04:** Validasi file CSV berukuran maksimal 5 MB dan 5.000 baris harus selesai dalam waktu maksimal 30 detik, dan penyimpanan baris valid dalam waktu maksimal 60 detik di luar waktu deteksi.
- **REQ-NF-05:** Deteksi kandidat untuk satu issue baru harus selesai dalam waktu maksimal 60 detik pada project hingga 5.000 issue selama queue worker aktif. Pada impor besar, deteksi boleh berlangsung lebih lama tetapi status deteksi harus selalu dapat diperiksa.
- **REQ-NF-06:** Sistem harus mampu melayani minimal 20 pengguna aktif secara bersamaan pada lingkungan pengujian tanpa penurunan fungsi (nilai target akan dikonfirmasi, lihat Appendix C, TBD-08).
- **REQ-NF-07:** Sistem harus mengelola batas laju (rate limit) GitHub REST API dengan menampilkan pesan yang jelas bila batas tercapai dan tidak membuat pratinjau yang tidak lengkap tersimpan sebagai issue.

5.2 Safety Requirements

Kebutuhan keselamatan menjaga data dari kehilangan atau kerusakan akibat kegagalan proses.

- **REQ-NF-08:** Operasi yang mengubah beberapa data sekaligus (impor batch, penetapan duplikat, auto-close duplikat, perubahan status) harus dijalankan dalam transaksi database. Bila salah satu langkah gagal, seluruh perubahan dibatalkan (rollback).
- **REQ-NF-09:** Kegagalan layanan NLP, GitHub, atau queue worker tidak boleh menghapus atau merusak issue yang telah tersimpan. Sistem harus menampilkan pesan kegagalan yang dapat dipahami pengguna dan tidak mengalami crash.
- **REQ-NF-10:** Penghapusan anggota hanya menonaktifkan keanggotaan. Issue, vote, dan histori aktivitas anggota tersebut harus tetap tersimpan.
- **REQ-NF-11:** Tindakan yang berdampak luas (batalkan undangan, hapus anggota, tandai duplikat, batalkan penandaan duplikat, ubah status induk yang menutup duplikat) harus meminta konfirmasi eksplisit dengan penjelasan dampaknya.
- **REQ-NF-12:** Percobaan ulang setelah kegagalan tidak boleh menggandakan data, termasuk log aktivitas, vote, undangan, dan issue hasil impor.
- **REQ-NF-13:** Database harus dicadangkan secara berkala (minimal harian pada server pengujian) ke lokasi penyimpanan yang terpisah dari data aplikasi.
- **REQ-NF-14:** Jumlah vote pada issues.vote_count harus selalu konsisten dengan jumlah baris pada tabel votes, termasuk saat terjadi kegagalan di tengah proses.

5.3 Security Requirements

Kebutuhan keamanan melindungi data project, akun pengguna, dan kredensial GitHub dari akses yang tidak sah.

- **REQ-NF-15:** Seluruh fitur selain halaman login harus mewajibkan autentikasi. Sistem menggunakan mekanisme sesi Laravel dengan masa berlaku sesi terbatas dan perlindungan CSRF pada semua formulir.
- **REQ-NF-16:** Sistem harus menerapkan Role-Based Access Control. Peran dan keanggotaan atau kepemilikan project harus diperiksa pada sisi server di setiap permintaan, bukan hanya disembunyikan pada antarmuka.
- **REQ-NF-17:** Seluruh pembacaan dan penulisan data issue, vote, kandidat, dan impor harus dibatasi pada project yang dapat diakses pengguna. Akses ke sumber daya project lain harus ditolak (tidak ada akses lintas project melalui manipulasi ID pada URL).
- **REQ-NF-18:** Kata sandi harus disimpan sebagai hash dengan algoritma yang kuat (bcrypt atau Argon2) dan tidak boleh disimpan atau dicatat dalam bentuk asli.

- **REQ-NF-19:** Personal Access Token (PAT) GitHub harus disimpan terenkripsi pada github_configs.pat_encrypted, hanya didekripsi di backend saat dipakai, tidak dikirim ke browser, dan tidak ditulis pada log aktivitas maupun log aplikasi.
- **REQ-NF-20:** Seluruh akses melalui jaringan publik harus menggunakan HTTPS. Layanan FastAPI hanya boleh diakses dari backend Laravel melalui jaringan internal dan tidak diekspos langsung ke pengguna.
- **REQ-NF-21:** Seluruh masukan pengguna (judul, deskripsi, alasan koreksi, nama project, email) harus divalidasi pada sisi server dan di-escape saat ditampilkan untuk mencegah XSS. Akses database harus memakai query terparameter (Eloquent atau query builder) untuk mencegah SQL injection.
- **REQ-NF-22:** Unggahan CSV harus diperiksa jenis, ukuran, dan encoding pada sisi server dan tidak boleh dijalankan sebagai kode.
- **REQ-NF-23:** Percobaan login berulang dan permintaan rekomendasi harus dibatasi laju (throttling) untuk mengurangi penyalahgunaan.
- **REQ-NF-24:** Pesan kesalahan tidak boleh membocorkan detail teknis internal (stack trace, kueri, atau struktur database) kepada pengguna.

5.4 Software Quality Attributes

- **Usability**: Antarmuka harus dapat dipakai Reporter dan Maintainer tanpa pelatihan khusus. Pesan error dan keadaan kosong harus jelas dan menawarkan langkah lanjutan (misalnya Reset Filter, Coba Lagi). Target skor System Usability Scale (SUS) minimal 70 pada pengujian usability.
- **Reliability**: Sistem harus tetap dapat dipakai untuk pelaporan dan pengelolaan issue walaupun layanan NLP atau GitHub tidak tersedia (graceful degradation). Target ketersediaan pada periode pengujian ditetapkan pada Appendix C (TBD-08).
- **Correctness**: Kualitas rekomendasi NLP harus dievaluasi pada dataset bug report berlabel duplikat dengan metrik seperti recall@k dan MRR, dan dibandingkan dengan baseline TF-IDF cosine similarity. Ambang batas keberterimaan ditentukan pada Appendix C (TBD-06).
- **Maintainability**: Kode harus terstruktur mengikuti konvensi Laravel (controller, service, model, queue job), memisahkan layanan NLP dari aplikasi utama, dan terdokumentasi sehingga komponen dapat diperbarui secara independen.

- **Portability**: Aplikasi harus berjalan pada Chrome, Firefox, Edge, dan Safari versi modern, serta dapat dijalankan pada lingkungan pengembangan Windows, macOS, atau Linux dan pada Ubuntu Server 24.04 LTS.
- **Interoperability**: Komunikasi Laravel dan FastAPI harus menggunakan REST API dengan format JSON yang terdokumentasi (identitas kandidat dan skor kemiripan).
- **Testability**: Setiap use case harus memiliki user story scenario (Gherkin) yang dapat dijadikan skenario uji penerimaan. Layanan NLP harus dapat diuji secara terpisah.
- **Robustness**: Sistem harus menangani masukan tidak valid, data basi, dan permintaan ganda (vote berulang, proses ulang ganda) tanpa merusak konsistensi data.
- **Prioritas atribut**: Correctness dan reliability data diutamakan di atas kecepatan pemrosesan NLP, karena kesalahan penetapan duplikat lebih merugikan daripada keterlambatan kandidat.

5.5 Business Rules

Tabel 5.1 Business Rules DupIssue

ID	Aturan Bisnis
BR-01	Sistem hanya memiliki dua peran: Reporter dan Maintainer. Satu project dimiliki satu Maintainer pembuat.
BR-02	Hanya Maintainer yang dapat membuat project, mengelola anggota, mengimpor sumber issue, memvalidasi duplikat, dan mengubah status issue.
BR-03	Reporter hanya dapat mengakses project setelah undangan berstatus ACCEPTED dan keanggotaan aktif. Undangan hanya ditujukan kepada akun berperan Reporter.
BR-04	Satu email tidak boleh memiliki lebih dari satu keanggotaan aktif atau undangan PENDING pada project yang sama.
BR-05	Setiap issue milik satu project. Rekomendasi, kandidat duplikat, dan relasi duplikat tidak pernah melintasi project.
BR-06	Reporter dapat membuat issue dan vote hanya pada project tempat ia anggota aktif. Satu Reporter hanya boleh memiliki satu vote aktif per issue.

BR-07	Vote diberikan pada issue induk atau issue yang tidak tertaut sebagai duplikat. Rekomendasi yang sudah duplikat diarahkan ke induknya.
BR-08	Hasil NLP hanya berupa rekomendasi. Status DUPLICATE hanya ditetapkan oleh Maintainer melalui proses Review & Validasi.
BR-09	Satu issue duplikat memiliki paling banyak satu induk, induk harus berada pada project yang sama, bukan issue duplikat, dan tidak boleh membentuk siklus.
BR-10	Penandaan duplikat tidak menggabungkan konten atau histori issue. Koreksi penandaan wajib disertai alasan dan dicatat.
BR-11	Status DUPLICATE tidak dapat ditetapkan melalui Ubah Status. Issue dengan duplicate_of aktif tidak memakai dropdown status biasa.
BR-12	Saat issue induk menjadi RESOLVED, seluruh duplikat tertaut menjadi CLOSED. Membuka kembali induk tidak membuka kembali duplikat lama.
BR-13	Impor CSV: UTF-8, maksimal 5 MB, 5.000 baris, kolom wajib source_id, title, description. Setiap project memiliki paling banyak satu konfigurasi GitHub aktif.
BR-14	Issue yang sama (identitas sumber yang sama pada project yang sama) tidak boleh tersimpan lebih dari satu kali akibat impor berulang.
BR-15	Deteksi duplikasi berjalan asynchronous dan dapat diproses ulang hanya bila berstatus FAILED. Kegagalan deteksi tidak membatalkan penyimpanan issue.
BR-16	Penghapusan anggota tidak menghapus laporan dan histori lama. Seluruh perubahan penting dicatat pada log aktivitas.

6. Other Requirements

6.1 Database Requirements

Database menggunakan MySQL 8.0 dengan struktur sebagaimana CDM dan PDM pada Appendix B.2, yang terdiri atas sepuluh tabel: users, projects, memberships, invitations, issues, votes, duplicate_candidates, github_configs, import_batches, dan activity_logs.

- **REQ-OR-01:** Setiap tabel harus memiliki primary key bertipe BIGINT dan relasi harus ditegakkan dengan foreign key sebagaimana PDM.
- **REQ-OR-02:** Tabel votes harus memiliki constraint unik pada kombinasi issue_id dan voter_id. Tabel github_configs harus memiliki constraint unik pada project_id agar satu project hanya memiliki satu konfigurasi.
- **REQ-OR-03:** Tabel users harus memiliki constraint unik pada email. Tabel duplicate_candidates harus memiliki constraint CHECK skor antara 0 dan 1 serta keunikan pasangan issue_id dan candidate_id.
- **REQ-OR-04:** Kolom issues.duplicate_of harus berupa foreign key rekursif ke issues.id dan bernilai kosong untuk issue tanpa induk.
- **REQ-OR-05:** Sistem harus menjaga identitas sumber issue impor (source_type dan source_id atau github_repo dan github_number) tetap unik per project untuk mencegah duplikasi akibat impor berulang.
- **REQ-OR-06:** Kolom status, detection_status, decision, dan status undangan harus dibatasi pada nilai ENUM yang didefinisikan pada PDM.
- **REQ-OR-07:** Tabel activity_logs harus menyimpan nilai lama dan baru (JSON) tanpa memuat kata sandi atau PAT.
- **REQ-OR-08:** Perubahan struktur database harus dikelola melalui migrasi Laravel agar skema dapat direproduksi pada lingkungan lokal dan server pengujian.

6.2 Internationalization

- **REQ-OR-09:** Antarmuka aplikasi menggunakan Bahasa Indonesia. Istilah teknis dan status (Open, In Progress, Resolved, Closed, Duplicate) tetap menggunakan istilah baku.
- **REQ-OR-10:** Isi issue dapat berbahasa Indonesia maupun Inggris. Model NLP multibahasa (paraphrase-multilingual-MiniLM-L12-v2) dipilih agar rekomendasi tetap relevan untuk kedua bahasa. Fine-tuning tidak termasuk dalam MVP.

- **REQ-OR-11:** Waktu disimpan dalam format DATETIME dan ditampilkan dengan zona waktu WIB.

6.3 Legal and Ethical Requirements

- **REQ-OR-12:** Penggunaan GitHub REST API harus mematuhi ketentuan layanan dan batas laju GitHub. Sistem hanya meminta izin baca yang diperlukan untuk mengambil issue.
- **REQ-OR-13:** Dataset bug report yang dipakai untuk evaluasi harus berasal dari sumber yang lisensinya mengizinkan penggunaan akademik, dan sumbernya dicantumkan.
- **REQ-OR-14:** Data issue bersifat milik project dan hanya dapat dilihat anggotanya. Sistem tidak membagikan konten issue kepada pihak ketiga selain pemrosesan oleh layanan NLP internal.

6.4 Reuse and Deployment

- **REQ-OR-15:** Layanan NLP harus dibangun sebagai komponen terpisah dengan antarmuka REST sehingga dapat dipakai ulang atau diganti modelnya tanpa mengubah aplikasi Laravel.
- **REQ-OR-16:** Sistem harus menyediakan data awal (seeder) berupa akun Maintainer dan Reporter serta project contoh untuk kebutuhan pengujian dan demonstrasi.
- **REQ-OR-17:** Sistem harus dapat dijalankan pada lingkungan lokal tanpa mewajibkan Docker, Redis, GPU, atau layanan cloud berbayar, sebagaimana batasan pada Bab 2.5.

Appendix A: Glossary

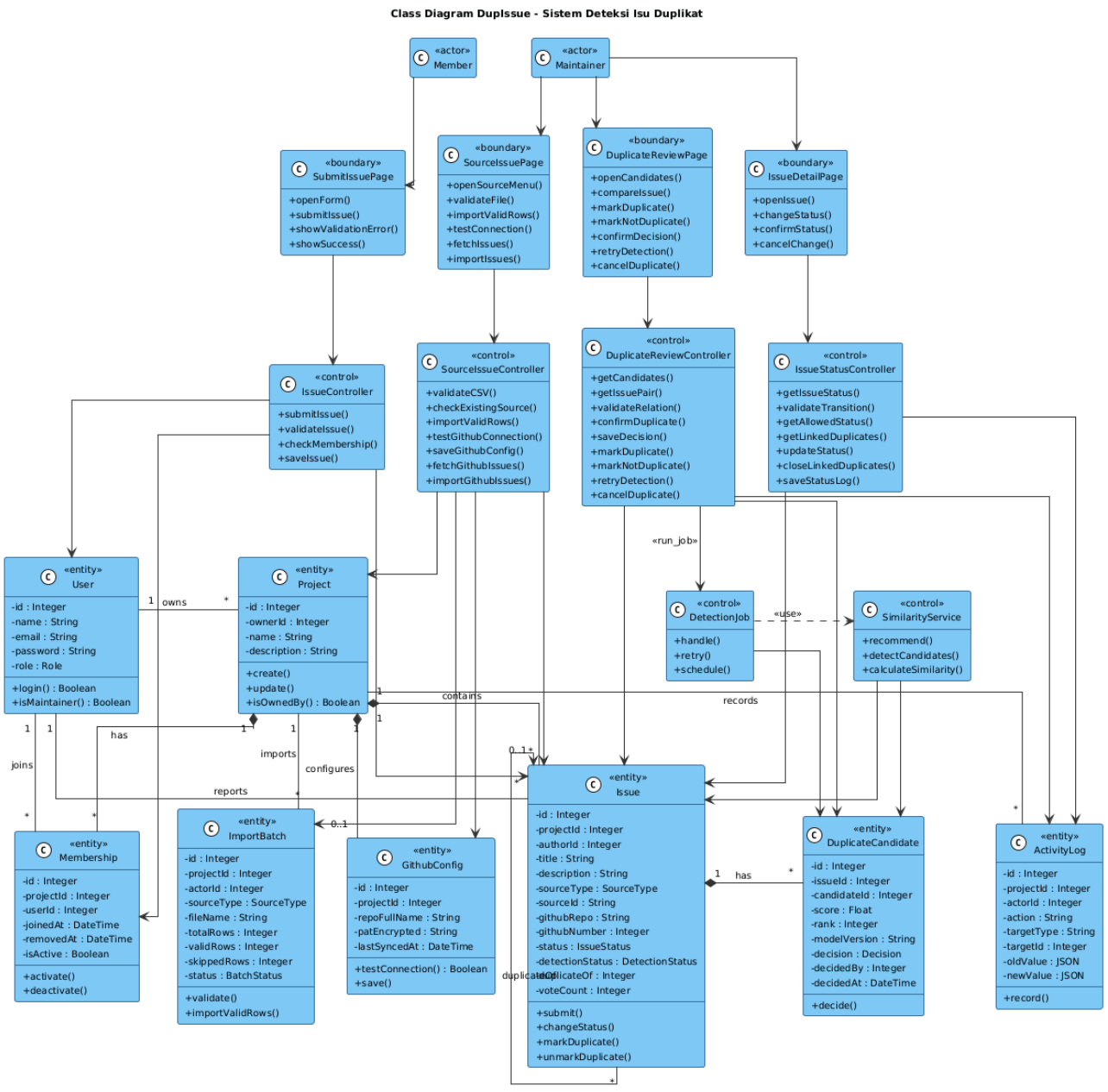
- **Activity Diagram**: Diagram UML yang menggambarkan alur aktivitas dan keputusan dalam suatu proses.
- **Asynchronous**: Pemrosesan yang berjalan di belakang layar sehingga pengguna tidak perlu menunggu hasilnya untuk melanjutkan.
- **Auto-close**: Penutupan otomatis seluruh issue duplikat tertaut ketika issue induknya berstatus Resolved.
- **Baseline**: Metode pembanding sederhana (di sini TF-IDF dengan cosine similarity) untuk menilai kualitas metode utama.
- **Bootstrap**: Kerangka kerja CSS untuk membangun antarmuka web yang responsif.
- **Bug / Issue Report**: Laporan masalah atau cacat pada perangkat lunak yang dibuat Reporter.
- **Cosine Similarity**: Ukuran kemiripan dua vektor berdasarkan sudut di antara keduanya, bernilai 0 sampai 1 untuk representasi teks non-negatif.
- **CDM / PDM**: Conceptual Data Model dan Physical Data Model, model data pada tingkat konseptual dan tingkat implementasi database.
- **CSRF (Cross-Site Request Forgery)**: Serangan yang memaksa pengguna terautentikasi mengirim permintaan tanpa disadari.
- **Dashboard**: Halaman ringkasan statistik issue dan kandidat duplikat bagi Maintainer.
- **Debounce**: Penundaan pemanggilan fungsi sampai pengguna berhenti mengetik, untuk mengurangi jumlah permintaan.
- **Detection Status**: Status proses deteksi NLP pada issue: PENDING, PROCESSING, COMPLETED, atau FAILED.
- **Duplicate Candidate (Kandidat Duplikat)**: Pasangan issue yang dinilai serupa oleh NLP dan menunggu keputusan Maintainer.
- **Duplicate Issue**: Issue yang melaporkan masalah yang sama dengan issue lain dan ditautkan ke issue induk.
- **Embedding**: Representasi teks dalam bentuk vektor angka yang menangkap makna semantik.
- **FastAPI**: Framework Python untuk membangun REST API, dipakai sebagai layanan NLP.
- **Gherkin**: Bahasa penulisan skenario berformat Given-When-Then yang dipakai pada user story scenario.
- **GitHub REST API**: Antarmuka GitHub untuk mengambil data seperti issue dari repository.
- **Human-in-the-loop**: Pendekatan di mana keputusan akhir tetap dilakukan manusia, bukan otomatis oleh sistem.

- **Induk (Parent) Issue**: Issue acuan yang dirujuk oleh satu atau lebih issue duplikat melalui duplicate_of.
- **Invitation**: Undangan Maintainer kepada Reporter untuk bergabung ke project, berstatus PENDING, ACCEPTED, REJECTED, atau CANCELED.
- **JSON**: Format pertukaran data berbasis teks yang ringan.
- **Laravel**: Framework PHP yang dipakai sebagai backend DupIssue.
- **Laravel Database Queue**: Mekanisme antrean pekerjaan Laravel yang menyimpan pekerjaan di tabel database dan diproses oleh queue worker.
- **Maintainer**: Aktor pemilik project yang mengelola anggota, sumber issue, validasi duplikat, dan status issue.
- **Membership**: Catatan keanggotaan pengguna pada project setelah undangan diterima.
- **MVP (Minimum Viable Product)**: Versi awal produk dengan fitur inti yang cukup untuk dievaluasi.
- **NLP (Natural Language Processing)**: Bidang yang memproses dan memahami bahasa alami oleh komputer.
- **PAT (Personal Access Token)**: Token autentikasi GitHub yang dipakai untuk mengakses repository.
- **Pull Request**: Usulan perubahan kode di GitHub, dikecualikan dari impor issue.
- **RBAC (Role-Based Access Control)**: Pengendalian akses berdasarkan peran pengguna.
- **Recall@k / MRR**: Metrik evaluasi pencarian: proporsi pasangan duplikat yang muncul pada k teratas, dan rata-rata kebalikan peringkat hasil benar pertama.
- **Reporter**: Aktor yang melaporkan bug atau issue dan memantau statusnya.
- **Semantic Similarity**: Kemiripan makna antar teks, bukan sekadar kesamaan kata.
- **Sentence Transformers**: Pustaka Python untuk menghasilkan embedding kalimat.
- **SRS (Software Requirements Specification)**: Dokumen spesifikasi kebutuhan perangkat lunak.
- **TF-IDF**: Pembobotan kata berdasarkan frekuensi dalam dokumen dan kelangkaannya di seluruh koleksi.
- **Triase (Triage)**: Proses memeriksa, memilah, dan menentukan prioritas atau penanganan laporan yang masuk.
- **Use Case**: Deskripsi interaksi aktor dengan sistem untuk mencapai suatu tujuan.
- **User Story**: Deskripsi kebutuhan singkat dari sudut pandang pengguna dengan format As a, I want, so that.
- **Vote**: Dukungan Reporter pada issue yang berarti ia mengalami masalah yang sama.

Appendix B: Analysis Models

B.1 Class Diagram

Class diagram berikut disusun dari PDM (Laporan Tugas Week 6 hlm. 36) dan sequence diagram fitur. Kelas entitas mewakili tabel database, sedangkan kelas service mewakili logika yang dipanggil controller.



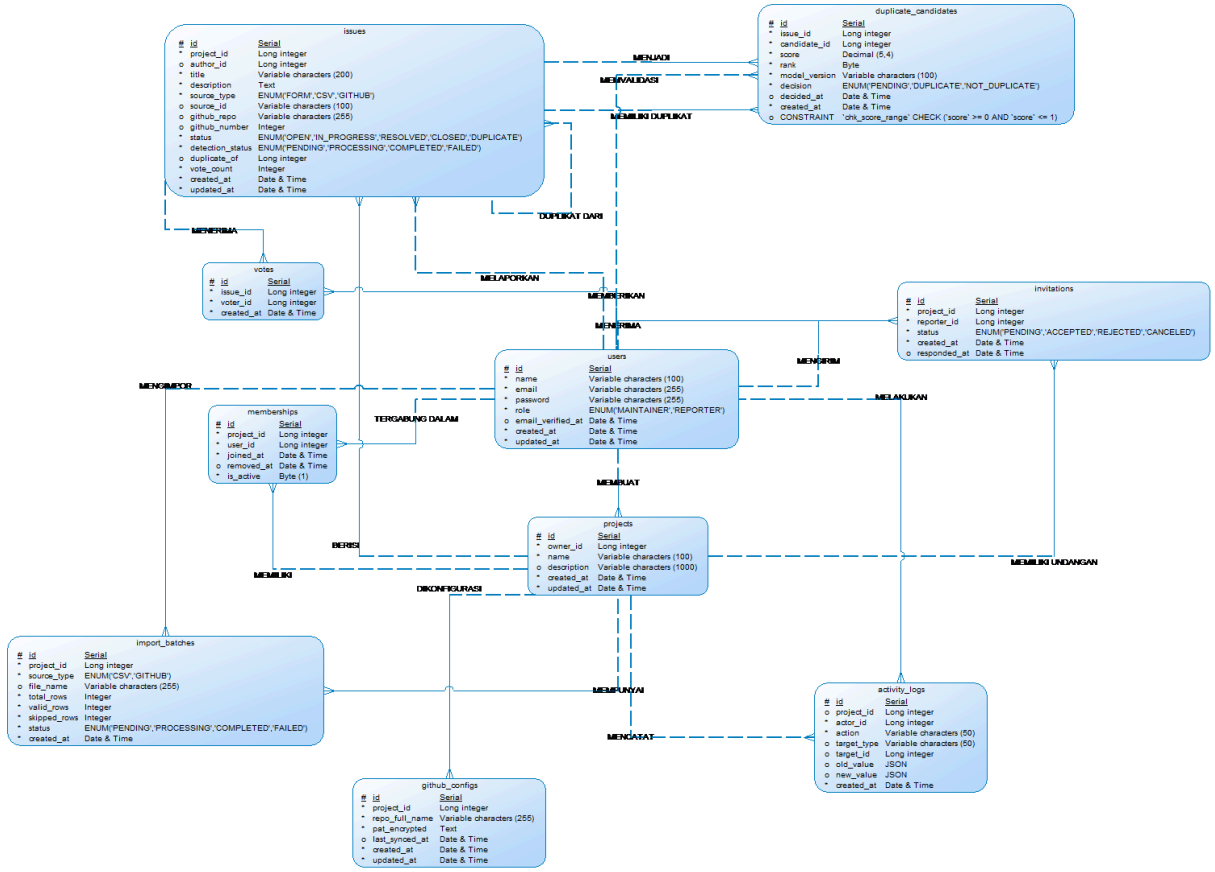
Gambar B.1 Class Diagram DupIssue

Penjelasan kelas utama:

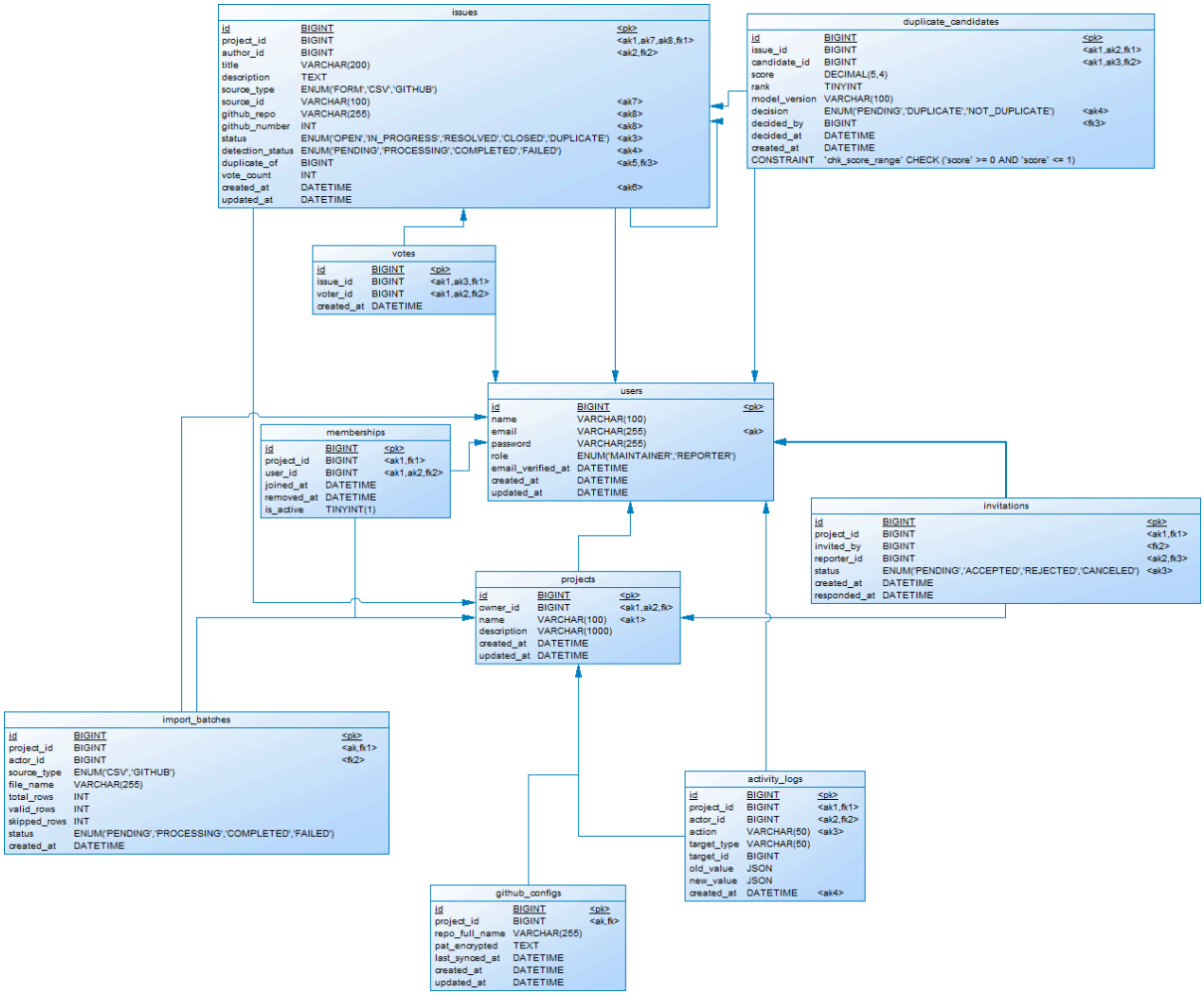
- **User**: akun pengguna dengan peran Reporter atau Maintainer. Method login memvalidasi kredensial dan isMaintainer dipakai untuk pemeriksaan RBAC.
- **Project**: wadah issue yang dimiliki satu Maintainer. Menjadi batas akses, impor, dan pencarian kemiripan.

- **Membership**: keanggotaan aktif Reporter pada project. activate dan deactivate mengatur is_active tanpa menghapus histori.
- **Invitation**: undangan Maintainer kepada Reporter dengan status PENDING, ACCEPTED, REJECTED, atau CANCELED.
- **Issue**: laporan dari form, CSV, atau GitHub. Memisahkan status penanganan (status) dari status deteksi (detectionStatus) dan menyimpan relasi induk melalui duplicateOf.
- **Vote**: dukungan Reporter pada issue, satu per kombinasi issue dan Reporter.
- **DuplicateCandidate**: pasangan issue dan kandidat beserta skor, peringkat, versi model, dan keputusan Maintainer.
- **GithubConfig** dan **ImportBatch**: konfigurasi repository GitHub per project dan catatan setiap kegiatan impor.
- **ActivityLog**: rekaman aktivitas project dengan nilai lama dan baru.
- **SimilarityService** dan **DetectionJob**: komponen logika yang memanggil NLP Service untuk rekomendasi pre-submit dan deteksi kandidat setelah issue tersimpan melalui antrean.

B.2 Database Design



Gambar B.2 Conceptual Data Model DupIssue



Gambar B.3 Physical Data Model DupIssue

CDM dan PDM menggambarkan sepuluh entitas beserta relasinya. Penjelasan lengkap atribut dan relasi (tabel entitas dan tabel relasi) dapat disalin dari Laporan Tugas Week 6 hlm. 37-43 dan dipakai tanpa perubahan. Ringkasan relasi utama:

- users membuat projects (1:N) dan memiliki memberships (1:N) melalui tabel penghubung dengan projects.
- users mengirim dan menerima invitations (1:N), dan projects memiliki invitations (1:N).
- projects berisi issues (1:N), dan users melaporkan issues (1:N, author_id boleh kosong untuk data impor).
- issues memiliki relasi rekursif duplicate_of (induk 1:N duplikat).
- issues menerima votes (1:N) dan users memberikan votes (1:N).

- issues memiliki duplicate_candidates melalui issue_id dan candidate_id, divalidasi oleh users melalui decided_by.
- projects dikonfigurasi github_configs (1:0..1), mempunyai import_batches (1:N), dan mencatat activity_logs (1:N).

Appendix C: To Be Determined List

Tabel C.1. Daftar TBD

ID	Referensi	Uraian yang Perlu Ditentukan	Usulan Sementara
TBD-01	REQ-01.3	Nilai jeda pengetikan (debounce) sebelum meminta rekomendasi	500 ms
TBD-02	REQ-06.2, Tabel 4.38	Matriks transisi status penanganan issue yang final (misalnya apakah OPEN dapat langsung ke RESOLVED, atau IN_PROGRESS kembali ke OPEN)	Sesuai Tabel 4.38
TBD-03	REQ-03.2	Cakupan keunikan nama project: global atau per Maintainer. Skenario "Nama sudah digunakan" belum menyebut cakupannya	Global
TBD-04	REQ-07.5	Definisi dan cara menghitung metrik "waktu triase" pada dashboard (rata-rata dari issue masuk sampai keputusan duplikat atau perubahan status pertama)	Dihitung dari created_at sampai keputusan pertama
TBD-05	REQ-04.7	Apakah CSV boleh memuat kolom opsional status. Low-fi menampilkan kolom status, sedangkan Bab 2.5 hanya menyebut tiga kolom wajib	Kolom status opsional, nilai di luar enum dilewati
TBD-06	Bab 5.4	Ambang skor kemiripan minimum untuk menampilkan rekomendasi atau kandidat, dan jumlah maksimum kandidat per issue (rekomendasi pre-submit sudah ditetapkan maksimal 3)	Ditentukan dari hasil evaluasi NLP

TBD-07	BR-01, Bab 2.3	Bab 2.3 menyebut satu project dapat memiliki beberapa Maintainer sebagai anggota, sedangkan use case, PDM, dan business rule hanya mengenal satu Maintainer pemilik. Perlu disepakati dengan penulis Bab 2	Satu pemilik pada MVP
TBD-08	REQ-NF-06, Bab 5.4	Target jumlah pengguna bersamaan dan target ketersediaan pada periode pengujian	20 pengguna, ketersediaan 95%
TBD-09	REQ-01.15	Apakah Reporter boleh memberi vote pada issue berstatus RESOLVED atau CLOSED	Boleh pada status non-duplikat
TBD-10	REQ-07.8	Bentuk pemberitahuan dalam aplikasi (badge atau daftar notifikasi)	Badge pada menu Undangan dan status deteksi

Tabel C.2. Kontribusi Anggota Kelompok

Foto	Nama (NIM)	Kontribusi
	Joshua Charis Giovanni Mangialu (187241023)	- Memimpin dan mengawasi progres proyek - Menyusun use case diagram, use case specification, user story dan scenario, rancangan CDM dan PDM, sequence diagram - Menyusun Laporan SRS

	Valentino Kevin Iversen (187241041)	- Menyusun use case diagram, use case specification, user story dan scenario, rancangan CDM dan PDM, sequence diagram - Menyusun Laporan SRS
	Elsa Alfika Dyah K. P. (187241007)	- Membuat mockup desain Lo-Fi dan Hi-Fi, menyusun user story dan scenario, activity diagram - Menyusun Laporan SRS
	Fakhri Reza Abinaya (187241093)	- Membuat mockup desain Lo-Fi, menyusun user story dan scenario, activity diagram, class diagram - Menyusun Laporan SRS